

Aigis-Enc⁺

Algorithm Specifications and Supporting Documentation (version 1.1)

Jiang Zhang¹, Yu Yu^{2,5}, Shuqin Fan¹, Kang Yang¹, Debiao He³, Runqing Xu³,
Jiangxia Ge⁴, Yuanmi Chen⁵, Weijia Wang⁶, Cong Peng³, Di Yan¹, Hui Guo¹,
Hui Zhu⁷, Ruikang Liu⁷, and Jing Pan⁷

¹ State Key Laboratory of Cryptology, Beijing, China

² Shanghai Jiao Tong University, Shanghai, China

³ Wuhan University, Wuhan, China

⁴ China Telecom Quantum Group Ltd., Hefei, China

⁵ East China Normal University, Shanghai, China

⁶ Shandong University, Qingdao, China

⁷ Xidian University, Xi'an, China

June 28, 2026

Table of Contents

1	Introduction	1
1.1	Feature Statements	1
1.2	Overview of Design Approach	2
2	Preliminaries and Syntax	4
2.1	Notations	4
2.2	Syntax of Public-Key Encryption	5
2.3	Syntax of Key Encapsulation Mechanism	6
2.4	Hardness Assumptions	7
3	Specification of Aigis-Enc⁺ Encryption	8
3.1	Plaintext Encoding and Decoding	8
3.2	Description of Aigis-PKE⁺	10
3.3	Description of Aigis-Enc⁺	12
4	Recommended Parameters	13
5	Implementation Details	14
5.1	Partial NTT Multiplication	14
5.2	Symmetric Primitives	16
5.3	Encode and Decode	16
5.4	Sampling Algorithms	19
5.5	The Aigis-PKE⁺ Scheme	20
5.6	The Aigis-Enc⁺ Scheme	22
6	Performance and Comparison	23
6.1	Performance on x86	24
6.2	Performance on ARM	25
7	Provable Security	27
7.1	IND-CPA Security Proof of Aigis-PKE⁺	27
7.2	IND-CCA Security Proof of Aigis-Enc⁺	29
8	Known Attacks	30
8.1	Attacks against ALWE	30
8.2	Concrete Security Estimates against Lattice Attacks	36
8.3	Attacks Exploiting Decryption Failures	37
8.4	Attacks against Symmetric Primitives	39
9	Side-Channel Vulnerabilities and Countermeasures	40
9.1	Security Analysis	40
9.2	A Discussion on Masking Countermeasures	41
A	Appendix	47
A.1	Proof of Theorem 2	47
A.2	Proof of Theorem 3	52

1 Introduction

1.1 Feature Statements

This document defines a key encapsulation mechanism (KEM) **Aigis-Enc⁺**, which is an improved version of the KEM scheme **Aigis-Enc** based on the asymmetric MLWE problems in [46]. The scheme was designed for being easy to securely implement, having very competitive overall performance, and supporting flexible choices of parameters. The **Aigis-Enc⁺** scheme has the following advantages:

- **Flexible and provably secure design:** **Aigis-Enc⁺** inherits the asymmetric design philosophy of **Aigis-Enc** [46] and improves efficiency via 4-dimensional lattice plaintext encoding. To this end, we depart from the original modular structure and instead utilize distinct rings that share the same NTT-friendly subrings for each security level, thereby accommodating various plaintext lengths. Rigorous security proofs are established under the asymmetric ring learning with errors (ARLWE) assumption, in both the random oracle model (ROM) and the quantum random oracle model (QROM).
- **No independence assumptions:** Our decryption failure analysis is grounded in a numeric worst-case union bound, deliberately avoiding any reliance on the assumption of independent coefficient failures. While such independence assumptions are common in schemes that employ complex error-correcting codes for analytical simplicity, they have been rigorously invalidated both theoretically and practically [26]. By discarding this problematic assumption, our method provides a provably sound upper bound on the decryption failure probability, thereby ensuring more robust parameter selection and enhancing the overall reliability of the scheme under real-world conditions.
- **Easy to implement securely:** To minimize side-channel attack surfaces, **Aigis-Enc⁺** is deliberately designed to avoid complex operations that are notoriously difficult to protect—such as higher-order error-correcting codes or floating-/fixed-point arithmetic. Instead, all parameter sets share a unified, lightweight foundation: a small modulus $q = 3329$ and a fixed dimension-128 NTT. This uniformity not only reduces algorithmic complexity but also facilitates more predictable and verifiable masking.
- **Conservative security estimates:** Our concrete security analysis adopts a conservative approach, relying primarily on the Core-SVP model introduced in [5]—a standard and widely referenced methodology for estimating the hardness of lattice problems. However, recognizing recent progress in hybrid attacks on LWE, as discussed in [18], we also account for the MATZOV algorithm [38], which represents a significant improvement in attacks against certain LWE parameter regimes. By including this algorithm in our estimates, we ensure that our security claims remain valid under the latest cryptanalytic understanding, particularly for long-term security requirements.
- **Competitive performance:** Even under several key conservative design principles—such as protection against side-channel leakage and the use of conservative security parameters mentioned above—NEV achieves highly competitive performance. The public key and ciphertext can be respectively stored in 698

bytes and 896 for 128-bit security. This is achieved without using (R)LWR-like assumptions or sparse binary/ternary LWE secrets that is more fragile to dual-hybrid LWE attacks. The total time for key generation, encryption and decryption for 128-bit classical security is less than 0.10 ms for reference implementation and 0.02 ms for AVX optimizations.

1.2 Overview of Design Approach

For simplicity and clarity, we explain the core idea using the LWE-based public-key encryption (PKE) scheme as an example. Note that most LWE-based PKE schemes mainly follow the framework in [37] up to the choices of parameters and noise distributions. Let $n, q \in \mathbb{Z}$ be positive integers, and let $\chi_\alpha \subset \mathbb{Z}$ be a discrete Gaussian distribution with standard variance $\alpha \in \mathbb{R}$. The LWE-based PKE works as follows:

- **Key generation:** randomly choose $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times n}$, $\mathbf{s}, \mathbf{e} \xleftarrow{\$} \chi_\alpha^n$ and compute $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$. Return the public key $pk = (\mathbf{A}, \mathbf{b})$ and secret key $sk = \mathbf{s}$.
- **Encryption:** given the public key $pk = (\mathbf{A}, \mathbf{b})$ and a plaintext $\mu \in \{0, 1\}$, randomly choose $\mathbf{r}, \mathbf{x}_1 \xleftarrow{\$} \chi_\alpha^n, x_2 \xleftarrow{\$} \chi_\alpha$ and compute $\mathbf{c}_1 = \mathbf{A}^T \mathbf{r} + \mathbf{x}_1$, $c_2 = \mathbf{b}^T \mathbf{r} + x_2 + \mu \cdot \lceil \frac{q}{2} \rceil$. Finally, return the ciphertext $C = (\mathbf{c}_1, c_2)$.
- **Decryption:** given the secret key $sk = \mathbf{s}$ and a ciphertext $C = (\mathbf{c}_1, c_2)$, compute $z = c_2 - \mathbf{s}^T \mathbf{c}_1$ and output $\lceil z \cdot \frac{2}{q} \rceil \bmod 2$ as the decryption result.

For a honestly generated ciphertext $C = (\mathbf{c}_1, c_2)$ that encrypts plaintext $\mu \in \{0, 1\}$, we have:

$$z = c_2 - \mathbf{s}^T \mathbf{c}_1 = \mu \cdot \left\lceil \frac{q}{2} \right\rceil + \underbrace{\mathbf{e}^T \mathbf{r} - \mathbf{s}^T \mathbf{x}_1}_{\text{noise } e'} + x_2. \quad (1)$$

Thus, the decryption algorithm is correct as long as $|e'| < \frac{q}{4}$. Since $|x_2| \ll |\mathbf{e}^T \mathbf{r} - \mathbf{s}^T \mathbf{x}_1|$, the magnitude of $|e'|$ mainly depends on $|\mathbf{e}^T \mathbf{r} - \mathbf{s}^T \mathbf{x}_1|$. That is, the LWE secret $(\mathbf{s}, \mathbf{r})$ and the noise $(\mathbf{e}, \mathbf{x}_1)$ contribute almost equally to the magnitude of $|e'|$. Moreover, for a fixed n the expected magnitude of $|\mathbf{e}^T \mathbf{r} - \mathbf{s}^T \mathbf{x}_1|$ is a monotonically increasing function of α :

$$\text{larger } \alpha \Rightarrow \text{larger } |\mathbf{e}^T \mathbf{r} - \mathbf{s}^T \mathbf{x}_1| \Rightarrow \text{larger } |e'|.$$

Let ν be the probability that the decryption algorithm fails, and let λ be the complexity of solving the underlying LWE problem. Ideally, for a targeted security strength κ , we hope that $\nu = 2^{-\kappa}$ and $\lambda = 2^\kappa$, since a large ν (i.e., $\nu > 2^{-\kappa}$) will sacrifice the overall security, and a large λ (i.e., $\lambda > 2^\kappa$) may compromise the overall performance. Since both ν and λ are strongly related to the ratio α/q of the Gaussian parameter α and the modulus q , it is hard to come up with an appropriate choice of (α, q) to simultaneously achieve the best of the two worlds.

To obtain smaller public keys and ciphertexts (and thus improve the communication efficiency), many schemes use the modulus switching technique [17,15] to compress public keys and ciphertexts. We refer to the following scheme that adopts modulus switching technique to compress public keys and ciphertexts, where $p_1, p_2, p_3 \in \mathbb{Z}$ are parameters for compression (p_1 for the public key and p_2, p_3 for ciphertexts).

- **Key generation:** pick $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times n}$ and $\mathbf{s}, \mathbf{e} \xleftarrow{\$} \chi_\alpha^n$ and compute $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$. Then, return the public key $pk = (\mathbf{A}, \bar{\mathbf{b}} = \lceil \mathbf{b} \rceil_{q \rightarrow p_1})$ and the secret key $sk = \mathbf{s}$.
- **Encryption:** given the public key $pk = (\mathbf{A}, \bar{\mathbf{b}})$ and a plaintext $\mu \in \{0, 1\}$, randomly choose $\mathbf{r}, \mathbf{x}_1 \xleftarrow{\$} \chi_\alpha^n, x_2 \xleftarrow{\$} \chi_\alpha$, and compute $\mathbf{c}_1 = \mathbf{A}^T \mathbf{r} + \mathbf{x}_1$ and $c_2 = \lceil \bar{\mathbf{b}} \rceil_{p_1 \rightarrow q}^T \mathbf{r} + x_2 + \mu \cdot \lceil \frac{q}{2} \rceil$. Return the ciphertext $C = (\bar{\mathbf{c}}_1 = \lceil \mathbf{c}_1 \rceil_{q \rightarrow p_2}, \bar{c}_2 = \lceil c_2 \rceil_{q \rightarrow p_3})$.
- **Decryption:** given the secret key $sk = \mathbf{s}$ and a ciphertext $C = (\bar{\mathbf{c}}_1, \bar{c}_2)$, compute $z = \lceil \bar{c}_2 \rceil_{p_3 \rightarrow q} - \mathbf{s}^T \lceil \bar{\mathbf{c}}_1 \rceil_{p_2 \rightarrow q}$ and output $\lceil z \rceil_{q \rightarrow 2} = \lceil z \cdot \frac{2}{q} \rceil \bmod 2$ as the decryption result.

Let

$$\bar{\mathbf{e}} = \lceil \lceil \mathbf{b} \rceil_{q \rightarrow p_1} \rceil_{p_1 \rightarrow q} - \mathbf{b}, \bar{\mathbf{x}}_1 = \lceil \lceil \mathbf{c}_1 \rceil_{q \rightarrow p_2} \rceil_{p_2 \rightarrow q} - \mathbf{c}_1, \bar{x}_2 = \lceil \lceil c_2 \rceil_{q \rightarrow p_3} \rceil_{p_3 \rightarrow q} - c_2.$$

It is easy to verify $\|\bar{\mathbf{e}}\|_\infty \leq \frac{q}{2p_1}, \|\bar{\mathbf{x}}_1\|_\infty \leq \frac{q}{2p_2}$, and $|\bar{x}_2| \leq \frac{q}{2p_3}$. For any valid ciphertext $C = (\bar{\mathbf{c}}_1, \bar{c}_2)$ that encrypts $\mu \in \{0, 1\}$ we have

$$\begin{aligned} z &= \lceil \bar{c}_2 \rceil_{p_3 \rightarrow q} - \mathbf{s}^T \lceil \bar{\mathbf{c}}_1 \rceil_{p_2 \rightarrow q} \\ &= \mu \cdot \lceil \frac{q}{2} \rceil + \underbrace{(\mathbf{e} + \bar{\mathbf{e}})^T \mathbf{r} - \mathbf{s}^T (\mathbf{x}_1 + \bar{\mathbf{x}}_1) + (x_2 + \bar{x}_2)}_{\text{noise } e'} \end{aligned} \quad (2)$$

Apparently, the smaller values for p_1, p_2, p_3 the better compression rate is achieved for public keys and ciphertexts. At the same time, however, by the definitions of $\bar{\mathbf{e}}, \bar{\mathbf{x}}_1, \bar{x}_2$ we know that smaller p_1, p_2, p_3 also result in a larger noise e' . Notice that when p_1, p_2, p_3 are much smaller than q , we will have $\|\bar{\mathbf{e}}\|_\infty \gg \|\mathbf{e}\|_\infty$, $\|\bar{\mathbf{x}}_1\|_\infty \gg \|\mathbf{x}_1\|_\infty$ and $|\bar{x}_2| \gg |x_2|$, which further leads to asymmetric roles of $(\mathbf{e}, \mathbf{x}_1, x_2)$ and $(\mathbf{s}, \mathbf{r})$ in contributing to the resulting size of $|e'|$, i.e., for specific (p_1, p_2, p_3) decreasing (resp., increasing) $\|\mathbf{s}\|_\infty$ or $\|\mathbf{r}\|_\infty$ would significantly reducing (resp., enlarging) the noise $|e'|$, and in contrast, changing the size of $\|\mathbf{e}\|_\infty, \|\mathbf{x}_1\|_\infty$ and $|x_2|$ would not result in substantial change to $|e'|$.

The asymmetry observed above motivates the design of our ALWE-based PKE, which uses different noise distributions χ_{α_1} and χ_{α_2} (i.e., same distribution with different parameters α_1 and α_2) for the secrets (i.e., $\mathbf{s}$ and $\mathbf{r}$) and the errors (i.e., $\mathbf{e}, \mathbf{x}_1, x_2$), respectively.

- **Key generation:** pick $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times n}, \mathbf{s} \xleftarrow{\$} \chi_{\alpha_1}^n$ and $\mathbf{e} \xleftarrow{\$} \chi_{\alpha_2}^n$, compute $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$. Then, return the public key $pk = (\mathbf{A}, \bar{\mathbf{b}} = \lceil \mathbf{b} \rceil_{q \rightarrow p_1})$ and the secret key $sk = \mathbf{s}$.
- **Encryption:** given the public key $pk = (\mathbf{A}, \bar{\mathbf{b}})$ and a plaintext $\mu \in \{0, 1\}$, randomly choose $\mathbf{r} \xleftarrow{\$} \chi_{\alpha_1}^n, \mathbf{x}_1 \xleftarrow{\$} \chi_{\alpha_2}^n, x_2 \xleftarrow{\$} \chi_{\alpha_2}$, compute $\mathbf{c}_1 = \mathbf{A}^T \mathbf{r} + \mathbf{x}_1$ and $c_2 = \lceil \bar{\mathbf{b}} \rceil_{p_1 \rightarrow q}^T \mathbf{r} + x_2 + \mu \cdot \lceil \frac{q}{2} \rceil$, and return the ciphertext $C = (\bar{\mathbf{c}}_1 = \lceil \mathbf{c}_1 \rceil_{q \rightarrow p_2}$ and $\bar{c}_2 = \lceil c_2 \rceil_{q \rightarrow p_3})$.
- **Decryption:** Given the secret key $sk = \mathbf{s}$ and the ciphertext $C = (\bar{\mathbf{c}}_1, \bar{c}_2)$, compute $z = \lceil \bar{c}_2 \rceil_{p_3 \rightarrow q} - \mathbf{s}^T \lceil \bar{\mathbf{c}}_1 \rceil_{p_2 \rightarrow q}$ and output $\lceil z \rceil_{q \rightarrow 2} = \lceil z \cdot \frac{2}{q} \rceil \bmod 2$ as the decryption result.

Similarly, for ciphertext $C = (\bar{\mathbf{c}}_1, \bar{c}_2)$ we have the same z and e' as defined in (2), where the difference is that now $\|\mathbf{s}\|_\infty$ and $\|\mathbf{r}\|_\infty$ are determined by α_1 , and that

$\|\mathbf{e}\|_\infty$, $\|\mathbf{x}_1\|_\infty$ and $|x_2|$ are determined by α_2 . Intuitively, we wish to use small α_1 in order to keep $|e'|$ small, and at the same time choose relatively large α_2 to remedy the potential security loss due to the choice of a small α_1 .

While the intuition seems reasonable, it does not shed light on the choices of parameters, in particular, how parameters α_1 and α_2 (jointly) affect security. To this end, we consider the best known attacks and their variants against (A)LWE problems, and obtain the following conclusions: Let χ_{α_1} and χ_{α_2} be subgaussians with standard variances $\alpha_1, \alpha_2 \in \mathbb{R}$ respectively, then we have the following approximate relation between the hardness of ALWE and LWE: the hardness of ALWE with subgaussian standard variances $\alpha_1, \alpha_2 \in \mathbb{R}$ is polynomially equivalent to the hardness of LWE with subgaussian standard variance $\sqrt{\alpha_1 \alpha_2}$. Clearly, the equivalence is trivial for $\alpha_1 = \alpha_2$. This confirms the feasibility of our idea: use a small α_1 to keep the probability ν of decryption failures small while pick a relatively larger α_2 remain the security of the resulting PKE scheme.

The above idea can be naturally generalized to the schemes based on the ring and module versions of LWE. Actually, we will use asymmetric RLWE for achieving a better trade-off between computational and communication costs.

2 Preliminaries and Syntax

2.1 Notations

Let n be a power of 2, and q a prime. We denote by R the ring $R = \mathbb{Z}[X]/(X^n + 1)$ and by R_q the ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. The regular font letters (e.g., a, b) represent elements in R or R_q (including elements in $\mathbb{Z}$ or $\mathbb{Z}_q$), and bold lower-case letters (e.g., $\mathbf{a}, \mathbf{b}$) denote vectors of R or $\mathbb{Z}$ elements. For a positive integer $\ell \in \mathbb{Z}$, by $[\ell]$ we denote the set $\{0, \dots, \ell - 1\}$. By $x \xleftarrow{\$} \mathcal{D}$ we denote sampling x according to a distribution $\mathcal{D}$ and by $\mathcal{U}(S)$ we denote the uniform distribution over a finite set S . When we write that sampling a polynomial $g \xleftarrow{\$} \mathcal{D}$ from a distribution $\mathcal{D}$ over $\mathbb{Z}$, we mean that sampling each coefficient of g from $\mathcal{D}$ individually. We use $\log_b$ to denote the logarithm function in base b (e.g., 2 or natural constant e) and $\log$ to represent $\log_e$. We say that a function $f : \mathbb{N} \rightarrow [0, 1]$ is *negligible*, if for every positive c and all sufficiently large κ it holds that $f(\kappa) < 1/\kappa^c$. We denote by $\text{negl} : \mathbb{N} \rightarrow [0, 1]$ an (unspecified) negligible function.

Binomial Distribution. The centered binomial distribution B_η with some positive $\eta \in \mathbb{Z}$ is defined as follows:

$$B_\eta = \left\{ \sum_{i=0}^{\eta-1} (a_i - b_i) : (a_0, \dots, a_{\eta-1}, b_0, \dots, b_{\eta-1}) \xleftarrow{\$} \{0, 1\}^{2\eta} \right\}$$

Modular reductions. For an even positive integer α , we define $r' = r \bmod^\pm \alpha$ as the unique element in the range $(-\frac{\alpha}{2}, \frac{\alpha}{2}]$ such that $r' = r \bmod \alpha$. For an odd positive integer α , we define $r' = r \bmod^\pm \alpha$ as the unique element in the range $[-\frac{\alpha-1}{2}, \frac{\alpha-1}{2}]$ such that $r' = r \bmod \alpha$. For any positive integer α , we define $r' =$

$r \bmod^+ \alpha$ as the unique element in the range $[0, \alpha)$ such that $r' = r \bmod \alpha$. When the exact representation is not important, we simply write $r \bmod \alpha$.

Sizes of elements. For an element $w \in \mathbb{Z}_q$, we write $\|w\|_\infty$ to mean $|w \bmod^\pm q|$. The ℓ_∞ and ℓ_2 norms of a ring element $w = w_0 + w_1X + \dots + w_{n-1}X^{n-1} \in R$ are defined as follows:

$$\|w\|_\infty = \max_i \|w_i\|_\infty, \quad \|w\| = \sqrt{\|w_0\|_\infty^2 + \dots + \|w_{n-1}\|_\infty^2}.$$

Similarly, for $\mathbf{w} = (w_1, \dots, w_k) \in R^k$, we define

$$\|\mathbf{w}\|_\infty = \max_i \|w_i\|_\infty, \quad \|\mathbf{w}\| = \sqrt{\|w_1\|^2 + \dots + \|w_k\|^2}.$$

Modulus switching. For any positive integers p, q , we define the modulus switching function $\lceil \cdot \rceil_{q \rightarrow p}$ as:

$$\lceil x \rceil_{q \rightarrow p} = \lceil (p/q) \cdot x \rceil \bmod^+ p.$$

It is easy to show that for any $x \in \mathbb{Z}_q$ and $p < q \in \mathbb{N}$, $x' = \lceil \lceil x \rceil_{q \rightarrow p} \rceil_{p \rightarrow q}$ is an element close to x , i.e.,

$$|x' - x \bmod^\pm q| \leq \left\lceil \frac{q}{2p} \right\rceil.$$

When $\lceil \cdot \rceil_{q \rightarrow p}$ is used to a ring element $x \in R_q$ or a vector $\mathbf{x} \in R_q^k$, the procedure is applied to each coefficient individually.

2.2 Syntax of Public-Key Encryption

A public-key encryption (PKE) Π_{PKE} with plaintext space $\mathcal{M}$ consists of three PPT algorithms (**KeyGen**, **Enc**, **Dec**):

- **KeyGen**(1^κ): given a security parameter κ as input, output a pair of public and secret keys (pk, sk) , denoted as $(pk, sk) = \text{KeyGen}(1^\kappa)$.
- **Enc**($pk, M; r$): given the public key pk , a plaintext $M \in \mathcal{M}$ and a randomness r (which might be an empty string) as inputs, output a ciphertext C , denoted as $C = \text{Enc}(pk, M; r)$ or $C = \text{Enc}(pk, M)$ in brief.
- **Dec**(sk, C): given the secret key sk and a ciphertext C as inputs, output a plaintext M' (which might be $\perp$), denoted as $M' = \text{Dec}(sk, C)$.

For our interest, we recall the IND-CPA and IND-CCA security for PKEs, which is modeled by games between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ in Fig. 1.

Definition 1 (δ -correct [25]). We say that a PKE scheme Π_{PKE} with message space $\mathcal{M}$ and randomness space $\mathcal{R}$ is δ -correct or has decryption failure rate δ if

$$\mathbb{E} \left[\max_{m \in \mathcal{M}} \Pr [\text{Dec}(sk, c) \neq m : c = \text{Enc}(pk, m; r), r \leftarrow_{\$} \mathcal{R}] \right] \leq \delta.$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}$.

Definition 2 (γ -spread [25]). We say that a PKE scheme Π_{PKE} with message space $\mathcal{M}$, ciphertext space $\mathcal{C}$, and randomness space $\mathcal{R}$ is (weakly) γ -spread if

$$\mathbb{E} \left[\max_{m \in \mathcal{M}, c \in \mathcal{C}} \Pr[c = \text{Enc}(pk, m; r) : r \leftarrow_{\S} \mathcal{R}] \right] \leq 2^{-\gamma},$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}$.

IND-CPA/CCA Games:	Oracle $\mathcal{O}_{\text{Dec}}(C)$:
1 $(pk, sk) = \Pi_{\text{PKE}}.\text{KeyGen}(1^\kappa)$; 2 $(M_0, M_1, st) = \mathcal{A}_1^{\mathcal{O}(\cdot)}(pk)$; 3 $\mu \xleftarrow{\S} \{0, 1\}$; 4 $C^* = \Pi_{\text{PKE}}.\text{Enc}(pk, M_\mu)$; 5 $\mu' = \mathcal{A}_2^{\mathcal{O}(\cdot)}(C^*, st)$; 6 return $\mu' = \mu^*$;	1 if $C = C^*$ then 2 return $\perp$; 3 end 4 $M = \Pi_{\text{PKE}}.\text{Dec}(sk, C)$; 5 return M;

Fig. 1: Security Games for PKEs

Definition 3 (IND-CPA PKE). We say that a PKE scheme Π_{PKE} is IND-CPA secure if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, its advantage

$$\text{Adv}_{\Pi_{\text{PKE}}, \mathcal{A}}^{\text{ind-cpa}}(\kappa) = \left| \Pr[\mu' = \mu^*] - \frac{1}{2} \right|$$

in the IND-CPA security game with $\mathcal{O}(\cdot) = \perp$ in Fig. 1 is negligible in security parameter κ .

Definition 4 (IND-CCA PKE). We say that a PKE scheme Π_{PKE} is IND-CCA secure if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, its advantage

$$\text{Adv}_{\Pi_{\text{PKE}}, \mathcal{A}}^{\text{ind-cca}}(\kappa) = \left| \Pr[\mu' = \mu^*] - \frac{1}{2} \right|$$

in the IND-CCA security game with $\mathcal{O}(\cdot) = \mathcal{O}_{\text{Dec}}(\cdot)$ in Fig. 1 is negligible in security parameter κ .

2.3 Syntax of Key Encapsulation Mechanism

A key encapsulation mechanism (KEM) Π_{KEM} with session key space $\mathcal{K}$ consists of three PPT algorithms (KeyGen, Encap, Decap):

- **KeyGen**(1^κ): given a security parameter κ as input, output a pair of public and secret keys (pk, sk) , denoted as $(pk, sk) = \text{KeyGen}(1^\kappa)$.
- **Encap**($pk; r$): given the public key pk , and a randomness r as inputs, output a ciphertext C and a session key $K \in \mathcal{K}$, denoted as $(C, K) = \text{Encap}(pk; r)$, or $(C, K) = \text{Encap}(pk)$ in brief.

KEM Security Games:	Oracle $\mathcal{O}_{\text{Dec}}(C)$:
<pre> 1 $(pk, sk) = \Pi_{\text{KEM}}.\text{KeyGen}(1^\kappa);$ 2 $\mu^* \xleftarrow{\\$} \{0, 1\};$ 3 $(C^*, K_0) = \Pi_{\text{KEM}}.\text{Encap}(pk);$ 4 $K_1 \xleftarrow{\\$} \mathcal{K};$ 5 $\mu' = \mathcal{A}^{\mathcal{O}(\cdot)}(pk, C^*, K_{\mu^*});$ 6 return $\mu' = \mu^*;$ </pre>	<pre> 1 if $C = C^*$ then 2 return $\perp;$ 3 end 4 $K = \text{Decap}(sk, C);$ 5 return $K;$ </pre>

Fig. 2: Game for IND-CCA Security of KEMs

- $\text{Decap}(sk, C)$: given a secret key sk and a ciphertext C as inputs, output a key K' (which might be a failure symbol $\perp$), denoted as $K' = \text{Decap}(sk, C)$.

We now recall the chosen-ciphertext security for KEMs from [22], which is modeled by the game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ in Fig. 2.

Definition 5 (IND-CPA KEM). *We say that a KEM scheme Π_{KEM} is IND-CPA secure if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, its advantage*

$$\text{Adv}_{\Pi_{\text{KEM}}, \mathcal{A}}^{\text{ind-cpa}}(\kappa) = \left| \Pr[\mu' = \mu^*] - \frac{1}{2} \right|$$

in the security game with $\mathcal{O}(\cdot) = \perp$ in Fig. 2 is negligible in security parameter κ .

Definition 6 (IND-CCA KEM). *We say that a KEM scheme Π_{KEM} is IND-CCA secure if for any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, its advantage*

$$\text{Adv}_{\Pi_{\text{KEM}}, \mathcal{A}}^{\text{ind-cca}}(\kappa) = \left| \Pr[\mu' = \mu^*] - \frac{1}{2} \right|$$

in the security game with $\mathcal{O}(\cdot) = \mathcal{O}_{\text{Dec}}(\cdot)$ in Fig. 2 is negligible in security parameter κ .

2.4 Hardness Assumptions

Let n, q be positive integers with n being a powers of 2. Let $R = \mathbb{Z}[X]/(X^n + 1)$ (resp., $R_q = \mathbb{Z}_q[X]/(X^n + 1)$) be integer polynomials of degree less than n .

2.4.1 ARLWE Problem (with Binomial Distributions). Let α_1, α_2 be positive integers, and $\chi_{\alpha_1}, \chi_{\alpha_2}$ be two distribution over R . The decisional RLWE problem $\text{ARLWE}_{n,q,k,\ell,\alpha_1,\alpha_2}$ asks to distinguish $(a, b = as + e)$ and uniform over $R_q \times R_q$, where $a \xleftarrow{\$} R_q, s \xleftarrow{\$} \chi_{\alpha_1}, e \xleftarrow{\$} \chi_{\alpha_2}$. Obviously, when $\alpha_1 = \alpha_2$, the ARLWE problem is the standard RLWE problem. Given the current state of know attacks against (R)LWE, we have the following hardness relations:

$$\text{RLWE}_{n,q,\min(\alpha_1,\alpha_2)} \leq \text{ARLWE}_{n,q,\alpha_1,\alpha_2} \leq \text{RLWE}_{n,q,\max(\alpha_1,\alpha_2)}.$$

For our purpose, we are particularly interested in the problem $\text{ARLWE}_{n,q,\eta_1,\eta_2}$ with binomial distributions $s \xleftarrow{\$} B_{\eta_1}, e \xleftarrow{\$} B_{\eta_2}$, for which the above inequalities indeed hold in theory [35]. The ARLWE assumption says that it is hard for any (quantum) PPT algorithms to solve the decisional $\text{ARLWE}_{n,q,\eta_1,\eta_2}$ problem with non-negligible advantage over a random guess.

2.4.2 ARLWE Problem with Semi-uniform Seeds. In the standard (R)LWE problem, the public seed matrix $\mathbf{A}$ (or a for RLWE) is typically required to be uniformly random. The (M)LWE problem with non-uniform seeds had been formally studied in [34], where the authors showed that under the standard (M)LWE assumption, the (M)LWE Problem can be still hard for $\mathbf{A}$ is chosen from some distribution $\mathcal{D}$ that is not (even statistically close to) the uniform distribution over $\mathbb{Z}_q^{m \times n}$. Specifically, they showed that if $\mathcal{D}$ is a semi-uniform distribution over $\mathbb{Z}_q^{m \times n}$ (namely, $\mathcal{D}$ can be publicly derived from and has “small difference” to the uniform distribution over $\mathbb{Z}_q^{m \times n}$), then for appropriate choices of parameter, the (M)LWE with seed distribution D is hard under the standard LWE assumption with uniform seeds. A prime practical example of semi-uniform distribution is the rounded uniform distribution, which appears in real-world cryptographic schemes (such as the round-1 Kyber [8]) that compress public keys for efficiency. In order to compress the public key, we give the formal definition of the decisional *semi*- $\text{ARLWE}_{n,q,p,\eta_1,\eta_2}$ problem, which asks to distinguish $(a, b = [u]_{q \rightarrow p}, as + e_1, [b]_{p \rightarrow q}s + e_2)$ from $(a, b = [u]_{q \rightarrow p}, v, w)$, where $a, u, v, w \xleftarrow{\$} R_q, s \xleftarrow{\$} \chi_{\alpha_1}, e_1, e_2 \xleftarrow{\$} \chi_{\alpha_2}$. The semi-ARLWE assumptions says that it is hard for any (quantum) PPT algorithms to solve the decisional *semi*- $\text{ARLWE}_{n,q,p,\eta_1,\eta_2}$ problem with non-negligible advantage over a random guess.

3 Specification of Aigis-Enc⁺ Encryption

Our Aigis-Enc⁺ scheme bears most resemblance to the KEM scheme in [6,37]. The main difference is that our scheme uses the asymmetric LWE problems, which is in a different parameter setting method. To simplify the description, we first give the Aigis-PKE⁺ scheme with IND-CPA security as the base.

3.1 Plaintext Encoding and Decoding

Our way of encoding and decoding plaintext is inspired by the method for RLWE-based encryption in [42], which essentially encodes a single plaintext bit into multiple coefficients of a ring element, and is also used in Newhope [6,4]. Formally, let n be a power of 2, and q be a prime. Let $R = \mathbb{Z}[x]/(x^n + 1)$ and $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Let $\mathcal{M} = \{0, 1\}^\ell$ be the plaintext space. Let k be the largest integer satisfying $k|n$ and $n/k \geq \ell$. Let $f = (1 - x^{n/k}) \in R_q^*$ be a ring element, whose inverse is $f^{-1} = \frac{q+1}{2}(1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q^*$. We define the following two algorithms Pt2poly and Poly2Pt for encoding and decoding:

- **Pt2poly**(M) : given a plaintext $M \in \{0, 1\}^\ell$ as input, return a polynomial $m = M_0 + M_1x + \dots + M_{\ell-1}x^{\ell-1} \in R_q$, where $M_i \in \{0, 1\}$ is the i -th bit of M , denoted as $m = \text{Pt2poly}(M)$.

- **Poly2Pt**(w) : given a polynomial $w = w_0 + w_1x + \dots + w_{n-1}x^{n-1} \in R_q$ as input, first compute $\tilde{w}_i = w_i - \frac{q+1}{2} \bmod^\pm q$ for all $i \in [n]$. Then, compute $t_j = \sum_{i=j \bmod n/k} |\tilde{w}_i|$ for all $j \in [\ell]$. Finally, set

$$M_j = \begin{cases} 1, & \text{if } t_j < \frac{k \cdot (q-1)}{4}; \\ 0, & \text{otherwise,} \end{cases}$$

and return the plaintext $M = (M_0, \dots, M_{\ell-1}) \in \{0, 1\}^\ell$.

We have the following lemma for the above two algorithms.

Lemma 1. *Let $n, q, k, \ell \in \mathbb{Z}$ and $v \in R_q^*$ be defined as above. Then, for any $M \in \{0, 1\}^\ell$, $m = \text{Pt2poly}(M) \in R_q$ and any polynomial $e = e_0 + e_1x + \dots + e_{n-1}x^{n-1} \in R_q$ satisfying the following condition*

$$\left(\sum_{i=j \bmod n/k} |e_i \bmod^\pm q| \right) < \frac{k \cdot (q-1)}{4} \text{ for } i \in [n] \text{ and } j \in [\ell] \quad (3)$$

we always have $\text{Poly2Pt}(f^{-1}m + e) = M$.

Proof. Let $m = \text{Pt2poly}(M) \in R_q$. By the definition of $\text{Pt2poly}(M)$, we have that m only has non-zero binary coefficients at the first $\ell \leq n/k$ coordinates. Thus, multiplying m with $f^{-1} = \frac{q+1}{2}(1 + x^{n/k} + \dots + x^{(k-1)n/k})$ is essentially equal to first multiply m by $\frac{q+1}{2}$ and then copy $k-1$ times the first n/k coefficients as a block to the next $(k-1)n/k$ coordinates. In other words, for all $u = f^{-1}m \in R_q$, we always have $u_i = M_j \frac{q+1}{2}$ for all $i = j \bmod n/k$ for $i \in [n]$ and $j \in [\ell]$, where $u = u_0 + u_1x + \dots + u_{n-1}x^{n-1}$ and $M = (M_0, \dots, M_{\ell-1})$. Let $w = u + e = f^{-1}m + e \in R_q$, it suffices to show that $\text{Poly2Pt}(w)$ will always correctly recover each bit of M . Formally, let $w = w_0 + w_1x + \dots + w_{n-1}x^{n-1}$, we continue the proof by considering the value of each $M_j \in \{0, 1\}$ for $j \in [\ell]$:

- $M_j = 1$: we have that $w_i = u_i + e_i = \frac{q+1}{2} + e_i$ for all $i = j \bmod n/k$, and that $\tilde{w}_i = w_i - \frac{q+1}{2} = e_i \bmod^\pm q$. This means that

$$t_j = \sum_{i=j \bmod n/k} |\tilde{w}_i| = \sum_{i=j \bmod n/k} |e_i \bmod^\pm q| < \frac{k \cdot (q-1)}{4},$$

and that $\text{Poly2Pt}(w)$ will output $M_j = 1$;

- $M_j = 0$: we have that $w_i = e_i$ for all $i = j \bmod n/k$, and that $\tilde{w}_i = w_i - \frac{q+1}{2} = e_i - \frac{q+1}{2} \bmod^\pm q$. Since we have either $e_i = |e_i \bmod^\pm q|$ or $e_i = q - |e_i \bmod^\pm q|$, it is easy to check that $|\tilde{w}_i| \geq \frac{q-1}{2} - |e_i \bmod^\pm q|$. This means that

$$t_j = \sum_{i=j \bmod n/k} |\tilde{w}_i| \geq \sum_{i=j \bmod n/k} \left(\frac{q-1}{2} - |e_i \bmod^\pm q| \right) > \frac{k \cdot (q-1)}{4},$$

and that $\text{Poly2Pt}(w)$ will output $M_j = 0$.

This completes the proof.

Remark 1. There is a tradeoff between the plaintext length ℓ and the decoding capacity. For a fixed n , a smaller k (e.g., $k = 1$) allows to support longer plaintext length (as we require $\ell \leq n/k$) but has worse noise tolerance. In particular, if each coefficient of e is chosen from a distribution such that the probability of $|e_i \bmod^\pm q| < \frac{q-1}{4}$ for all $i \in [n]$ is $1 - p$, then the probability that $\text{Poly2Pt}(f^{-1}m + e) = M$ is roughly about $1 - p^k$. This is why we prefer to choose the largest integer k such that $n/k \geq \ell$. For the typical application of PKE in encrypting a session key $\ell = 128, 256$ or 512 , one could fix $k = n/\ell$ to obtain the best noise tolerance.

3.2 Description of Aigis-PKE⁺

Let n, q, d_t, d_u, d_v be positive integers. Let χ_α, χ_β be two noise distributions over R_q . Let $\mathcal{H} : \{0, 1\}^n \rightarrow R_q$ be a hash function, which is modeled as a random oracle. Let $\mathcal{M} = \{0, 1\}^\ell$ be the plaintext space. Let k be the largest integer satisfying $k|n$ and $n/k \geq \ell$. Let $f^{-1} = \frac{q+1}{2}(1 + x^{n/k} + \dots + x^{(k-1)n/k}) \in R_q^*$. We now give our PKE scheme Aigis-PKE⁺, which consists of the following three algorithms (KeyGen, Enc, Dec):

- Aigis-PKE⁺.KeyGen(κ): randomly choose $\rho \xleftarrow{\$} \{0, 1\}^n$, $s \xleftarrow{\$} \chi_\alpha$, $e \xleftarrow{\$} \chi_\beta$, compute $a = \mathcal{H}(\rho) \in R_q$, $t = as + e \in R_q$ and $\bar{t} = \lceil t \rceil_{q \rightarrow 2^{d_t}}$. Then, return the public key $pk = (\rho, \bar{t})$ and the secret key $sk = s$.
- Aigis-PKE⁺.Enc(pk, M): given the public key $pk = (\rho, \bar{t})$ and a plaintext $M \in \{0, 1\}^\ell$, randomly choose $r \xleftarrow{\$} \chi_\alpha$, $e_1 \xleftarrow{\$} \chi_\beta$, $e_2 \xleftarrow{\$} \chi_\beta$, compute $a = \mathcal{H}(\rho)$, $u = a^T r + e_1$, $v' = \lceil \bar{t} \rceil_{2^{d_t} \rightarrow q} r + e_2$ and $m = \text{Pt2poly}(M)$, and return the ciphertext

$$C = (\bar{u} = \lceil u \rceil_{q \rightarrow 2^{d_u}}, \bar{v} = \lceil v' + f^{-1}m \rceil_{q \rightarrow 2^{d_v}}).$$

- Aigis-PKE⁺.Dec(sk, C): given the secret key $sk = s$ and a ciphertext $C = (\bar{u}, \bar{v})$, compute $z = \lceil \bar{v} \rceil_{2^{d_v} \rightarrow q} - s \lceil \bar{u} \rceil_{2^{d_u} \rightarrow q}$, and output $M' = \text{Poly2Pt}(z)$.

Correctness and Decryption Failures. Let $c_t \in R$ satisfy that

$$\lceil \bar{t} \rceil_{2^{d_t} \rightarrow q} = \lceil \lceil as + e \rceil_{q \rightarrow 2^{d_t}} \rceil_{2^{d_t} \rightarrow q} = as + e - c_t.$$

Let $c_u \in R$ satisfy that

$$\lceil \bar{u} \rceil_{2^{d_u} \rightarrow q} = \lceil \lceil ar + e_1 \rceil_{q \rightarrow 2^{d_u}} \rceil_{2^{d_u} \rightarrow q} = ar + e_1 - c_u.$$

Let $c_v \in R$ satisfy that

$$\begin{aligned} \lceil \bar{v} \rceil_{2^{d_v} \rightarrow q} &= \lceil \lceil \lceil \bar{t} \rceil_{2^{d_t} \rightarrow q} r + e_2 + f^{-1}m \rceil_{q \rightarrow 2^{d_v}} \rceil_{2^{d_v} \rightarrow q} \\ &= \lceil \bar{t} \rceil_{2^{d_t} \rightarrow q} r + e_2 + f^{-1}m - c_v \\ &= (as + e - c_t)r + e_2 + f^{-1}m - c_v \\ &= (as + e)r + e_2 + f^{-1}m - c_v - c_t r. \end{aligned}$$

Using the above equations, we have

$$\begin{aligned}
z &= \lceil \bar{v} \rceil_{2^{d_v} \rightarrow q} - s \lceil \bar{u} \rceil_{2^{d_u} \rightarrow q} \\
&= \underbrace{(e - c_t)r - (e_1 - c_u)s + e_2 - c_v}_{= w} + f^{-1}m \\
&= w + f^{-1}m.
\end{aligned}$$

By Lemma 1, we have that $M = \text{Poly2Pt}(z)$ holds as long as $\|w\|_\infty < k(q-1)/4$. For our concrete choices of parameters, we numerically compute an upper bound on the probability of decryption failures by using an approach adapted from [5,13]. Formally, we are more interested in the case of k, n being powers of 2 and χ_α, χ_β being centrally symmetric distributions (namely, $-\chi = \chi$). By $S = \mathbb{Z}[Y]/(Y^k + 1)$ we denote the $2k$ -th cyclotomic ring, having rank k over $\mathbb{Z}$. Since $k|n$, we can view S as a subring of $R = \mathbb{Z}[X]/(X^n + 1)$ by identifying $Y = X^{n/k}$. In particular, an element $f \in R$ can be identified to a vector $(\hat{f}_0, \dots, \hat{f}_{n/k-1}) \in S^{n/k}$ such that

$$f(X) = \hat{f}_0(X^{n/k}) + X\hat{f}_1(X^{n/k}) + \dots + X^{n/k-1}\hat{f}_{n/k-1}(X^{n/k}).$$

In other words, the coefficients of $\hat{f}_j \in S$ exactly consists of $f_j, f_{j+n/k}, \dots, f_{j+(k-1)n/k}$ of $f \in R$. Using the above notations, the condition for correct decryption can be restated as

$$\|\hat{w}_j\|_1 < \frac{k \cdot (q-1)}{4} \text{ for all } j \in [n/k].$$

Furthermore, let $y \in S$ be a polynomial ranges over all polynomials with ± 1 coefficients. Then, we have $\|\hat{w}_j\|_1 = \max_y \langle \hat{w}_j, y \rangle$ and

$$\Pr \left[\|\hat{w}_j\|_1 \geq \frac{k \cdot (q-1)}{4} \right] = \Pr \left[\max_y \langle \hat{w}_j, y \rangle \geq \frac{k \cdot (q-1)}{4} \right].$$

By using a union bound, we have

$$\Pr \left[\|\hat{w}_j\|_1 \geq \frac{k \cdot (q-1)}{4} \right] \leq \sum_y \Pr \left[\langle \hat{w}_j, y \rangle \geq \frac{k \cdot (q-1)}{4} \right].$$

Since $w = (e - c_t)r - (e_1 - c_u)s + e_2 - c_v$, we have

$$\hat{w}_j = \sum_{i \in [n/k]} Y^{z_j} (\hat{e}'_i \hat{r}_{(j-i) \bmod n/k} + \hat{e}''_i \hat{s}_{(j-i) \bmod n/k}) + \hat{e}'''_j,$$

where the powers z_j are integers and $e' = e - c_t$, $e'' = e_1 - c_u$ and $e''' = e_2 - c_v$. Since all $\hat{r}_i, \hat{s}_i$ have coefficients independently chosen from centrally symmetric distribution (which are invariant under multiplication by Y^{z_i} and by -1), we can numerically compute the probability

$$\begin{aligned}
p_j &= \sum_y \Pr \left[\langle \hat{w}_j, y \rangle \geq \frac{k \cdot (q-1)}{4} \right] \\
&= \sum_y \Pr \left[\left\langle \sum_{i \in [n/k]} (\hat{e}'_i \hat{r}_i + \hat{e}''_i \hat{s}_i) + \hat{e}'''_j, y \right\rangle \geq \frac{k \cdot (q-1)}{4} \right]
\end{aligned} \tag{4}$$

by randomly choosing $\hat{r}_i, \hat{s}_i \xleftarrow{\$} \chi_\alpha, \hat{e}, \hat{e}_1, \hat{e}_2 \xleftarrow{\$} \chi_\beta$ and c_t, c_u, c_v from the distributions derived by taking an appropriate modulus switching to uniform element in R_q .⁸ Moreover, by taking a union bound we can obtain an upper bound on the probability of decryption failures

$$1 - \Pr \left[\|\hat{w}_j\|_1 < \frac{k \cdot (q-1)}{4} \text{ for all } j \in [n/k] \right] \leq n/k \cdot p_j. \quad (5)$$

It is worth to note that the above analysis does not rely on extra assumption on the coefficients distribution of the final error term w_j , and gives a very conservative upper bound on the probability of decryption failures. In particular, it does not need the identically and independently distributed assumption on the coefficients of w , which is usually used to simplify the analysis of lattice encryption schemes but was disproved both theoretically and practically in [27]. In Section 4, we will choose concrete parameters such that the decryption failure is negligibly small.

3.3 Description of Aigis-Enc⁺

Let $\text{Aigis-PKE}^+ = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be defined in the above subsection with message length $\ell = \kappa$. Let $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$, $H_2 : \{0, 1\}^{2\kappa} \rightarrow \{0, 1\}^\kappa \times \{0, 1\}^\kappa$ and $H_3 : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$ be three hash functions, which will be modeled as random oracles in the security proof. We now transform Aigis-PKE^+ into a IND-CCA secure KEM $\text{Aigis-Enc}^+ = (\text{KeyGen}, \text{Encap}, \text{Decap})$ following the generic FO-transformation.

- $\text{Aigis-Enc}^+.\text{KeyGen}(\kappa)$: given the security parameter κ as input, randomly choose $s \xleftarrow{\$} \{0, 1\}^\kappa$ and compute $(pk', sk') = \text{Aigis-PKE}^+.\text{KeyGen}(1^\kappa)$. Then, return the public key $pk = pk'$, and secret key $sk = (sk', pk, H_1(pk), s)$.
- $\text{Aigis-Enc}^+.\text{Encap}(pk)$: given the public key pk as input, randomly choose $M \xleftarrow{\$} \{0, 1\}^\kappa$, and compute

$$(K, \rho) = H_2(M, H_1(pk)) \text{ and } c = \text{Aigis-Enc}^+.\text{Enc}(pk, M; \rho).$$

Then, return the ciphertext and session key pair (c, K) .

- $\text{Aigis-Enc}^+.\text{Decap}(sk, c)$: given the secret key $sk = (sk', pk, H_1(pk), s)$ and a ciphertext c as inputs, compute

$$M' = \text{Aigis-PKE}^+.\text{Dec}(sk', c), (K', \rho') = H_2(M', H_1(pk)),$$

and $c' = \text{Aigis-Enc}^+.\text{Enc}(pk, M'; \rho')$. If $c' = c$, return $K = K'$, otherwise, return $K = H_3(s, c)$.

Since Aigis-Enc^+ is obtained by a standard application of the FO transformation (with implicit rejection) to Aigis-PKE^+ , the correctness of Aigis-Enc^+ directly follows from that of Aigis-PKE^+ .

⁸ We note that this assumption is mild under the corresponding RLWE assumptions.

4 Recommended Parameters

In Table 1, we give three sets of parameters (namely, Aigis-Enc⁺-512, Aigis-Enc⁺-1024 and Aigis-Enc⁺-2048) for Aigis-Enc⁺, aiming at providing classical (Quantum) security of at least 128 (80), 256 (128) and 512 (256) bits, respectively. The third column gives the distributions used for key generation and encryption. The fourth column presents the parameters used for compressing the public key and ciphertexts. The columns labeled by $|pk|$, $|sk|$, $|C|$ present the corresponding sizes of public key, secret key and ciphertext in bytes. The column labeled by $|ss|$ gives the sizes of the shared secret key in bytes. The last column presents the decryption failure probability, which is computed by using a python script following the analysis given in Sec. 3.2.⁹ We set our decryption failure probabilities $< 2^{-128}$ with some margin so that it is infeasible to obtain a single decryption failure using at most 2^{64} decryption queries (see the directional failure boosting attacks [23]).

In Table 2, we give the estimated security bits. The second column gives the estimated security of the secret key obtained by running the LWE estimator [3] using the core-SVP model [5], where C/Q denotes the lowest classical/quantum bits given by the estimator, and ATK indicates the concrete attacks achieving the lowest estimates. Given the recent progress of dual-hybrid LWE attacks, we believe it is prudent to consider the (refined) MATZOV model in concrete security estimates as also commented in [29]. For this, we also present the estimated security bits in a (refined) MATZOV model (see Section 8) recently shown to have lowered the concrete security estimates of Kyber below the NIST requirements [18].

Table 1: Parameters Sets for Aigis-Enc⁺

Parameters	(n, q)	$(\chi_\alpha, \chi_\beta)$	(d_t, d_u, d_v)	$ pk $	$ sk $	$ C $	$ ss $	Dec. Fail.
Aigis-Enc ⁺ -512	(512, 3329)	(B_5, B_6)	(10, 10, 4)	656	16	896	16	-145
Aigis-Enc ⁺ -1024	(1024, 3329)	(B_2, B_6)	(10, 10, 3)	1312	32	1664	32	-145
Aigis-Enc ⁺ -2048	(2048, 3329)	(B_2, B_3)	(10, 10, 4)	2624	64	3584	64	-157

Table 2: Estimated Security of Practical Parameters for Aigis-Enc⁺

Parameter Sets	Primal (C,Q)	Dual (C,Q)	MATZOV [†] (C,Q)
Aigis-Enc ⁺ -512	(130,114)	(128,113)	(130,108)
Aigis-Enc ⁺ -1024	(273,240)	(271,238)	(266,217)
Aigis-Enc ⁺ -2048	(572,503)	(568,499)	(516,420)

⁹ The decryption failure computation rely on the same heuristic assumption commonly adopted in rounding-based schemes like Kyber [13]: namely, that each coefficient of the rounding error is randomly and independently distributed. This assumption is distinct from the independence-of-coefficient-failures heuristic, which assumes that decryption failures are identically and independently distributed across coefficients—an assumption that has been disproven both theoretically and empirically [26].

5 Implementation Details

We have implemented our schemes on both x86 and ARM platforms. For each platform, we provide two implementations: a reference C version and a version optimized with vector instructions (AVX2 for x86, NEON for ARM). The following details highlight the implementation choices that most significantly affect performance, with a primary focus on the x86 platform.

Before presenting implementation details of our schemes, we first outline the conventions of the pseudocode used to describe the algorithms.

Arrays. We write $B[i]$ for the entry of an array B at index i . This notation applies to bit strings, byte strings, and integer arrays. All arrays are indexed from 0. For integers $0 \leq k \leq m$, we write

$$B[k : m] = (B[k], B[k + 1], \dots, B[m - 1])$$

for the subarray of B consisting of the entries with indices from k to $m - 1$. An element $f \in R_q$ is a polynomial of the form

$$f = f_0 + f_1X + \dots + f_{n-1}X^{n-1}.$$

In the pseudocode, we represent f by the array

$$(f_0, f_1, \dots, f_{n-1}) \in \mathbb{Z}_q^n, \tag{6}$$

whose entries are the coefficients of f . By abuse of notation, we also denote this array by f . Thus, for $0 \leq i < n$, the array entry $f[i]$ denotes the coefficient f_i .

5.1 Partial NTT Multiplication

Let $\zeta = 17 \in \mathbb{Z}_q$ is a primitive 256-th root of unity modulo q . Define $\text{BitRev}_7(k)$ to be the integer represented by bit-reversing the unsigned 7-bit value that corresponds to the input integer $r \in \{0, \dots, 127\}$. Specifically, if $r = r_0 + 2r_1 + 4r_2 + \dots + 64r_6$ with $r_i \in \{0, 1\}$, then $\text{BitRev}_7(k) = r_6 + 2r_5 + 4r_4 + \dots + 64r_0$. The NTT and NTT^{-1} algorithms are defined in Algorithm 1 and 2.

Algorithm 1: NTT(f)

Input: array $f \in \mathbb{Z}_q^n$
Output: array $\hat{f} \in \mathbb{Z}_q^n$

```

1  $\hat{f} := f$ 
2 for  $i := 0; i < n; i := i + 128$  do
3    $k := 0$ 
4   for  $len := 32; len > 1; len := len/2$  do
5     for  $start := 0; start < 128; start := start + len$  do
6        $zeta := \zeta^{\text{BitRev}_7(k)} \bmod q$ 
7        $k := k + 1$ 
8       for  $j := start; j < start + len; j ++$  do
9          $t := zeta \cdot \hat{f}[i + j + len]$ 
10         $\hat{f}[i + j + len] := \hat{f}[i + j] - t$ 
11         $\hat{f}[i + j] := \hat{f}[i + j] + t$ 
12      end
13    end
14  end
15 end
16 return  $\hat{f}$ 

```

Algorithm 2: NTT⁻¹($\hat{f}$)

Input: array $\hat{f} \in \mathbb{Z}_q^n$
Output: array $f \in \mathbb{Z}_q^n$

```

1  $f := \hat{f}$ 
2 for  $i := 0; i < n; i := i + 128$  do
3    $k := 127$ 
4   for  $len := 1; len \leq 128; len := 2 \cdot len$  do
5     for  $start := 0; start < 128; start := start + len$  do
6        $zeta := \zeta^{\text{BitRev}_7(k)} \bmod q$ 
7        $k := k - 1$ 
8       for  $j := start; j < start + len; j ++$  do
9          $t := f[i + j]$ 
10         $f[i + j] := t + f[i + j + len]$ 
11         $f[i + j + len] := zeta \cdot (t - f[i + j + len])$ 
12      end
13    end
14  end
15 end
16  $f := f \cdot 3303 \bmod q$  ; // multiply every entry by  $3303 \equiv 128^{-1} \bmod q$ 
17 return  $f$ 

```

5.2 Symmetric Primitives

In our default implementations, we use SHA3 for both the hash function and the pseudorandom generator (PRG), as in the NIST ML-KEM standard. Given an input bit string N and an output length d , the function $\text{KECCAK}[c](N, d)$ is defined following [40]. Then, we define four hash functions as follows:

$$\begin{aligned}\text{Hash-128}(M) &= \text{KECCAK}[256](M\|01, 128); \\ \text{Hash-256}(M) &= \text{KECCAK}[512](M\|01, 256) = \text{SHA3-256}(M); \\ \text{Hash-512}(M) &= \text{KECCAK}[1024](M\|01, 512) = \text{SHA3-512}(M). \\ \text{Hash-1024}(M) &= \text{KECCAK}[1024](M\|01, 1024).\end{aligned}$$

Then, we define three XOFs as follows:

$$\begin{aligned}\text{XOF-128}(M, d) &= \text{KECCAK}[256](M\|1111, 128) = \text{SHAKE128}(M, 128); \\ \text{XOF-256}(M, d) &= \text{KECCAK}[512](M\|1111, 256) = \text{SHAKE256}(M, 256); \\ \text{XOF-512}(M, d) &= \text{KECCAK}[1024](M\|1111, 512).\end{aligned}$$

The instantiating of XOF_1 , XOF_2 , H_1 , H_2 , and H_3 is listed in Table 3.

Table 3: Instantiating of XOF and Hash functions

	$\text{XOF}_1(M, d)$	$\text{XOF}_2(M, d)$	$\text{H}_1(M)$	$\text{H}_2(M)$	$\text{H}_3(M)$	$\kappa' = \kappa/8$
Aigis-Enc^+-512	$\text{XOF-128}(M, d)$	$\text{XOF-128}(M, d)$	$\text{Hash-128}(M)$	$\text{Hash-256}(M)$	$\text{XOF-128}(M, 128)$	16
Aigis-Enc^+-1024	$\text{XOF-128}(M, d)$	$\text{XOF-256}(M, d)$	$\text{Hash-256}(M)$	$\text{Hash-512}(M)$	$\text{XOF-256}(M, 256)$	32
Aigis-Enc^+-2048	$\text{XOF-128}(M, d)$	$\text{XOF-512}(M, d)$	$\text{Hash-512}(M)$	$\text{Hash-1024}(M)$	$\text{XOF-512}(M, 512)$	64

XOFs can be used to absorb a sequence of arbitrary-length strings and squeeze a sequence of arbitrary-length strings. These functions perform buffering to handle any incomplete data blocks while absorbing or squeezing. Take XOF_1 as an example:

- $\text{ctx} \leftarrow \text{XOF}_1.\text{Init}()$
Initializes a hash function context.
- $\text{ctx} \leftarrow \text{XOF}_1.\text{Absorb}(\text{ctx}, \text{str})$
Injects data to be used in the absorbing phase of XOF_1 and updates context ctx .
- $(\text{ctx}, \text{out}) \leftarrow \text{XOF}_1.\text{Squeeze}(\text{ctx}, r)$
Extracts r output bytes produced during the squeezing phase of XOF_1 and updates context ctx .

5.3 Encode and Decode

In Aigis-Enc^+ , we need to encode the polynomials (or integer arrays) into byte arrays and decode the byte arrays into polynomials (or integer arrays).

The Algorithms from 3 to 6 specify the converting between bit arrays, byte arrays, and integers. Based on these algorithms, let $\alpha \in \mathbb{Z}$ be the length of the byte string B , we define the following functions:

- $\text{IntegerToBytes}(x, \alpha) = \text{BitsToBytes}(\text{IntegerToBits}(x, 8\alpha), 8\alpha)$;
- $\text{BytesToInteger}(B, \alpha) = \text{BitsToInteger}(\text{BytesToBits}(B, \alpha), 8\alpha)$.

In Algorithm 7 and 8, we define the encoding and decoding functions for polynomials. These functions are used for conversion between integers modulo 2^d and bytes. We use Compress_d to implement $\lceil x \rceil_{q \rightarrow 2^d}$ and Decompress_d to implement $\lceil x \rceil_{2^d \rightarrow q}$. The division by q in Compress_d can be implemented by multiplying the coefficients $f[i]$ by 20156 followed by a right-shift of 26 bits.

Algorithm 3: $\text{BitsToBytes}(y, \alpha)$

Input: Bit string y of length α .

Output: Byte string z of length $\lceil \alpha/8 \rceil$.

```

1  $z \in \mathcal{B}^{\lceil \alpha/8 \rceil} := 0^{\lceil \alpha/8 \rceil}$ ;
2 for  $i : 0 \rightarrow \alpha - 1$  do
3    $z[\lfloor i/8 \rfloor] := z[\lfloor i/8 \rfloor] + y[i] \cdot 2^{i \bmod 8}$ ;
4 end
5 return  $z$ ;
```

Algorithm 4: $\text{BytesToBits}(z, \alpha)$

Input: Byte string z of length α .

Output: Bit string y of length 8α .

```

1 for  $i : 0 \rightarrow \alpha - 1$  do
2   for  $j : 0 \rightarrow 7$  do
3      $y[8i + j] := z[i] \bmod 2$ ;
4      $z[i] := \lfloor z[i]/2 \rfloor$ ;
5   end
6 end
7 return  $y$ ;
```

Algorithm 5: $\text{BitsToInteger}(y, \alpha)$

Input: Positive integer α and bit string y of length α .

Output: Nonnegative integer x .

```

1  $x := 0$ ;
2 for  $i : 1 \rightarrow \alpha$  do
3    $x := 2x + y[\alpha - i]$ ;
4 end
5 return  $x$ ;
```

Algorithm 6: IntegerToBits(x, α)

Input: Nonnegative integer x and positive integer α .**Output:** Bit string y of length α .

```

1 for  $i : 0 \rightarrow \alpha - 1$  do
2    $y[i] := x \bmod 2$ ;
3    $x := \lfloor x/2 \rfloor$ ;
4 end
5 return  $y$ ;

```

Algorithm 7: Encode $_r$ (f)

Input : Integer array $f \in \mathbb{Z}_q^t$.**Output:** Byte string of length $\lceil rn/8 \rceil$.

```

1 for  $i : 0 \rightarrow n$  do
2    $b := b || \text{IntegerToBits}(f[i], r)$ ;
3 end
4 return BitsToBytes( $b$ );

```

Algorithm 8: Decode $_r$ (B)

Input : Byte string B .**Output:** Integer array $f \in \mathbb{Z}_q^t$.

```

1  $b := \text{BytesToBits}(B, \lceil rn/8 \rceil)$ ;
2 for  $i : 0 \rightarrow n$  do
3    $f[i] := \text{BitsToInteger}(b[ir : ir + r], r)$ ;
4 end
5 return  $f$ ;

```

Algorithm 9: Compress $_d$ (f)

Input : Integer array $f \in \mathbb{Z}_q^t$.**Output:** Integer array $f \in \mathbb{Z}_{2^d}^t$.

```

1 for  $i : 0 \rightarrow n$  do
2    $f[i] := \lceil f[i] \cdot (2^d/q) \rceil \bmod 2^d$ ;
3 end
4 return  $f$ ;

```

Algorithm 10: Decompress $_d$ (f)

Input : Integer array $f \in \mathbb{Z}_{2^d}^t$.**Output:** Integer array $f \in \mathbb{Z}_q^t$.

```

1 for  $i : 0 \rightarrow n$  do
2    $f[i] := \lceil f[i] \cdot (q/2^d) \rceil$ ;
3 end
4 return  $f$ ;

```

5.4 Sampling Algorithms

Sample uniformly polynomials in R_q . We use the function **GenA** in Algorithm 11 to do deterministic sample uniform elements in R_q . Since the NTT transformation maps uniformly distributed elements in R_q to uniformly distributed vector in $\mathbb{Z}_q^n$, we can assume that the polynomial a generated by **GenA** is the result of the NTT transformation of a random element.

Sample polynomials in B_η . We use the Algorithm 13 to generate polynomials in binomial distribution. **SamplePolyCBD** generate a byte string through the random seed ρ and a nonce $N \in \mathbb{Z}$, and then pass it to the function CBD_η , which is defined in Algorithm 12. The function CBD_η samples polynomials in B_η directly for $\eta = 1, 2, 4$. For $\eta = 3, 5, 6$, it adopts a recursion strategy to generate the polynomial.

Algorithm 11: $\text{GenA}(\rho)$

Input : Byte string $\rho \in \mathcal{B}^{\kappa'}$.
Output: Polynomial $\hat{a} \in R_q$.

```

1 for  $i : 0 \rightarrow 4$  do
2    $j := 0$ ;
3    $\tilde{\rho} = \rho \parallel \text{IntegerToBytes}(i, 1)$ ;
4    $ctx := \text{XOF}_1.\text{Init}()$ ;
5    $ctx := \text{XOF}_1.\text{Absorb}(ctx, \tilde{\rho})$ ;
6   while  $j < n/4$  do
7      $(ctx, z) := \text{XOF}_1.\text{Squeeze}(ctx, 3)$ ;
8      $t_1 := z[0] + 256 \cdot (z[1] \bmod 16)$ ;
9      $t_2 := \lfloor z[1]/16 \rfloor + 16 \cdot z[2]$ ;
10    if  $t_1 < q$  then
11       $\hat{a}[in/4 + j] := t_1$ ;
12       $j := j + 1$ ;
13    end
14    if  $t_2 < q$  and  $j < n$  then
15       $\hat{a}[in/4 + j] := t_2$ ;
16       $j := j + 1$ ;
17    end
18  end
19 end
20 return  $\hat{a}$ ;

```

Algorithm 12: $\text{CBD}_\eta(z)$

Input: Bytes string $z \in \mathcal{B}^{\lceil n\eta/4 \rceil}$.
Output: Polynomial $f \in R_q$.

```

1 if  $\eta = 1$  or  $\eta = 2$  or  $\eta = 4$  then
2    $b := \text{BytesToBits}_q(z)$ ;
3   for  $i := 0; i < n; i++$  do
4      $x := \sum_{j=0}^{\eta-1} b[2i\eta + j]$ ;
5      $y := \sum_{j=0}^{\eta-1} b[2i\eta + \eta + j]$ ;
6      $f[i] := x - y \bmod q$ ;
7   end
8   return  $f$ ;
9 end
10 else if  $\eta = 3$  then
11    $f := \text{CBD}_2(z[0 : n/2 - 1])$ ;
12    $t := \text{CBD}_1(z[n/2 : 3n/4 - 1])$ ;
13 end
14 else if  $\eta = 5$  then
15    $f := \text{CBD}_4(z[0 : n - 1])$ ;
16    $t := \text{CBD}_1(z[n : 5n/4 - 1])$ ;
17 end
18 else if  $\eta = 6$  then
19    $f := \text{CBD}_4(z[0 : n - 1])$ ;
20    $t := \text{CBD}_2(z[n : 3n/2 - 1])$ ;
21 end
22  $f := f + t$ ;
23 return  $f$ ;
```

Algorithm 13: $\text{SamplePolyCBD}(B_\eta, \rho, N)$

Input: Distribution B_η , where $\eta = \{0, 1, 2, \dots, 6\}$, byte string $\rho \in \mathcal{B}^{\kappa'}$,
 nonce $N \in \mathbb{Z}$.
Output: Polynomial $f \in R_q$.

```

1  $z := \text{XOF}_2(\rho || N, \lceil n\eta/4 \rceil)$ ;
2  $f := \text{CBD}_\eta(z)$ ;
3 return  $f$ ;
```

5.5 The **Aigis-PKE⁺** Scheme

This section describes the component scheme **Aigis-PKE⁺**. **Aigis-PKE⁺** serves only as a collection of subroutines for use in the algorithms of the scheme **Aigis-Enc⁺**. **Aigis-PKE⁺** consists of four algorithms:

- Key generation (**Aigis-PKE⁺.KeyGen**)
- Encryption (**Aigis-PKE⁺.Enc_{internal}** and **Aigis-PKE⁺.Enc**)
- Decryption (**Aigis-PKE⁺.Dec**)

Aigis-PKE⁺.Enc_internal is defined to control the random seed used in encryption. When Aigis-PKE⁺ is instantiated as part of Aigis-Enc⁺, Aigis-PKE⁺ inherits the parameter set selected for Aigis-Enc⁺.

Algorithm 14: Aigis-PKE⁺.KeyGen()

Output: Public key $pk \in \mathcal{B}^{\kappa' + nd_t/8}$, secret key $sk \in \mathcal{B}^{3n/2}$.

- 1 $\xi \xleftarrow{\$} \mathcal{B}^{\kappa'}$;
 - 2 $(\rho, \sigma) \in \mathcal{B}^{\kappa'} \times \mathcal{B}^{\kappa'} := H_2(\xi)$;
 - 3 $\hat{a} \in R_q = \text{GenA}(\rho)$;
 - 4 $s \in R_q := \text{SamplePolyCBD}(\chi_\alpha, \sigma, 0)$;
 - 5 $e \in R_q := \text{SamplePolyCBD}(\chi_\beta, \sigma, 1)$;
 - 6 $\hat{s} := \text{NTT}(s)$;
 - 7 $t := \text{NTT}^{-1}(\hat{a} \circ \hat{s}) + \hat{e}$;
 - 8 $\bar{t} := \text{Compress}_{d_t}(t)$;
 - 9 $pk := \rho || \text{Encode}_{d_t}(\bar{t})$;
 - 10 $sk := \text{Encode}_{12}(\hat{s})$;
-

Algorithm 15: Aigis-PKE⁺.Enc_internal(pk, M, ρ')

Input : Public key $pk \in \mathcal{B}^{\kappa' + nd_t/8}$, message $M \in \mathcal{B}^{\kappa'}$, and seed $\rho' \in \mathcal{B}^{\kappa'}$.

Output: Ciphertext $c \in \mathcal{B}^{n(d_u + d_v)/8}$.

- 1 $\rho \in \mathcal{B}^{\kappa'} := pk[0 : \kappa' - 1]$;
 - 2 $\bar{t} := \text{Decode}_{d_t}(pk[\kappa' : \lceil nd_t/8 \rceil - 1])$;
 - 3 $t' = \text{Decompress}_{d_t}(\bar{t})$;
 - 4 $\hat{a} \in R_q := \text{GenA}(\rho)$;
 - 5 $r \in R_q := \text{SamplePolyCBD}(\chi_\alpha, \rho', 0)$;
 - 6 $e_1 \in R_q := \text{SamplePolyCBD}(\chi_\beta, \rho', 1)$;
 - 7 $e_2 \in R_q := \text{SamplePolyCBD}(\chi_\beta, \rho', 2)$;
 - 8 $\hat{r} := \text{NTT}(r)$;
 - 9 $\hat{t}' := \text{NTT}(t')$;
 - 10 $u := \text{NTT}^{-1}(\hat{a}^T \circ \hat{r}) + e_1$;
 - 11 $y := \text{NTT}^{-1}(\hat{t}' \circ \hat{r}) + e_2$;
 - 12 $m := \text{Pt2poly}(M)$;
 - 13 $\bar{u} := \text{Compress}_{d_u}(u)$;
 - 14 $\bar{v} := \text{Compress}_{d_v}(y + f^{-1}m)$;
 - 15 $C := \text{Encode}_{d_u}(\bar{u}) || \text{Encode}_{d_v}(\bar{v})$;
 - 16 **return** C ;
-

Algorithm 16: Aigis-PKE⁺.Enc(pk, M)

Input : Public key $pk \in \mathcal{B}^{\kappa' + nd_t/8}$, message $M \in \mathcal{B}^{\kappa'}$.

Output: Ciphertext $C \in \mathcal{B}^{n(d_u + d_v)/8}$.

- 1 $\rho' \xleftarrow{\$} \mathcal{B}^{\kappa'}$;
 - 2 $C := \text{Aigis-PKE}^+.\text{Enc_internal}(pk, M, \rho')$;
 - 3 **return** C ;
-

Algorithm 17: Aigis-PKE⁺.Dec(sk, C)

Input : Secret key $sk \in \mathcal{B}^{3n/2}$, ciphertext $C \in \mathcal{B}^{n(d_u + d_v)/8}$.

Output: Message $M' \in \mathcal{B}^{\kappa'}$.

- 1 $\hat{s} := \text{Decode}_{12}(sk)$;
 - 2 $\bar{u} := \text{Decode}_{d_u}(C[0 : nd_u/8 - 1])$;
 - 3 $\bar{v} := \text{Decode}_{d_v}(C[nd_u/8 - 1 : n(d_u + d_v)/8 - 1])$;
 - 4 $C = (\bar{u}, \bar{v})$;
 - 5 $v' := \text{Decompress}_{d_v}(\bar{v})$;
 - 6 $u' := \text{Decompress}_{d_u}(\bar{u})$;
 - 7 $z = v' - \text{NTT}^{-1}(\hat{s} \circ \text{NTT}(u'))$;
 - 8 $M' := \text{Poly2Pt}(z)$;
 - 9 **return** M' ;
-

5.6 The Aigis-Enc⁺ Scheme

This section describes the three main algorithms of the Aigis-Enc⁺ scheme:

- Key generation (Aigis-Enc⁺.KeyGen)
- Encapsulation (Aigis-Enc⁺.Encap)
- Decapsulation (Aigis-Enc⁺.Decap)

Algorithm 18: Aigis-Enc⁺.KeyGen()

Output: Public key $pk \in \mathcal{B}^{\kappa' + nd_t/8}$, secret key $sk \in \mathcal{B}^{3n/2 + 3\kappa' + nd_t/8}$.

- 1 $s \xleftarrow{\$} \mathcal{B}^{\kappa'}$;
 - 2 $(pk', sk') := \text{Aigis-PKE}^+.\text{KeyGen}(1^\kappa)$;
 - 3 $pk := pk'$;
 - 4 $sk := sk' || pk || H_1(pk) || s$;
 - 5 **return** (pk, sk) ;
-

Algorithm 19: Aigis-Enc⁺.Encap(pk)

Input : Public key $pk \in \mathcal{B}^{\kappa' + nd_t/8}$.
Output: Shared secret key $K \in \mathcal{B}^{32}$, ciphertext $c \in \mathcal{B}^{n(d_u+d_v)/8}$

- 1 $M \xleftarrow{\$} \mathcal{B}^{\kappa'}$;
- 2 $h := H_1(pk)$;
- 3 $(K, \rho') \in \mathcal{B}^{\kappa'} \times \mathcal{B}^{\kappa'} := H_2(M \| h)$;
- 4 $c := \text{Aigis-Enc}^+.\text{Enc_internal}(pk, M, \rho')$;
- 5 **return** (c, K) ;

Algorithm 20: Aigis-Enc⁺.Decap(sk, c)

Input : Secret key $sk \in \mathcal{B}^{3n/2+3\kappa'+nd_t/8}$, ciphertext $c \in \mathcal{B}^{n(d_u+d_v)/8}$
Output: Shared secret key $K \in \mathcal{B}^{32}$

- 1 $sk' := sk[0 : 3n/2 - 1]$;
- 2 $pk := sk[3n/2 : 3n/2 + \kappa' + nd_t/8 - 1]$;
- 3 $h := sk[3n/2 + \kappa' + nd_t/8 : 3n/2 + 2\kappa' + nd_t/8 - 1]$;
- 4 $s := sk[3n/2 + 2\kappa' + nd_t/8 : 3n/2 + 3\kappa' + nd_t/8 - 1]$;
- 5 $M' := \text{Aigis-PKE}^+.\text{Dec}(sk', c)$;
- 6 $(K', \rho'') \in \mathcal{B}^{\kappa'} \times \mathcal{B}^{\kappa'} := H_2(M' \| h)$;
- 7 $c' := \text{Aigis-Enc}^+.\text{Enc_internal}(pk, M', \rho'')$;
- 8 $K \in \mathcal{B}^{\kappa'} := H_3(s, c)$;
- 9 **if** $c = c'$ **then**
- 10 $K := K'$;
- 11 **end**
- 12 **return** K ;

6 Performance and Comparison

We evaluate the performance of Aigis-Enc⁺ and compare it with Kyber on the following two platforms:

x86 AMD Ryzen 9 7950X3D processor based on the Zen 4 microarchitecture. The CPU frequency was fixed at 4.20 GHz, with Core Performance Boost and Simultaneous Multithreading (SMT) disabled. The RAM is 64 GiB. All benchmarks were executed in a single-threaded configuration. The software environment consisted of Ubuntu 26.04 LTS and GCC 15.2.0. The implementations were compiled using the flags: `-O3 -fomit-frame-pointer -march=native`.

ARM HUAWEI Kirin 9000C Processor based on the AArch64 architecture. The CPU frequency was fixed at 2.487 GHz. The RAM is 8 GiB. The software environment consisted of Kylin Linux Desktop V10 SP1 and GCC 9.3.0. The implementations were compiled using the flags: `-O3 -fomit-frame-pointer -march=native`.

On the x86 platform, we profile two implementations for all schemes: REF (plain C) and AVX2. On the ARM platform, we profile the AArch64 implementation for

both **Aigis-Enc⁺** and **Kyber**, and additionally report the NEON implementation of **Aigis-Enc⁺**. The performance is measured in terms of CPU cycles and reported as the average over 30,000 runs of each function. The performance of **Kyber** is profiled using the round 3 submission package to NIST¹⁰.

6.1 Performance on x86

Table 4: The Performance of **Aigis-Enc⁺** (CPU cycles)

Scheme	Parameter Set	REF			AVX2		
		KeyGen	Enc	Dec	KeyGen	Enc	Dec
Aigis-Enc⁺	Aigis-Enc⁺-512	87,090	133,941	185,768	27,971	31,868	33,580
	Aigis-Enc⁺-1024	176,999	290,534	411,454	46,253	58,714	64,675
	Aigis-Enc⁺-2048	409,663	675,944	965,551	113,261	144,047	169,408
Kyber	Kyber-512	115,173	136,182	181,164	20,528	21,456	21,982
	Kyber-768	191,903	214,943	275,047	33,225	33,758	35,788
	Kyber-1024	294,902	313,763	389,691	43,809	43,684	47,021

Table 4 and Figure 3 compare **Aigis-Enc⁺** and **Kyber** on x86 across 128- to 512-bit security levels (with x-axis labels (**CoreSVP**, **MATZOV**) for classical security estimates). Under REF, **Aigis-Enc⁺** consistently matches or exceeds **Kyber** in performance while offering stronger security: at both 128-bit and 256-bit levels, the **Aigis-Enc⁺** variants achieve competitive or better operation-wise speeds and lower total computational costs than their **Kyber** counterparts, despite providing higher security coordinates.

Figure 4 shows the total costs of **Aigis-Enc⁺** and **Kyber** under a transmission cost of 2000 cycles/byte. Even after adding this overhead, the compact public-key and ciphertext sizes of the **Aigis-Enc⁺** variants offset any computational gap, resulting in AVX2 total costs that are consistently below those of their **Kyber** counterparts.

¹⁰ <https://github.com/pq-crystals/kyber>

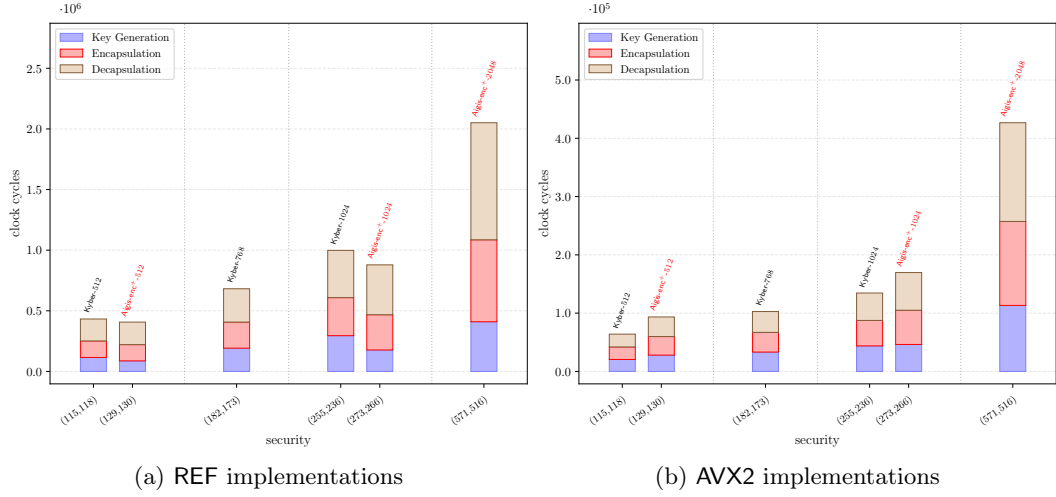


Fig. 3: Benchmarks on x86 platform

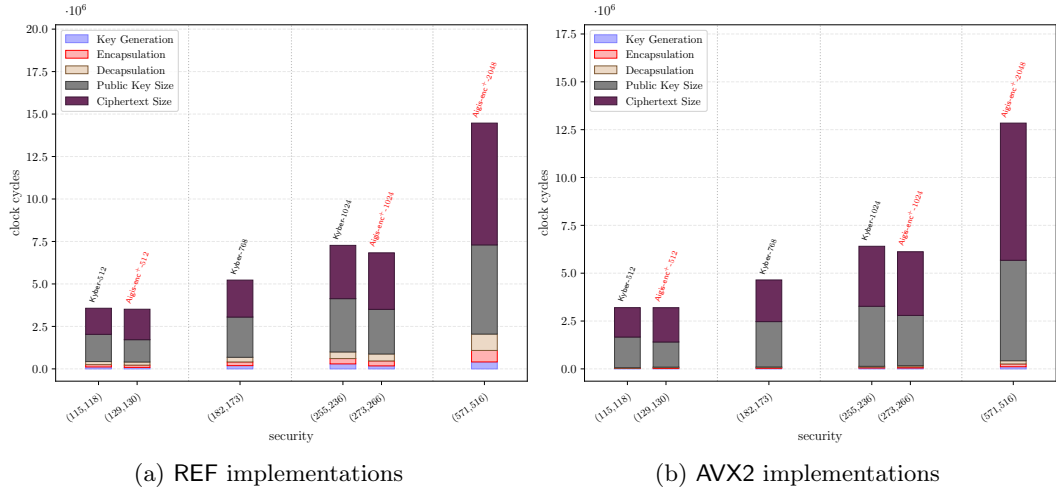


Fig. 4: Benchmarks on x86 platform with 2000 cycles/byte transmission costs

6.2 Performance on ARM

In Table 5, we report the ARM performance of Aigis-Enc⁺ and Kyber. Fig. 5 illustrates the pure computational costs, while Fig. 6 includes the same transmission-cost estimate of 2000 cycles/byte as in the x86 comparison. As in the x86 figures, the x-axis is indexed by (CoreSVP, MATZOV), namely the pair of classical security estimates under the Core-SVP and refined MATZOV models.

On AArch64, Aigis-Enc⁺ shows competitive performance with Kyber across both security levels: it excels in key generation, matches Kyber in encapsulation for the 256-bit case, and incurs only moderate overhead in decapsulation—all while providing stronger security guarantees.

Table 5: The Performance of Aigis-Enc⁺ on ARM (CPU cycles)

Scheme	Parameter Set	KeyGen	Enc	Dec
AArch64				
Aigis-Enc ⁺	Aigis-Enc ⁺ -512	62,990	97,830	137,827
	Aigis-Enc ⁺ -1024	121,021	195,522	277,099
	Aigis-Enc ⁺ -2048	289,126	467,949	667,734
Kyber	Kyber-512	73,843	87,739	112,718
	Kyber-768	124,635	132,371	168,949
	Kyber-1024	192,916	195,656	241,220
NEON				
Aigis-Enc ⁺	Aigis-Enc ⁺ -512	56,048	70,608	89,604
	Aigis-Enc ⁺ -1024	105,053	138,808	178,617
	Aigis-Enc ⁺ -2048	248,108	336,847	441,266

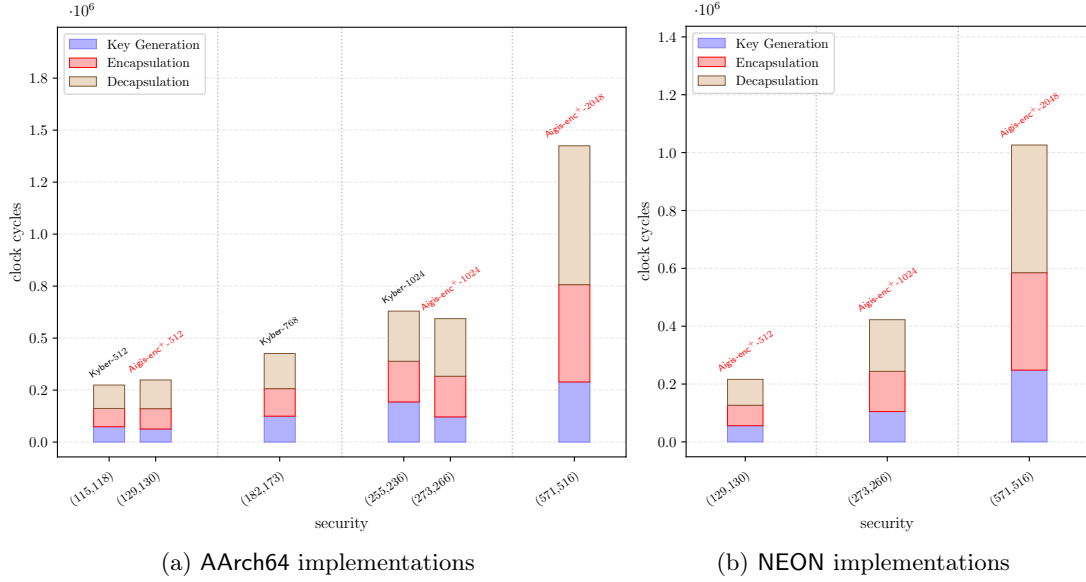


Fig. 5: Benchmarks on ARM platform

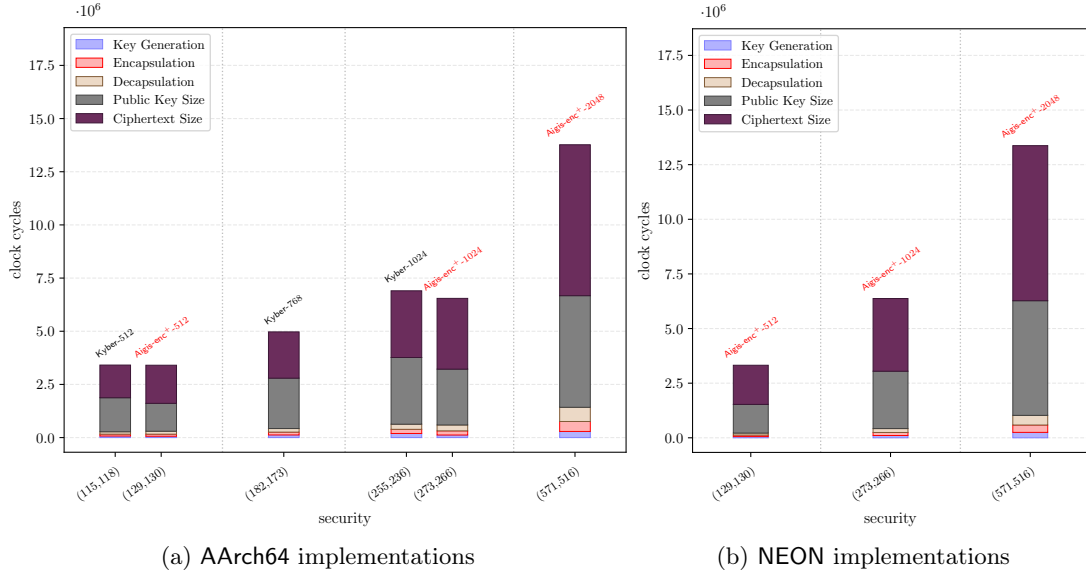


Fig. 6: Benchmarks on ARM platform with 2000 cycles/byte transmission costs

7 Provable Security

In this section, we give detail proofs of the IND-CPA security of Aigis-PKE⁺ and the IND-CCA security of Aigis-Enc⁺ in the ROM [10] and QROM [12] respectively. We denote $\text{Fcs}[X, Y]$ as the set containing all functions with input space X and output space Y . When $X = \{0, 1\}^m$ and $Y = \{0, 1\}^n$, we also simplify $\text{Fcs}[X, Y]$ to $\text{Fcs}[m, n]$.

7.1 IND-CPA Security Proof of Aigis-PKE⁺

The Aigis-PKE⁺ is IND-CPA secure under the ARLWE and semi-ARLWE assumptions. Formally, we have the following theorem.

Theorem 1 *Let $\mathcal{H} : \{0, 1\}^n \rightarrow R_q$ be a random oracle. Let $\mathcal{A}$ be an IND-CPA adversary against Aigis-PKE⁺, making q_H classical (or quantum) queries to $\mathcal{H}$. Then, we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{Aigis-PKE}^+}^{\text{IND-CPA}}(\mathcal{A}) \leq \text{Adv}_{n, q, \eta_1, \eta_2}^{\text{ARLWE}}(\mathcal{B}_1) + \text{Adv}_{n, q, 2^{dt}, \eta_1, \eta_2}^{\text{semi-ARLWE}}(\mathcal{B}_2).$$

The running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. We introduce a sequence of games as shown in Fig. 7 to prove this theorem.

Game $\mathbf{G}_0$: This game is the IND-CPA security game of Aigis-PKE⁺ with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{Aigis-PKE}^+}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr[1 \leftarrow \mathbf{G}_0] - \frac{1}{2} \right|. \quad (7)$$

Game G_0 1. $\mathcal{H} \xleftarrow{\$} \text{Fcs}[\{0, 1\}^n, R_q]$ $\rho \xleftarrow{\$} \{0, 1\}^n, s \xleftarrow{\$} \chi_\alpha, e \xleftarrow{\$} \chi_\beta$ $a = \mathcal{H}(\rho), t = as + e$ $\bar{t} = \lceil t \rceil_{q \rightarrow 2^{d_t}}, pk = (\rho, \bar{t})$ 2. $(M_0^*, M_1^*) \leftarrow \mathcal{A}^{\mathcal{H}}(pk)$ 3. $b \xleftarrow{\$} \{0, 1\}, m^* = \text{Pt2poly}(M_b^*)$ $r \xleftarrow{\$} \chi_\alpha, e_1 \xleftarrow{\$} \chi_\beta, e_2 \xleftarrow{\$} \chi_\beta$ $u = a^T r + e_1, v' = \lceil \bar{t} \rceil_{2^{d_t} \rightarrow q} r + e_2$ $c_1^* := \lceil u \rceil_{q \rightarrow 2^{d_u}}, c_2^* := \lceil v' + f^{-1} m^* \rceil_{q \rightarrow 2^{d_v}}$ 4. $b' \leftarrow \mathcal{A}^{\mathcal{H}}(pk, (c_1^*, c_2^*))$ 5. return $\llbracket b' = b \rrbracket$	Game G_1 1. $\mathcal{H} \xleftarrow{\$} \text{Fcs}[\{0, 1\}^n, R_q]$ $\rho \xleftarrow{\$} \{0, 1\}^n, s \xleftarrow{\$} \chi_\alpha, e \xleftarrow{\$} \chi_\beta$ $a \xleftarrow{\$} R_q, \mathcal{H} = \mathcal{H}[\rho \rightarrow a], t = as + e$ $\bar{t} = \lceil t \rceil_{q \rightarrow 2^{d_t}}, pk = (\rho, \bar{t})$ 2. $(M_0^*, M_1^*) \leftarrow \mathcal{A}^{\mathcal{H}}(pk)$ 3. $b \xleftarrow{\$} \{0, 1\}, m^* = \text{Pt2poly}(M_b^*)$ $r \xleftarrow{\$} \chi_\alpha, e_1 \xleftarrow{\$} \chi_\beta, e_2 \xleftarrow{\$} \chi_\beta$ $u = a^T r + e_1, v' = \lceil \bar{t} \rceil_{2^{d_t} \rightarrow q} r + e_2$ $c_1^* := \lceil u \rceil_{q \rightarrow 2^{d_u}}, c_2^* := \lceil v' + f^{-1} m^* \rceil_{q \rightarrow 2^{d_v}}$ 4. $b' \leftarrow \mathcal{A}^{\mathcal{H}}(pk, (c_1^*, c_2^*))$ 5. return $\llbracket b' = b \rrbracket$
Game G_2 1. $\mathcal{H} \xleftarrow{\$} \text{Fcs}[\{0, 1\}^n, R_q]$ $\rho \xleftarrow{\$} \{0, 1\}^n, s \xleftarrow{\$} \chi_\alpha, e \xleftarrow{\$} \chi_\beta$ $a \xleftarrow{\$} R_q, \mathcal{H} = \mathcal{H}[\rho \rightarrow a], t \xleftarrow{\$} R_q$ $\bar{t} = \lceil t \rceil_{q \rightarrow 2^{d_t}}, pk = (\rho, \bar{t})$ 2. $(M_0^*, M_1^*) \leftarrow \mathcal{A}^{\mathcal{H}}(pk)$ 3. $b \xleftarrow{\$} \{0, 1\}, m^* = \text{Pt2poly}(M_b^*)$ $r \xleftarrow{\$} \chi_\alpha, e_1 \xleftarrow{\$} \chi_\beta, e_2 \xleftarrow{\$} \chi_\beta$ $u = a^T r + e_1, v' = \lceil \bar{t} \rceil_{2^{d_t} \rightarrow q} r + e_2$ $c_1^* := \lceil u \rceil_{q \rightarrow 2^{d_u}}, c_2^* := \lceil v' + f^{-1} m^* \rceil_{q \rightarrow 2^{d_v}}$ 4. $b' \leftarrow \mathcal{A}^{\mathcal{H}}(pk, (c_1^*, c_2^*))$ 5. return $\llbracket b' = b \rrbracket$	Game G_3 1. $\mathcal{H} \xleftarrow{\$} \text{Fcs}[\{0, 1\}^n, R_q]$ $\rho \xleftarrow{\$} \{0, 1\}^n, s \xleftarrow{\$} \chi_\alpha, e \xleftarrow{\$} \chi_\beta$ $a \xleftarrow{\$} R_q, \mathcal{H} = \mathcal{H}[\rho \rightarrow a], t \xleftarrow{\$} R_q$ $\bar{t} = \lceil t \rceil_{q \rightarrow 2^{d_t}}, pk = (\rho, \bar{t})$ 2. $(M_0^*, M_1^*) \leftarrow \mathcal{A}^{\mathcal{H}}(pk)$ 3. $b \xleftarrow{\$} \{0, 1\}, m^* = \text{Pt2poly}(M_b^*)$ $r \xleftarrow{\$} \chi_\alpha, e_1 \xleftarrow{\$} \chi_\beta, e_2 \xleftarrow{\$} \chi_\beta$ $u \xleftarrow{\$} R_q, v' \xleftarrow{\$} R_q$ $c_1^* := \lceil u \rceil_{q \rightarrow 2^{d_u}}, c_2^* := \lceil v' + f^{-1} m^* \rceil_{q \rightarrow 2^{d_v}}$ 4. $b' \leftarrow \mathcal{A}^{\mathcal{H}}(pk, (c_1^*, c_2^*))$ 5. return $\llbracket b' = b \rrbracket$

Fig. 7: Games G_0 to G_3 in the proof of Theorem 1.

Game G_1 : This game is the same as G_0 , except that a is uniformly sampled and $\mathcal{H}$ is reprogrammed to $\mathcal{H}[\rho \rightarrow a]$, a new random oracle identical to $\mathcal{H}$ but $\mathcal{H}[\rho \rightarrow a](\rho) = a$. Since this reprogramming is performed at the beginning of the game, the $\mathcal{H}$ in games G_0 and G_1 have the same distribution, and thus

$$\Pr[1 \leftarrow G_0] = \Pr[1 \leftarrow G_1]. \quad (8)$$

Game G_2 : This game is the same as G_1 , except that picking $t \xleftarrow{\$} R_q$ at random and using $\bar{t} = \lceil t \rceil_{q \rightarrow 2^{d_t}}$ as a part of pk .

According to constructions of games G_1 and G_2 , one can notice that if $\mathcal{A}$ can distinguish G_1 from G_0 , then we can construct an algorithm $\mathcal{B}_1$ with roughly the same running time as $\mathcal{A}$ to solve the $\text{ARLWE}_{n,q,\eta_1,\eta_2}$ problem. Therefore,

$$|\Pr[1 \leftarrow G_1] - \Pr[1 \leftarrow G_2]| = \text{Adv}_{n,q,\eta_1,\eta_2}^{\text{ARLWE}}(\mathcal{B}_1). \quad (9)$$

Game G₃: This game is the same as **G₂**, except that using uniformly random values to replace $u = a^T r + e_1$ and $v' = \lceil \bar{t} \rceil_{2^{d_t} \rightarrow q} r + e_2$ in generating the challenge ciphertext (c_1^*, c_2^*) .

If $\mathcal{A}$ can distinguish **G₃** from **G₂**, then we can construct an algorithm $\mathcal{B}_2$ with roughly the same running time as $\mathcal{A}$ to solve the *semi*-ARLWE $_{n,q,2^{d_t},\eta_1,\eta_2}$ problem. Specifically, given an instance $(a, \lceil t \rceil_{q \rightarrow 2^{d_t}}, u, v')$ of the semi-ARLWE problem, $\mathcal{B}_2$ aims to decide whether (u, v') is sampled from a uniform distribution in $R_q \times R_q$. Formally, $\mathcal{B}_2$ returns $pk = (\rho, \bar{t} = \lceil t \rceil_{q \rightarrow 2^{d_t}})$ to $\mathcal{A}$, and after receiving two plaintexts M_0^* and M_1^* from $\mathcal{A}$, $\mathcal{B}_2$ picks $b \xleftarrow{\$} \{0, 1\}$ at random, computes $\bar{u} = \lceil u \rceil_{q \rightarrow 2^{d_u}}, \bar{v} = \lceil v' + f^{-1} \text{Pt2poly}(M_b^*) \rceil_{q \rightarrow 2^{d_v}}$, and then gives the ciphertext $(c_1^*, c_2^*) = (\bar{u}, \bar{v})$ to $\mathcal{A}$. If (u, v') is uniformly random in $R_q \times R_q$, then $\mathcal{B}_2$ simulates as in game **G₃**. Otherwise, $\mathcal{B}_2$ simulates as in game **G₂**. Hence,

$$|\Pr[1 \leftarrow \mathbf{G}_2] - \Pr[1 \leftarrow \mathbf{G}_3]| = \text{Adv}_{n,q,2^{d_t},\eta_1,\eta_2}^{\text{semi-ARLWE}}(\mathcal{B}_2). \quad (10)$$

In the game **G₃**, the challenge plaintext M_b^* is perfectly hidden by the uniformly random v' . Therefore,

$$\Pr[1 \leftarrow \mathbf{G}_3] = \frac{1}{2}. \quad (11)$$

Combining Eq. (7) to Eq. (11), we obtain the inequality claimed in Theorem 1. $\square$

7.2 IND-CCA Security Proof of Aigis-Enc⁺

Recall that Aigis-Enc⁺ is obtained by applying a slightly tweaked Fujisaka-Okamoto (FO) transformation [16,28] to the IND-CPA secure PKE scheme Aigis-PKE⁺. In this section, we prove that Aigis-Enc⁺ is IND-CCA secure under the ARLWE and semi-ARLWE assumptions in the ROM and QROM. Formally, we have the following two theorems.

Theorem 2 *Let $\mathcal{A}$ be an IND-CCA adversary against Aigis-Enc⁺ in the ROM, making q_{H_1} , q_{H_2} , and q_{H_3} classical queries to the random oracle H_1 , H_2 , and H_3 , respectively, and making q_D classical queries to the decapsulation oracle. Then we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that*

$$\begin{aligned} \text{Adv}_{\text{Aigis-Enc}^+}^{\text{IND-CCA}}(\mathcal{A}) \leq & 2 \cdot \text{Adv}_{n,q,\eta_1,\eta_2}^{\text{ARLWE}}(\mathcal{B}_1) + 2 \cdot \text{Adv}_{n,q,2^{d_t},\eta_1,\eta_2}^{\text{semi-ARLWE}}(\mathcal{B}_2) + 2q_{H_2} \cdot \delta \\ & + \frac{2q_{H_2} + q_{H_3}}{2^\kappa}. \end{aligned}$$

Here δ is the decryption failure rate of Aigis-PKE⁺ and κ is the length of the plaintext. The running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. The full proof of this theorem is given in Appendix A.1. $\square$

Theorem 3 *Let $\mathcal{A}$ be an IND-CCA adversary against Aigis-Enc⁺ in the QROM, making q_{H_1} , q_{H_2} , and q_{H_3} quantum queries to the random oracle H_1 , H_2 , and H_3 ,*

respectively, and making q_D classical queries to the decapsulation oracle. Then we can construct two new adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that

$$\begin{aligned} \text{Adv}_{\text{Aigis-Enc}^+}^{\text{IND-CCA}}(\mathcal{A}) &\leq 4\sqrt{q \cdot \text{Adv}_{n,q,\eta_1,\eta_2}^{\text{ARLWE}}(\mathcal{B}_1)} + 4\sqrt{q \cdot \text{Adv}_{n,q,2^{dt},\eta_1,\eta_2}^{\text{semi-ARLWE}}(\mathcal{B}_2)} \\ &\quad + \frac{q + 2q_{H_3}}{\sqrt{2^\kappa}} + 16q^2 \cdot \delta. \end{aligned}$$

Here $q := 12q_{H_2} + 10q_D + 4$, δ is the decryption failure rate of Aigis-PKE^+ , and the running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$.

Proof. The full proof of this theorem is given in Appendix A.2. $\square$

8 Known Attacks

As shown in Sec. 7, Aigis-Enc^+ is provably secure under the hardness of the ARLWE problems in both the classical and quantum random oracle model. Since the best known attacks against the underlying ARLWE problems do not make use of the structure in the corresponding lattices, we therefore analyze the concrete security of the public keys and the signatures by considering the best known attacks against ALWE problems.

8.1 Attacks against ALWE

Solvers for ALWE mainly include primal attacks, dual attacks (against the underlying lattice problems) and direct solving algorithms such as BKW and Arora-Ge [3]. BKW and Arora-Ge attacks need sub-exponentially (or even exponentially) many samples, and thus they are not relevant to the public-key cryptography scenario where only a restricted amount of samples are available. Therefore, for analyzing and evaluating practical lattice-based crypto-systems, we typically consider primal attacks and dual attacks.

Since any bounded centrally symmetric distribution can be regarded as sub-gaussian for a certain parameter, for simplicity and without loss of generality, we consider the case that secret vector and error vector in $\text{ALWE}_{n,q,m,\alpha_1,\alpha_2}$ are sampled from subgaussians with parameters α_1 and α_2 respectively. Formally, the problem is to recover $\mathbf{s}$ from samples

$$(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m,$$

where $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \leftarrow \chi_{\alpha_1}^n$ and $\mathbf{e} \leftarrow \chi_{\alpha_2}^m$. In the following, we first consider the primal/dual attacks adapted to the asymmetric setting in [46], and then consider a variant of the MATZOV attack in [18] also adapted to the asymmetric setting.

8.1.1 Primal Attack. The basic intuition behind primal attack is to convert ALWE problem to the corresponding bounded distance decoding (BDD) problem or shortest vector problem by embedding. First, define a m -dimensional lattice

$$\Lambda = \{\mathbf{y} \in \mathbb{Z}^m \mid \mathbf{y} = \mathbf{A}\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\},$$

For $m > n$ the $m \times n$ matrix $\mathbf{A}$ has full rank with high probability. Assume that the first n rows of $\mathbf{A}$ are independent (otherwise swap the rows to make it), and denote by $\mathbf{A}_1 \in \mathbb{Z}_q^{n \times n}$ the submatrix by keeping the first n rows of $\mathbf{A}$, i.e.,

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_1 \\ \mathbf{A}_2 \end{pmatrix}. \text{ Denote } \mathbf{A}_1 \text{'s inverse by } \mathbf{A}_1^{-1} \in \mathbb{Z}_q^{n \times n}, \text{ and let } \mathbf{A}' = \begin{pmatrix} \mathbf{I}_n \\ \mathbf{A}_2 \mathbf{A}_1^{-1} \end{pmatrix}.$$

Then, we have

$$\begin{aligned} \Lambda &= \{\mathbf{y} | \mathbf{y} = \mathbf{A}\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\} \\ &= \{\mathbf{y} | \mathbf{y} = \mathbf{A}'\mathbf{x} \bmod q, \mathbf{x} \in \mathbb{Z}_q^n\} \subset \mathbb{Z}^m, \end{aligned}$$

and the columns of the following matrix

$$\mathbf{B} = \begin{pmatrix} \mathbf{I}_n & 0 \\ \mathbf{A}_2 \mathbf{A}_1^{-1} & q\mathbf{I}_{m-n} \end{pmatrix} \in \mathbb{Z}^{m \times m}$$

constitutes a basis of Λ . It is easy to see $\det(\Lambda) = q^{m-n}$. We have by $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$ that the distance between vector $\mathbf{b}$ and lattice Λ is $\|\mathbf{e}\|$. If we could find the closest lattice point $\mathbf{u} \in \Lambda$ to $\mathbf{b}$, then we have $\mathbf{e} = \mathbf{b} - \mathbf{u}$ with high probability. In other words, the problem can be reduced to bounded distance decoding (BDD) problem on Λ , and then one applies the nearest planes algorithm [37] to solve. Furthermore, using Kannan's embedding [36,1] one can reduce BDD to unique shortest vector problem (uSVP). In detail, consider lattice Λ' spanned by the column vectors of the following matrix:

$$\mathbf{B}' = \begin{pmatrix} \mathbf{I}_n & 0 & \mathbf{b} \\ \mathbf{A}_2 \mathbf{A}_1^{-1} & q\mathbf{I}_{m-n} & \\ 0 & 0 & t \end{pmatrix} \in \mathbb{Z}^{(m+1) \times (m+1)},$$

where $t \in \mathbb{Z}$ is an adjustable parameter. Theoretically, the algorithm works best when $t = \|\mathbf{e}\|$. However, empirical experiments suggest that best performance is achieved when $t = 1$, which is the typical value we choose. In this case, we have that $\mathbf{v} = (\mathbf{e}^T, 1)^T \in \mathbb{Z}^{m+1}$ is the unique shortest vector in Λ' with high probability. To summarize, we work on uSVP problem in Λ' to find out the error vector $\mathbf{e} \in \mathbb{Z}_q^m$, which is in turn used to solve the system of linear congruences to recover $\mathbf{s} \in \mathbb{Z}_q^n$, i.e., $\mathbf{A}\mathbf{s} = \mathbf{b} - \mathbf{e}$. Besides, since every element in vector $\mathbf{e}$ is sampled from subgaussian χ_{α_2} , we have $\|\mathbf{v}\| \approx \|\mathbf{e}\| \approx \alpha_2 \sqrt{m}$, namely, $\mathbf{v}$ is of small norm.

One can also make use of the fact that $\mathbf{s} \stackrel{\$}{\leftarrow} \chi_{\alpha_1}^n$ has small norm in practical lattice-based cryptosystems by considering a variant of the LWE embedding [9,5]. Formally, let $c = \alpha_2/\alpha_1$ and consider lattice

$$\Lambda = \left\{ \mathbf{v} = \begin{pmatrix} c\mathbf{x} \\ \mathbf{y} \\ \alpha_2 z \end{pmatrix} \in \mathbb{R}^{n+m+1} \left| (\mathbf{A}\|\mathbf{I}_m\| - \mathbf{b})\mathbf{u} = 0 \bmod q, \mathbf{u} = \begin{pmatrix} \mathbf{x} \\ \mathbf{y} \\ z \end{pmatrix} \in \mathbb{Z}^{n+m+1} \right. \right\},$$

Then the column vectors of matrix

$$\mathbf{B} = \begin{pmatrix} c\mathbf{I}_n & 0 & 0 \\ -\mathbf{A} & q\mathbf{I}_m & \mathbf{b} \\ 0 & 0 & \alpha_2 \end{pmatrix} \in \mathbb{R}^{(n+m+1) \times (n+m+1)},$$

constitutes a basis of Λ . We observe that Λ is of dimension $m + n + 1$, $\det(\Lambda) = \alpha_2 c^n q^m$, and that $\mathbf{v} = (c\mathbf{s}^T, \mathbf{e}^T, \alpha_2)^T \in \mathbb{R}^{m+n+1}$ is a short vector in Λ (and $\|\mathbf{v}\| \approx \alpha_2 \sqrt{n + m + 1}$). Therefore, we recover $\mathbf{s}$ by solving uSVP on lattice Λ .

The Cost of Primal Attack. As shown above, the computational costs of primal attack is mainly dominated by the cost of solving uSVP problem. The best known SVP solver is the BKZ- b lattice basis reduction and its variants. Given a d -dimensional basis as input, the algorithm translates the lattice basis reduction problem to the SVP problem on b -dimensional sublattice ($b < d$). BKZ- b repeats the SVP solving procedure $O(d)$ times (the actual number may depend on the specific algorithm used) on different b -dimensional sublattices to reduce the norm of the basis (of the original d -dimensional basis). Therefore, the cost of BKZ- b basis reduction is mainly decided by SVP solving algorithm for b -dimensional lattice.

Assume that the reduced basis obtained from running BKZ- b on d -dimensional lattice basis $\mathbf{B} \in \mathbb{Z}^{d \times d}$ satisfies the Geometry Series Assumption (GSA), which is the best scenario for adversary. Let $\hat{\mathbf{B}} = (\hat{\mathbf{b}}_1, \dots, \hat{\mathbf{b}}_d)$ be a reduced basis, then we have

$$\|\hat{\mathbf{b}}_1\| = \delta^d \det(\Lambda)^{1/d}, \quad \|\hat{\mathbf{b}}_i^*\| = \delta^{-2d(i-1)/(d-1)} \|\hat{\mathbf{b}}_1\|,$$

where $\delta = ((\pi b)^{1/b} \cdot b / 2\pi e)^{\frac{1}{2(b-1)}} [20]$, $\hat{\mathbf{B}}^* = (\hat{\mathbf{b}}_1^*, \dots, \hat{\mathbf{b}}_d^*)$ is the orthogonalized matrix of $\hat{\mathbf{B}}$ (i.e., $\hat{\mathbf{b}}_1 = \hat{\mathbf{b}}_1^*$). Specifically, the literatures [5,1] show that when the norm of the projection of the unique shortest vector $\mathbf{v}$ into the space spanned by the last b orthogonal vectors $(\hat{\mathbf{b}}_{d-b+1}^*, \dots, \hat{\mathbf{b}}_d^*)$ is less than $\|\hat{\mathbf{b}}_{d-b+1}^*\|$, then the BKZ- b lattice basis reduction recovers $\mathbf{v}$.

The norm of $\mathbf{v}$'s projection into the space spanned by b orthogonal vectors $(\hat{\mathbf{b}}_{d-b+1}^*, \dots, \hat{\mathbf{b}}_d^*)$ approximately equals $\ell \sqrt{b/d}$, where $\ell = \|\mathbf{v}\|$. That is, we assume every component of vector $\mathbf{v}$'s projection is of roughly the same size. In this case, the computational cost of primal attack is mainly decided by the minimal value of b that satisfies the constraint

$$\ell \sqrt{b/d} \leq \delta^{(-d^2+2db-d)/(d-1)} \det(\Lambda)^{1/d}. \quad (12)$$

Following [5], our estimation of the complexity of solving uSVP on d -dimensional lattice is based on that of solving SVP on b -dimensional lattice, where b satisfy inequality (12) (namely, the Core-SVP model). Furthermore, we use $\text{cost}_b = 2^{0.292b}$ [5,13] and $\text{cost}_b = 2^{0.257b}$ [19] to estimate the complexities of classical and quantum algorithms (for solving SVP on b -dimensional lattice) respectively.

8.1.2 Dual Attack. Dual attack translates the LWE problem to a ℓ_2 -norm SIS problem, whose solution is used to solve the decisional LWE by distinguishing

between subgaussians (for certain parameters) and uniform distribution over $\mathbb{Z}_q^n$. The first dual attack was given in [39], which only cares about the error distribution of the LWE-like problem, and thus does not distinguish between LWE and ALWE. In other words, the original dual attack solves ALWE the same way as it does to LWE, and it appears more efficient when $\alpha_1 \gg \alpha_2$. Formally, let

$$\Lambda = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}^T \mathbf{x} = \mathbf{0} \bmod q\},$$

be a lattice of dimension $d = m \gg n$. Note that $\mathbf{A}$ has full rank with high probability and we assume the submatrix $\mathbf{A}_1 \in \mathbb{Z}_q^{n \times n}$ made up by the first n columns of $\mathbf{A}^T$ has rank n and $\mathbf{A}^T = (\mathbf{A}_1 \parallel \mathbf{A}_2) \in \mathbb{Z}_q^{n \times m}$. Let $\mathbf{A}' = (\mathbf{I}_n \parallel \mathbf{A}_1^{-1} \mathbf{A}_2)$. Then, we have that

$$\Lambda = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}^T \mathbf{x} = \mathbf{0} \bmod q\} = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}' \mathbf{x} = \mathbf{0} \bmod q\},$$

and the column vectors of

$$\mathbf{B} = \begin{pmatrix} q\mathbf{I}_n & -\mathbf{A}_1^{-1} \mathbf{A}_2 \\ 0 & \mathbf{I}_{m-n} \end{pmatrix} \in \mathbb{Z}^{m \times m}$$

constitute a basis of lattice Λ and $\det(\Lambda) = q^n$. The dual attack first solves the SIS problem by finding out a short vector $\mathbf{v} \in \mathbb{Z}^m$ with $\mathbf{A}^T \mathbf{v} = \mathbf{0} \bmod q$ in lattice Λ with basis $\mathbf{B}$. It then computes $u = \langle \mathbf{v}, \mathbf{b} \rangle = \langle \mathbf{v}, \mathbf{e} \rangle \bmod q$ in order to distinguish between subgaussian and uniform distributions. Specifically, if the norm $\ell = \|\mathbf{v}\|$ is relatively small, then the value of $\langle \mathbf{v}, \mathbf{e} \rangle$ is relatively small too and u can be seen as random variable drawn from subgaussian distribution with standard variance $\ell \alpha_2$, which can be distinguished from uniform distribution on $\mathbb{Z}_q$ with probability $4 \exp(-2\pi^2 \tau^2)$, where $\tau = \ell \alpha_2 / q$.

Similar as for primal attacks, we can also make use of the fact that $\mathbf{s} \stackrel{\$}{\leftarrow} \chi_{\alpha_1}^n$ has small norm [5] to optimize the dual attack. Formally, define lattice

$$\Lambda = \{(\mathbf{c}\mathbf{x}^T, \mathbf{y}^T)^T \in \mathbb{R}^{m+n} \mid \mathbf{A}^T \mathbf{x} = \mathbf{y} \bmod q, \mathbf{x} \in \mathbb{Z}^m, \mathbf{y} \in \mathbb{Z}^n\},$$

where Λ has dimension $d = m + n$ and $c = \frac{\alpha_2}{\alpha_1}$. Then, the column vectors of matrix

$$\mathbf{B} = \begin{pmatrix} c\mathbf{I}_m & 0 \\ \mathbf{A}^T & q\mathbf{I}_n \end{pmatrix} \in \mathbb{R}^{(m+n) \times (m+n)}$$

constitutes a basis of lattice Λ with $\det(\Lambda) = c^m q^n$. Likewise, we find out a short vector $\mathbf{v} = (\mathbf{c}\mathbf{x}^T, \mathbf{y}^T)^T \in \mathbb{R}^{m+n}$ with $\mathbf{A}^T \mathbf{x} = \mathbf{y} \bmod q$ from $(m+n)$ -dimensional lattice Λ with basis $\mathbf{B}$, and then compute

$$u = \langle \mathbf{x}, \mathbf{b} \rangle = \langle \mathbf{y}, \mathbf{s} \rangle + \langle \mathbf{c}\mathbf{x}, \mathbf{e}/c \rangle \bmod q,$$

and therefore distinguish the above u from uniform. Specifically, the values of $\langle \mathbf{y}, \mathbf{s} \rangle$ and $\langle \mathbf{x}, \mathbf{e} \rangle$ are relatively small for small $\ell = \|\mathbf{v}\|$. Thus, $u = \langle \mathbf{y}, \mathbf{s} \rangle + \langle \mathbf{c}\mathbf{x}, \mathbf{e}/c \rangle$ can be almost regarded as following subgaussian distribution with parameter $\ell \alpha_1$. This enable efficient algorithms that distinguish the subgaussian-distributed u from uniform with probability $4 \exp(-2\pi^2 \tau^2)$, where $\tau = \ell \alpha_1 / q$.

The Cost of Dual Attack. As shown above, the computational costs of dual attack is mainly dominated by the cost of solving SIS problem and transforming distinguishing attacks into secret recovery attacks (with success probability more than half) for the ALWE problem. We use the model of primal attack to estimate the cost of solving SIS problem. That is, applying BKZ- b to reduce the basis of d -dimensional lattice yields a short vector $\mathbf{v}$ of norm $\ell = \|\mathbf{v}\| = \delta^d \det(\Lambda)^{1/d}$. Substituting this into the previous analysis, we distinguish ALWE problem with advantage $\epsilon = 4 \exp(-2\pi^2 \tau^2)$, where τ is fully decided by ℓ and the actual dual attack algorithm. To obtain the final secret recovery attack with probability greater than $1/2$, we need $1/\epsilon^2$ short vectors. Taking into account that each invocation of the sieving yields $2^{0.2075b}$ short vectors, we need to repeat it $R = 1/\max(1, 1/(2^{0.2075b} \epsilon^2))$ times.

Following the cost estimation framework for primal attacks, we adopt the Core-SVP model, which uses the complexity of solving SVP on a b -dimensional lattice as a (conservative) estimate for solving SVP on a d -dimensional lattice. Specifically, we set $\text{cost}_b = 2^{0.292b}$ for classical algorithms and $\text{cost}_b = 2^{0.257b}$ for quantum algorithms when solving SVP on b -dimensional lattices. Under this model, the complexity of dual attack is estimated as

$$1/\max(1, 1/(2^{0.2075b} \cdot 16 \exp(-4\pi^2 \tau^2))) \cdot \text{cost}_b, \quad (13)$$

where τ is decided by b and other parameters of the actual dual attack.

8.1.3 Refined MATZOV Attack (MATZOV[†]). The MATZOV attack given in [38] is basically a variant of the hybrid-dual attack which splits the attack into three parts: a BKZ lattice reduction stage, a guessing stage and a FFT-based distinguishing stage. Formally, given a LWE problem

$$(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m,$$

where $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \leftarrow \chi_{\alpha_1}^n$ and $\mathbf{e} \leftarrow \chi_{\alpha_2}^m$. Let $n_{\text{lat}}, n_{\text{enu}}, n_{\text{fft}}$ be integers satisfying $n = n_{\text{lat}} + n_{\text{enu}} + n_{\text{fft}}$. The matrix $\mathbf{A}$ and secret $\mathbf{s}$ can be divided accordingly:

$$\mathbf{A} = [\mathbf{A}_{\text{enu}}, \mathbf{A}_{\text{fft}}, \mathbf{A}_{\text{lat}}] \in \mathbb{Z}_q^{m \times n_{\text{enu}}} \times \mathbb{Z}_q^{m \times n_{\text{fft}}} \times \mathbb{Z}_q^{m \times n_{\text{lat}}},$$

$$\mathbf{s} = \begin{pmatrix} \mathbf{s}_{\text{enu}} \\ \mathbf{s}_{\text{fft}} \\ \mathbf{s}_{\text{lat}} \end{pmatrix} \in \mathbb{Z}_q^{n_{\text{enu}}} \times \mathbb{Z}_q^{n_{\text{fft}}} \times \mathbb{Z}_q^{n_{\text{lat}}}.$$

The attack works by first performing BKZ lattice reduction on the lattice generated by

$$\mathbf{B}_{\text{lat}} = \begin{pmatrix} \mathbf{I}_m & \mathbf{0} \\ \mathbf{A}_{\text{lat}}^T & q\mathbf{I}_{n_{\text{lat}}} \end{pmatrix} \in \mathbb{Z}_q^{(m+n_{\text{lat}}) \times (m+n_{\text{lat}})}$$

to obtain a set $\mathcal{S}$ of N vectors $(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}_q^{m+n_{\text{lat}}}$ with small Euclidean norm such that $\mathbf{y} = \mathbf{A}_{n_{\text{lat}}}^T \mathbf{x}$. Then, for each vector $(\mathbf{x}, \mathbf{y}) \in \mathcal{S}$, by the equation $\mathbf{b} = \mathbf{A}_{\text{enu}} \mathbf{s}_{\text{enu}} + \mathbf{A}_{\text{fft}} \mathbf{s}_{\text{fft}} + \mathbf{A}_{\text{lat}} \mathbf{s}_{\text{lat}} + \mathbf{e}$ we have that

$$\underbrace{\langle \mathbf{x}, \mathbf{b} - \mathbf{A}_{\text{enu}} \mathbf{s}_{\text{enu}} \rangle}_{= \hat{b}} = \underbrace{\langle \mathbf{x}, \mathbf{A}_{\text{fft}} \mathbf{s}_{\text{fft}} \rangle}_{= \langle \hat{\mathbf{a}}, \mathbf{s}_{\text{fft}} \rangle} + \underbrace{\langle \mathbf{x}, \mathbf{e} \rangle + \langle \mathbf{y}, \mathbf{s}_{\text{lat}} \rangle}_{= \hat{e}}, \quad (14)$$

where $\hat{\mathbf{a}} = \mathbf{A}_{\text{fft}}^T \mathbf{x}$, $\hat{b} = \mathbf{x}^T (\mathbf{b} - \mathbf{A}_{\text{enu}} \mathbf{s}_{\text{enu}})$ and $\hat{e} = \mathbf{x}^T \mathbf{e} + \mathbf{y}^T \mathbf{s}_{\text{lat}}$. Clearly, if $(\mathbf{e}, \mathbf{s}_{\text{lat}})$ has small Euclidean norm, then $\hat{e}$ is small. Thus, by correctly guessing $\mathbf{s}_{\text{enu}}$, we can obtain a new LWE sample $\hat{b} = \hat{\mathbf{a}}^T \mathbf{s}_{\text{fft}} + \hat{e}$ in much smaller dimension $n_{\text{fft}} \ll n$ for each $(\mathbf{x}, \mathbf{y}) \in \mathcal{S}$. Finally, the attack ends by running a FFT-based distinguishing algorithm over $\mathbb{Z}_q^{n_{\text{fft}}}$ which can determine whether a guessed value for $\mathbf{s}_{\text{enu}}$ is correct or not. The core improvement of the MATZOV attack over previous hybrid-dual attacks such as [30] is the introduction of modulus-switching to speedup the FFT-based distinguishing stage, which greatly reduces the FFT cost from previous $O(n_{\text{fft}} q^{n_{\text{fft}}})$ to $O(n_{\text{fft}} p^{n_{\text{fft}}})$ for some much smaller $p \ll q$ by switching the obtained LWE samples in Equation (14) from $\mathbb{Z}_q^{n_{\text{fft}}}$ to $\mathbb{Z}_p^{n_{\text{fft}}}$. The main problem of the MAZOV attack is the requirement of some controversial assumptions in the complexity analysis [24,43].

Recently, Carrier et al. [18] replaced the modulus switching part of the MATZOV attack with a lossy source/quantizing approach, and obtained a variant that is supported by experimentally validated assumptions. Specifically, given integers $k_{\text{fft}} \ll n_{\text{fft}}$ and a linear code generated by some $\mathbf{G} \in \mathbb{Z}_q^{n_{\text{fft}} \times k_{\text{fft}}}$ such that for each $\mathbf{A}_{\text{fft}}^T \mathbf{x}$ there exists a vector $\mathbf{u} \in \mathbb{Z}_q^{k_{\text{fft}}}$ and a small vector $\mathbf{e}_{\mathbf{x}} \in \mathbb{Z}_q^{n_{\text{fft}}}$ satisfying $\mathbf{A}_{\text{fft}}^T \mathbf{x} = \mathbf{G} \mathbf{u} + \mathbf{e}_{\mathbf{x}}$, we can rewrite Equation (14) as follows:

$$\underbrace{\langle \mathbf{x}, \mathbf{b} - \mathbf{A}_{\text{enu}} \mathbf{s}_{\text{enu}} \rangle}_{= \hat{b}} = \underbrace{\langle \mathbf{u}, \mathbf{G}^T \mathbf{s}_{\text{fft}} \rangle}_{= \langle \mathbf{u}, \mathbf{s}'_{\text{fft}} \rangle} + \underbrace{\langle \mathbf{x}, \mathbf{e} \rangle + \langle \mathbf{y}, \mathbf{s}_{\text{lat}} \rangle + \langle \mathbf{e}_{\mathbf{x}}, \mathbf{s}_{\text{fft}} \rangle}_{= \hat{e}'}. \quad (15)$$

Now, the attack can be finished by running the FFT-based distinguishing algorithm over $\mathbb{Z}_q^{k_{\text{fft}}}$, and the FFT cost is reduced from previous $O(n_{\text{fft}} q^{n_{\text{fft}}})$ to $O(k_{\text{fft}} q^{k_{\text{fft}}})$. In [18], Carrier et al. demonstrated that their variant of the MATZOV attack is not only supported by experimental evidence but also outperforms the original MATZOV attack [38]. Thus, for more conservative concrete security estimates, one should consider the MATZOV attack in [18], as also remarked in [29].

Since Carrier et al.'s attack [18] only considers symmetric distributions (namely, $\alpha_1 = \alpha_2$), we slightly adapt their attack to handle asymmetric distributions. Formally, let $c = \alpha_2/\alpha_1$, we scale the lattice used in the BKZ lattice reduction stage as follows:

$$\mathbf{B}'_{\text{lat}} = \begin{pmatrix} c\mathbf{I}_m & \mathbf{0} \\ \mathbf{A}_{\text{lat}}^T & q\mathbf{I}_{n_{\text{lat}}} \end{pmatrix} \in \mathbb{R}^{(m+n_{\text{lat}}) \times (m+n_{\text{lat}})}.$$

By doing this, we can model the distribution of $\hat{e}'$ in Equation (15) as a Gaussian distribution with parameter $\ell\alpha_1$, where $\ell = \|(c\mathbf{x}, \mathbf{y}, \mathbf{e}_{\mathbf{x}})\|_2$ for each $\mathbf{y} = \mathbf{A}_{n_{\text{lat}}}^T \mathbf{x}$. For ease of presentation, we refer to the above refined MATZOV attack as MATZOV[†].

The Cost of MATZOV[†] Attack. As shown in [18], the total cost of the MATZOV[†] attack depends on the cost T_{sample} for obtaining the set $\mathcal{S}$ of N short vectors, R for the expected number of times to guess $\mathbf{s}_{\text{enu}}$ correctly, T_{dec} for decoding $\mathbf{e}_x$ from $\mathbf{A}_{\text{fft}}^T \mathbf{x}$, and T_{fft} for running the FFT-based distinguishing algorithm, namely,

$$T_{\text{cost}} = T_{\text{sample}} + R \cdot (N \cdot T_{\text{dec}} + T_{\text{fft}}),$$

where $T_{\text{sample}} = 2^{0.292b}$ and $N = 2^{0.2075b}$ for performing SVP sieving algorithm in b -dimensional sublattice. For quantum security estimates, in addition to considering quantum SVP sieving cost $T'_{\text{sample}} = 2^{0.257b}$, we additionally assume that the guessing-then-FFT part can achieve a quantum quadratic speedup [2], namely,

$$T'_{\text{cost}} = T'_{\text{sample}} + \sqrt{R \cdot (N \cdot T_{\text{dec}} + T_{\text{fft}})}.$$

We note that this will only make our concrete estimates more conservative. Because T_{dec} and T_{fft} depend strongly on the modulus size, and R on the sparsity of the secret vector, the above analysis immediately imply that the MATZOV[†] attack may be particularly advantageous for sparse secrets and small modulus.

8.2 Concrete Security Estimates against Lattice Attacks

The complexity of the attacks depends on the attack type, the number of samples m , and the block size $b \in \mathbb{Z}$ used in the BKZ- b algorithm. Therefore, to obtain a conservative overall security estimation for Aigis-Enc⁺ under the three recommended parameter sets, we enumerate all possible values of m and b . In details, Table 6 estimate the complexities of RLWE problem against the aforementioned attacks, where the concrete security estimates against MATZOV[†] are derived from a code that modifies the publicly available implementation by Carrier et al. [18], which can be found at <https://github.com/kevin-carrier/CodedDualAttack>. For comparison, we also give the figures against the original MATZOV attack [38] given by the LWE estimator [3]. As shown in 6, the MATZOV[†] attack almost dominates the concrete estimates for all parameters.

Table 6: The Concrete Security Estimates of Aigis-Enc⁺ (where C and Q denotes classical and quantum security, respectively)

Parameter Set	LWE Hardness (m, b, sec)			
	Primal	Dual	MATZOV	MATZOV [†]
Aigis-Enc ⁺ -512	C: (509,445,130)	C: (510,440,128)	C: (512,459,134)	C: (508,438,130)
	Q: (509,445,114)	Q: (510,440,113)	Q: (512,462,118)	Q: (511,417,108)
Aigis-Enc ⁺ -1024	C: (844,936,273)	C: (853,928,271)	C: (1024,885,258)	C: (818,907,266)
	Q: (844,936,240)	Q: (853,928,238)	Q: (1024,894,229)	Q: (778,842,217)
Aigis-Enc ⁺ -2048	C: (1545,1958,572)	C: (1556,1945,568)	C: (2048,1832,535)	C: (1466,1765,516)
	Q: (1545,1958,503)	Q: (1556,1945,499)	Q: (2048,1857,477)	Q: (1383,1630,420)

8.3 Attacks Exploiting Decryption Failures

In line with standard practice for practical lattice-based schemes such as Kyber [13], we select concrete parameters that achieve a negligible decryption failure probability to ensure compactness. Although our estimation model is quite conservative as discussed in Sec. 3.2, it only considers the scenario where all random coins are honestly sampled from the intended distributions. In practice, however, an adversary may depart from this setting and search across many different values of M until finding one for which the induced random coins ρ result in a decryption failure, thus leaking information about the secret key. More alarmingly, after triggering the first failure, the adversary can exploit the directional failure boosting attack [23] to obtain additional decryption failures at a significantly reduced cost. Consequently, to remain on the safe side, one must ensure that even a single failure is difficult to provoke.

We follow the approach of [13] to demonstrate that, for our concrete parameters in Table 1, the corresponding probability of decryption failures is sufficiently low to resist attacks that exploit such failures. Our analysis crucially relies on the simple decryption formula of Aigis-Enc⁺. Formally, from inequalities (4) and (5) in Sec. 3.2, the decryption failure of Aigis-Enc⁺ is determined by the term

$$\begin{aligned} p_j &= \sum_y \Pr \left[\langle \hat{w}_j, y \rangle \geq \frac{k \cdot (q-1)}{4} \right] \\ &= \sum_y \Pr \left[\left\langle \sum_{i \in [n/k]} (\hat{e}'_i \hat{r}_i + \hat{e}''_i \hat{s}_i) + \hat{e}'''_j, y \right\rangle \geq \frac{k \cdot (q-1)}{4} \right], \end{aligned}$$

where the probability is taken over the random choice of $e' = e - c_t$, $e'' = e_1 - c_u$, $e''' = e_2 - c_v$, $\hat{r}_i, \hat{s}'_i \stackrel{\$}{\leftarrow} \chi_\alpha$, $e, e_1, e_2 \stackrel{\$}{\leftarrow} \chi_\beta$. For a fixed y , let

$$p_{j,y} = \Pr \left[\left\langle \sum_{i \in [n/k]} (\hat{e}'_i \hat{r}_i + \hat{e}''_i \hat{s}_i) + \hat{e}'''_j, y \right\rangle \geq \frac{k \cdot (q-1)}{4} \right].$$

and $t_y = \left\langle \sum_{i \in [n/k]} (\hat{e}'_i \hat{r}_i + \hat{e}''_i \hat{s}_i), y \right\rangle$. Clearly, the probability of $p_{j,y}$ is primarily determined by the value of t_y . Moreover, let $\hat{\mathbf{R}}_i, \hat{\mathbf{E}}''_i$ be the coefficient matrices of $\hat{r}_i, \hat{e}''_i$ defined via multiplication over the subring $S = \mathbb{Z}[Y]/(Y^k + 1)$, and let $\hat{\mathbf{e}}'_i, \hat{\mathbf{s}}_i, \mathbf{y}$ be the coefficient vectors of $\hat{e}'_i, \hat{s}_i, y$ in column representation. Then,

$$t_y = \left\langle \sum_{i \in [n/k]} (\hat{e}'_i \hat{r}_i + \hat{e}''_i \hat{s}_i), y \right\rangle = \left\langle \sum_{i \in [n/k]} (\hat{\mathbf{R}}_i \hat{\mathbf{e}}'_i + \hat{\mathbf{E}}''_i \hat{\mathbf{s}}_i), \mathbf{y} \right\rangle \quad (16)$$

Now, assume that $\hat{\mathbf{e}}'$ (with distribution χ_e) and $\hat{\mathbf{s}}$ (with distribution χ_s) are subgaussian random variables with parameters σ_e and σ_s , respectively. By the properties of subgaussian distributions, we have $\frac{\sigma_s}{\sigma_e} \chi_e$ is subgaussian with parameter σ_s . Define

$$\mathbf{v}_{i,y} = \mathbf{y}^t \left(\frac{\sigma_s}{\sigma_e} \hat{\mathbf{R}}_i \hat{\mathbf{E}}''_i \right) \in \mathbb{Z}^{2k}.$$

Using the identity

$$\begin{aligned}
t_y &= \left\langle \sum_{i \in [n/k]} (\hat{\mathbf{R}}_i \hat{\mathbf{e}}'_i + \hat{\mathbf{E}}''_i \hat{\mathbf{s}}_i), \mathbf{y} \right\rangle \\
&= \mathbf{y}^T \begin{pmatrix} \frac{\sigma_e}{\sigma_s} \hat{\mathbf{R}}_1 \dots \frac{\sigma_e}{\sigma_s} \hat{\mathbf{R}}_{n/k} & \hat{\mathbf{E}}''_1 \dots \hat{\mathbf{E}}''_{n/k} \end{pmatrix} \begin{pmatrix} \frac{\sigma_s}{\sigma_e} \hat{\mathbf{e}}'_1 \\ \vdots \\ \frac{\sigma_s}{\sigma_e} \hat{\mathbf{e}}'_{n/k} \\ \hat{\mathbf{s}}_1 \\ \vdots \\ \hat{\mathbf{s}}_{n/k} \end{pmatrix}, \tag{17}
\end{aligned}$$

we have

$$\Pr[t_y > \kappa \sigma \|\mathbf{v}_y\|] \leq e^{-\kappa^2/2} \tag{18}$$

by concentration inequalities for subgaussian variables [45,5]. For each $y \in \{0, 1\}^k$, we compute $p_{j,y}$ numerically and denote the largest such value by $p_{j,y}^* = \max_y p_{j,y}$. Setting the right-hand side of inequality (18) equal to $p_{j,y}^*$ gives a specific value $\kappa = \kappa^*$ (reported in Table 7).

Note that the adversary can make at most a bounded number N of decryption queries—for example, $N = 2^{80}$ as specified in the NGCC Call for Proposals [32]. To obtain a decryption failure, the adversary must increase $\|\mathbf{v}_y\|$ by a factor c (e.g., by appropriately choosing (r, e_1)). This factor c reduces κ^* by the same factor in inequality (18), so that the resulting failure probability becomes at least $1/N$. Let c^* denote the threshold value satisfying

$$e^{-(\kappa^*/c^*)^2/2} = 2^{-80 - \log(n/k) - k}, \tag{19}$$

where the terms $\log(n/k) + k$ account for the union bound over $y \in \{0, 1\}^k$ in equation (4) and over $j \in [n/k]$ in equation (5). This union bound is introduced artificially for the analysis of decryption failures and only relaxes the conditions under which the adversary can trigger a decryption failure within $N = 2^{80}$ queries. The resulting values of c^* for our concrete parameters are listed in Table 7.

Now it suffices to estimate the probability of obtaining a vector $\mathbf{v}_y$ whose norm is c times larger than the expected value. Let

$$\mathbf{v}_{i,y} = \mathbf{y}^T \begin{pmatrix} \frac{\sigma_e}{\sigma_s} \hat{\mathbf{R}}_i & \hat{\mathbf{E}}''_i \end{pmatrix} \in \mathbb{Z}^{2k}.$$

All $\mathbf{v}_{i,y}$ are identically distributed, and

$$\|\mathbf{v}_y\|^2 = \sum_{i \in \{1, \dots, n/k\}} \|\mathbf{v}_{i,y}\|^2.$$

Let D_y be the distribution of $\|\mathbf{v}_{i,y}\|^2$ with mean μ . Let $M_{D_y}(\theta) = \mathbb{E}_{X \sim D_y} e^{\theta X}$ the moment-generating function of D_y , assumed finite for some $\theta > 0$. Then, by the Chernoff-Cramer inequality [14], we have that

$$\Pr[\|\mathbf{v}_y\|^2 \geq c^2 \cdot n\mu/k] \leq e^{-n(c^2\mu\theta - \log(M_{D_y}(\theta)))/k} \tag{20}$$

We perform a numerical computation of $\|\mathbf{v}_{i,y}\|^2$ to determine the distribution (D_y, μ) . For the subsequent analysis, we fix y (and thus (D_y, μ)) that maximizes the probability in equation (20) — this choice represents the most favorable conditions for the adversary’s attack. Finally, the total cost for the adversary to trigger a single decryption failure is given by

$$\min_{c \geq c^*} \frac{1}{e^{-(\kappa^*/c)^2/2} \cdot e^{-n(c^2\mu\theta - \log(M_{D_y}(\theta)))/k}}$$

in the classical setting, and

$$\min_{c \geq c^*} \frac{1}{e^{-(\kappa^*/c)^2/2} \cdot \sqrt{e^{-n(c^2\mu\theta - \log(M_{D_y}(\theta)))/k}}}$$

in the quantum setting. Here, we directly assume that Grover’s algorithm yields a square-root speed-up for quantum search, which only makes our analysis more conservative.

We develop a Python script to compute the adversary’s optimal cost—attained at $c = c^*$ for all parameter sets—and report the resulting values in Table 7. In this table, θ is the constant appearing in the Chernoff–Cramér inequality, μ^* denotes the optimal μ for the adversary’s attack, and C and Q represent the minimum classical and quantum security bits, respectively. These results confirm that our concrete parameters maintain the claimed security levels against attacks that exploit decryption failures.

Table 7: Attacks Exploiting Decryption Failures

Parameters	Dec. Failure	$p_{j,y}^*$	κ^*	c^*	μ^*	θ	Costs (C,Q)
Aigis-Enc ⁺ -512	2^{-145}	2^{-156}	14.75	1.31	125.56	0.009	(172, 126)
Aigis-Enc ⁺ -1024	2^{-145}	2^{-157}	14.80	1.31	125.56	0.01	(275, 177)
Aigis-Enc ⁺ -2048	2^{-157}	2^{-170}	15.40	1.36	77.57	0.018	(605, 342)

8.4 Attacks against Symmetric Primitives

The overall security of Aigis-Enc⁺ relies not only on the hardness of the underlying mathematical problems, but also closely on the security of several symmetric primitives it invokes, such as collision-resistant hash functions and pseudorandom number generators. In Sec. 5.2, we instantiate these primitives with SHA3. This instantiation rests on a clear security assumption: SHA3 provides the required collision resistance, preimage resistance, and pseudorandomness at the level matching our concrete security parameter in both the classical and quantum setting.

Crucially, Aigis-Enc⁺ is designed and proven secure by treating these symmetric primitives as idealised black boxes throughout the security reduction. This means that the proof does not depend on the internal details or specific algebraic structure of SHA3, but only on its external input-output behaviour and the expected

security metrics. Such a highly modular abstraction offers strong cryptographic agility: in the future, should any major cryptanalytic breakthrough against SHA3 emerge—for instance, collision or preimage attacks significantly more efficient than anticipated—that would undermine the prescribed security strength in either the classical or quantum setting, we would not need to redesign the core public-key framework of **Aigis-Enc⁺** nor invalidate its existing security reduction. In practice, one would simply replace SHA3 with another well-analysed, standardised algorithm that provides an equivalent interface, and the overall scheme architecture and core logic would remain unchanged, thereby swiftly restoring and maintaining the actual security margin of the scheme.

9 Side-Channel Vulnerabilities and Countermeasures

9.1 Security Analysis

To the best of our knowledge, existing design and implementation of **Aigis-Enc⁺** follow a constant-time design paradigm, where the execution time is independent of secret-dependent values. Consequently, timing-based side-channel leakage is not expected to be exploitable in practice.

Regarding physical side channels such as power consumption and electromagnetic leakage, although there exist constructions that aim to achieve leakage resilience, they often incur prohibitive overhead and are therefore impractical for real-world deployments. Instead, we argue that having a clear and systematic understanding of the inherent side-channel vulnerabilities of the algorithm, together with targeted and efficient countermeasures, is of primary importance for achieving practical security.

9.1.1 Key Generation Phase. The initial seed is processed via a 512-bit hash function to derive s . Then, $t = As + e$ is computed in the NTT domain. Hence, both seed and s are sensitive and require protection. The key generation phase must be protected using masking or equivalent countermeasures, as leakage at this stage may lead to full recovery of the secret key. Figure 8 illustrate **Aigis-Enc⁺** key generation flow, where red denotes sensitive information and operations, and blue denotes non-sensitive information.

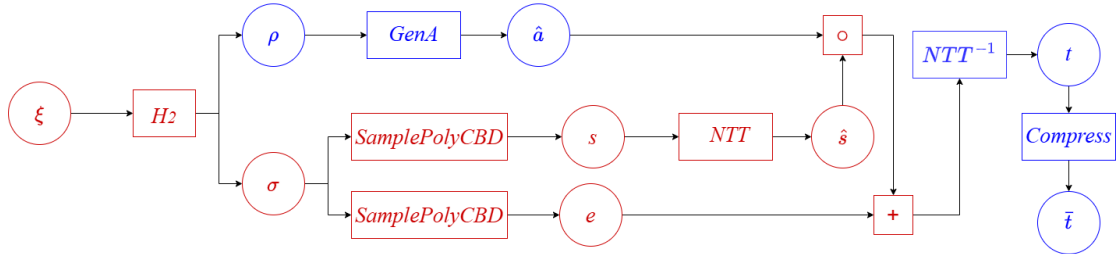


Fig. 8: **Aigis-Enc⁺** key-generation procedure and side-channel vulnerabilities

9.1.2 Decapsulation Phase. The secret key is involved in the inner product $s^T u$, whose result propagates through subsequent computations. Specifically, one computes $t' = v - s^T u$. followed by a coefficient-wise compression $m' = \text{Compress}_q(t', 1)$. Accordingly, the sensitive data flow can be summarized as follows:

$$s \rightarrow s^T u \rightarrow v - s^T u \rightarrow \text{Compress}_q(\cdot) \rightarrow m'.$$

The decapsulation process consists of ciphertext decryption and re-encryption verification. Operations depending only on public ciphertext do not require masking, whereas all computations involving the secret key must be protected. In the verification phase, all operations must be masked since the secret key is involved throughout. Figure 9 illustrate Aigis-Enc⁺ decapsulation flow, where red denotes sensitive information and operations, and blue denotes non-sensitive information.

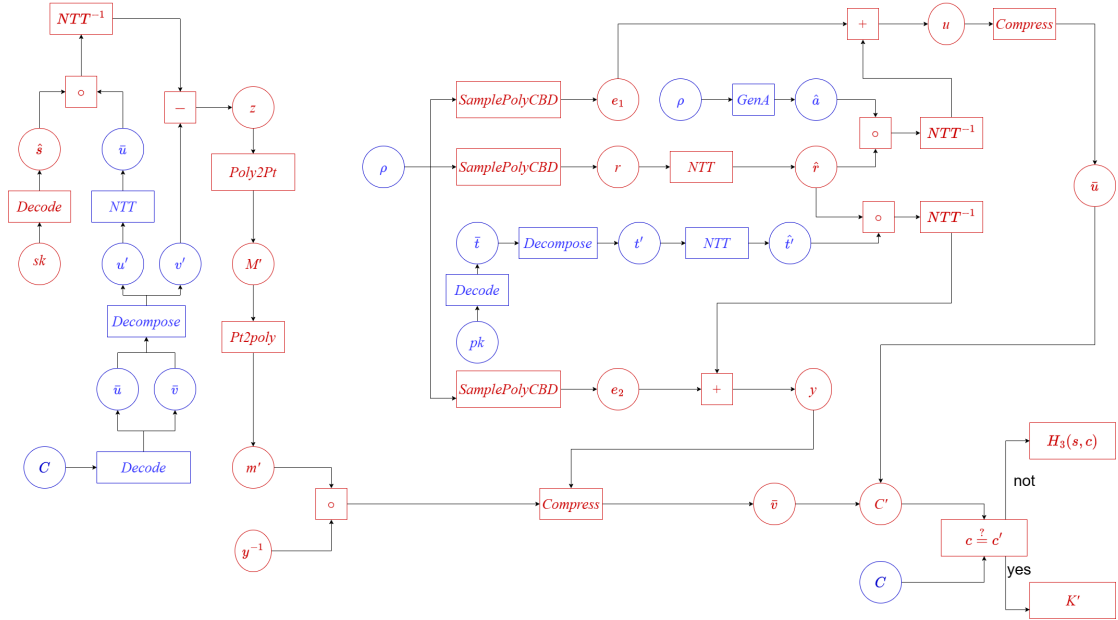


Fig. 9: Aigis-Enc⁺ decapsulation procedure and side-channel vulnerabilities.

9.2 A Discussion on Masking Countermeasures

It is worth noting that Aigis-Enc⁺ shares substantial structural and implementation-level similarities with ML-KEM (e.g., Kyber). This close correspondence enables existing masking techniques and design methodologies developed for ML-KEM to be systematically transferred and adapted to the Aigis-Enc⁺ setting. Linear operations (e.g., polynomial addition, NTT, and point-wise multiplication) can be performed in a share-wise manner. In contrast, we focus on nonlinear components, which are not homomorphic under additive masking and therefore require dedicated secure constructions. The main components that require dedicated masking redesign include CBD, Compress, and Decompress. In the remainder of this subsection, we denote by $\mathfrak{d}$ the masking order.

9.2.1 Masked CBD Function. The CBD_η function maps random bits to polynomial coefficients following a discrete Gaussian-like distribution. The standard method computes $f_i = a - b$, where a and b are Hamming weights of bit partitions (say, x and y). The main challenge of Masked CBD Function is the conversion from Boolean-masked inputs to arithmetic outputs.

A straightforward approach is proposed in [41]. It first applies multiple Boolean-to-Arithmetic (B2A) conversions to map the input into the arithmetic domain, where the Hamming weight is subsequently computed. More precisely, for a k -bit Boolean-masked input with two shares (x_1, x_2) , the bits are processed individually. For each bit position j , a one-bit B2A conversion is performed to transform the j -th bit from Boolean masking to arithmetic masking, yielding arithmetic shares. The masked Hamming weight is then obtained by summing over all k bit positions. This approach requires $2k$ B2A calls and incurs a complexity of $O(k\mathfrak{d}^2)$. A more efficient method is introduced in [44]. Leveraging a bit-slicing technique, it computes the Hamming weight directly in the Boolean domain, and performs only a single B2A conversion at the end, with input bit-width $\lambda = \log_2(k + 1) + 1$, to convert the result into the arithmetic domain. This approach achieves a reduced complexity of $O(\mathfrak{d}^2 \log_2 k)$.

9.2.2 Masked Compression. The function $\text{Compress}_d(f)$ is used in Aigis-Enc^+ encryption to compress the values generated during encryption before encoding them into the ciphertext. The essence of this function is to perform a rounding operation on each coefficient of the polynomial, where $f[i] := \lceil f[i] \cdot (q/2^d) \rceil$. This is equivalent to computing $\left\lfloor \frac{2^d f[i] + \frac{q}{2}}{q} \right\rfloor$. When implementing this process in a masked setting, the main technical challenge is that additions under arithmetic masking may introduce accumulated rounding errors, thereby compromising the correctness of the computation. To address the above issue, Coron et al. [21] proposes a technical approach based on increasing computational precision to mitigate rounding errors. Specifically, it introduces α bits of additional precision during the intermediate computations. The arithmetic masking modulo q is first mapped to a higher-precision domain modulo $2^{d+\alpha}$, and then reduced back to modulo 2^d through truncation. The overhead complexity of masked Compress_d for each coefficient is $O(\mathfrak{d}^2 \log(d + \alpha))$ with $\alpha = \lceil \log_2(q \cdot \mathfrak{d}) \rceil$.

9.2.3 Masked Poly2Pt The Poly2Pt module converts a polynomial $w = w_0 + w_1x + \dots + w_{n-1}x^{n-1}$ into a plaintext bit string M .

For each coefficient, the procedure consists of three stages:

1. **Centering:** $\tilde{w}_i = w_i - \frac{q+1}{2} \bmod \pm q$,
2. **Aggregation:** $t_j = \sum_{i \equiv j \bmod (n/k)} |\tilde{w}_i|$;
3. **Decision:** $M_j = \begin{cases} 1, & t_j < \frac{k(q-1)}{4}, \\ 0, & \text{otherwise.} \end{cases}$

First of all, we consider the masked centering. The centering operation is essentially a conditional reduction: $\tilde{w}_i = \begin{cases} w_i, & w_i < \frac{q}{2}, \\ w_i - q, & \text{otherwise,} \end{cases}$ which is nonlinear

and cannot be implemented directly in the arithmetic masked domain. A practical approach is to convert this decision into the Boolean domain using the most

significant bit (MSB): $\text{MSB}(w_i) = \begin{cases} 1, & w_i < \frac{q}{2}, \\ 0, & \text{otherwise} \end{cases}$. For w_i in arithmetic masking

form $w_i = w_i^{(1)} + \dots + w_i^{(\mathfrak{d})} \bmod q$, the procedure is as follows:

1. Compute masked shares of $w_i - \frac{q+1}{2}$.
2. Convert to Boolean masking using an Arithmetic-to-Boolean (A2B) conversion (e.g., [44]), with complexity $O(\mathfrak{d}^2 \log k)$, where k is the bit-length.
3. Extract the MSB in the Boolean domain share-wise.
4. Compute the centered value using masked multiplication: $\tilde{w}_i = w_i \cdot \text{MSB}(w_i) + (w_i - q) \cdot (1 - \text{MSB}(w_i))$.

Therefore, the cost of centering a single coefficient is $O(\mathfrak{d}^2 \log p)$.

Then, we consider the masked absolute value and aggregation. The computation of $t_j = \sum |\tilde{w}_i|$ consists of following two steps.

- Step 1: Absolute value $|\tilde{w}_i| = \begin{cases} \tilde{w}_i, & \tilde{w}_i < \frac{q}{2}, \\ q - \tilde{w}_i, & \text{otherwise} \end{cases}$ is nonlinear and implemented in the Boolean domain using MSB: $|\tilde{w}_i| = \tilde{w}_i \cdot (1 - \text{MSB}(\tilde{w}_i))$. Note that the A2B conversion computed during centering can be reused here. A masked refreshing step is applied to obtain fresh Boolean shares with cost $O(\mathfrak{d}^2)$, followed by one masked multiplication with cost $O(\mathfrak{d}^2)$.
- Step 2: Summation. Since summation is not modulo q , it must be performed in a larger domain $\mathbb{Z}_r$ such that $r > k^2$.
 - First, convert $|\tilde{w}_i|$ from Boolean to arithmetic masking in $\mathbb{Z}_r$ using B2A with overhead complexity of $O(\mathfrak{d}^2 \log(\log r))$.
 - Then directly perform masked summation $t_j = \sum |\tilde{w}_i|$.

Thus, the total complexity of aggregation is $O(\mathfrak{d}^2 \log(\log r))$

Finally, we consider the masked decision. It is also nonlinear and must be implemented in the Boolean domain. This requires an A2B conversion with complexity $O(\mathfrak{d}^2 \log(\log r))$ and a Boolean comparison. Therefore, the total complexity of the decision step is dominated by A2B conversion $O(\mathfrak{d}^2 \log(\log r))$.

9.2.4 Masked Pt2Poly The Poly2Pt module converts a bit string M a polynomial $m = M_0 + M_1x + \dots + M_{n-1}x^{n-1}$. Since each coefficient M_i is the corresponding bit in M . The Masked Pt2Poly can be trivially implemented by using the one-bit B2A algorithm from [44], with complexity $O(\mathfrak{d}^2)$.

9.2.5 Masked Zero-Testing. For a secret value $x \in \mathbb{Z}_q$, the objective is to distinguish between the cases $x = 0$ and $x \neq 0$ without leaking any information about x , while keeping the result in a masked form. This can be achieved adopting the zero-testing method based on Fermat's little theorem proposed in [21].

The procedure is as follows:

1. Compute the exponentiation in the masked domain: $x^{q-1} \bmod q = 1$. That is, we compute $y_0, \dots, y_d$ such that $y_0 + \dots + y_d \bmod q = x^{q-1} \bmod q = 1$. This can be achieved by using a multiplicative chain with ISW multiplication [33].
2. Apply an arithmetic-to-Boolean conversion (A2B) to $(y_0, \dots, y_d)$, resulting in the $(b_0, \dots, b_d)$.
3. Output the shares $(b_0, \dots, b_d)$.

The complexity of this masked zero-test is dominated by masked multiplications in the exponentiation, resulting in an overall cost of $O(d^2 \log p)$.

9.2.6 Masked Zero-Testing for Multiple Variables. For zero-testing k masked variables, a straightforward approach is to perform masked zero-testing independently k times, which leads to a high computational cost. Instead, we can adopt the method of [11]. The main idea is to construct κ independent random linear combinations of the k variables (denoted as $a_1, \dots, a_k$), and perform zero-testing on these combinations. If all κ linear combinations evaluate to zero, then the probability that all k masked variables are zero increases exponentially with κ (equivalently, the error probability decreases exponentially). Consequently, instead of performing k independent masked zero-tests, it suffices to perform only κ masked zero-tests on the random linear combinations, significantly reducing the computational cost.

References

1. Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to lwe. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 297–322, Cham, 2017. Springer International Publishing.
2. Martin R. Albrecht and Yixin Shen. Quantum augmented dual attack. Cryptology ePrint Archive, Paper 2022/656, 2022.
3. Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. In *Journal of Mathematical Cryptology*, 9:169–203, oct 2015.
4. Erdem Alkim, Roberto Avanzi, Joppe Bos, Léo Ducas, Antonio de la Piedra, Thomas Pöppelmann, Peter Schwabe, Douglas Stebila, Martin R. Albrecht, Emmanuela Orsini, Valery Osheter, Kenneth G. Paterson, Guy Peer, and Nigel P. Smart. Newhope – submission to the NIST post-quantum project, 2020.
5. Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange-a new hope. In *USENIX Security Symposium 2016*, 2016.
6. Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. NewHope without reconciliation. Cryptology ePrint Archive, Report 2016/1157, 2016.
7. Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part II*, pages 269–295. Springer, 2019.
8. Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber–submission to the nist post-quantum project. specification document (part of the submission package), 2019.
9. Shi Bai and Steven D. Galbraith. Lattice decoding attacks on binary lwe. In Willy Susilo and Yi Mu, editors, *Information Security and Privacy*, pages 322–337, Cham, 2014. Springer International Publishing.
10. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proceeding of the 1st ACM Conference on Computer and Communications Security*, pages 62–73, 1993.

11. Shivam Bhasin, Jan-Pieter D’Anvers, Daniel Heinz, Thomas Pöppelmann, and Michiel Van Beirendonck. Attacking and defending masked polynomial comparison for lattice-based cryptography. *IACR Cryptol. ePrint Arch.*, 2021:104, 2021.
12. Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In DongHoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 41–69. Springer Berlin Heidelberg, 2011.
13. J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehle. Crystals - kyber: A cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS P)*, pages 353–367, April 2018.
14. Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration inequalities: A nonasymptotic theory of independence*. Oxford University Press, 2013.
15. Z. Brakerski and V. Vaikuntanathan. Efficient fully homomorphic encryption from (standard) LWE. In *Foundations of Computer Science (FOCS), 2011 IEEE 52nd Annual Symposium on*, pages 97–106, oct. 2011.
16. Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical gapsvp. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886. Springer Berlin Heidelberg, 2012.
17. Zvika Brakerski and Vinod Vaikuntanathan. Fully homomorphic encryption from ring-LWE and security for key dependent messages. In Phillip Rogaway, editor, *Advances in Cryptology – CRYPTO 2011*, volume 6841 of *Lecture Notes in Computer Science*, pages 505–524. Springer Berlin Heidelberg, 2011.
18. Kevin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of matzov’s dual attack on kyber. In *Advances in Cryptology – CRYPTO 2025*, pages 444–476. Springer, 2025.
19. André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, pages 63–91, Cham, 2021. Springer International Publishing.
20. Yuanmi Chen. Lattice reduction and concrete security of fully homomorphic encryption. *Dept. Informatique, ENS, Paris, France, PhD thesis*, 2013.
21. Jean-Sebastien Coron, Francois Gerard, Simon Montoya, and Rina Zeitoun. High-order polynomial comparison and masking lattice-based encryption. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(1):153–192, 2023.
22. Ronald Cramer and Victor Shoup. A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In Hugo Krawczyk, editor, *Advances in Cryptology – CRYPTO ’98*, volume 1462 of *Lecture Notes in Computer Science*, pages 13–25. Springer Berlin / Heidelberg, 1998.
23. Jan-Pieter D’Anvers, Mélissa Rossi, and Fernando Virdia. (one) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020*, pages 3–33, Cham, 2020. Springer.
24. Léo Ducas and Ludo N. Pulles. Does the dual-sieve attack on learning with errors even work? In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, pages 37–69, Cham, 2023. Springer Nature Switzerland.
25. Julien Duman, Kathrin Hövelmanns, Eike Kiltz, Vadim Lyubashevsky, Gregor Seiler, and Dominique Unruh. A thorough treatment of highly-efficient NTRU instantiations. *Cryptology ePrint Archive*, Paper 2021/1352, 2021.
26. Jan-Pieter D’Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. The impact of error dependencies on ring/mod-lwe/lwr based schemes. In *International Conference on Post-Quantum Cryptography*, pages 103–115. Springer, 2019.
27. Jan-Pieter D’Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. The impact of error dependencies on ring/mod-lwe/lwr based schemes. In *International Conference on Post-Quantum Cryptography*, pages 103–115. Springer, 2019.
28. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure intergration of asymmetric and symmetric encryption schemes. In *Annual international cryptography conference*, pages 537–554, 1999.
29. Lewis Glabush, Patrick Longa, Michael Naehrig, Chris Peikert, Douglas Stebila, and Fernando Virdia. FrodoKEM: A CCA-secure learning with errors key encapsulation mechanism. *IACR Communications in Cryptology*, 2(3), 2025.
30. Qian Guo and Thomas Johansson. Faster dual lattice attacks for solving lwe with applications to crystals. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, pages 33–62, Cham, 2021. Springer International Publishing.

31. Kathrin Hövelmanns, Eike Kiltz, Sven Schäge, and Dominique Unruh. Generic authenticated key exchange in the quantum random oracle model. In *Public-Key Cryptography - PKC 2020 - 23rd IACR International Conference on Practice and Theory of Public-Key Cryptography, Edinburgh, UK, May 4-7, 2020, Proceedings, Part II*, pages 389–422. Springer, 2020.
32. China Institute of Commercial Cryptography Standards. Call for proposals for the next-generation public-key cryptographic algorithms, 2025.
33. Yuval Ishai, Amit Sahai, and David A. Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, Lecture Notes in Computer Science, pages 463–481. Springer, 2003.
34. Wenjuan Jia, Jiang Zhang, Binwu Xiang, and Baocang Wang. Hardness of (m)lwe with semi-uniform seeds. *Theoretical Computer Science*, 994:114481, 2024.
35. Shuqin FAN Jiang ZHANG. On the hardness of the asymmetric learning with errors problem. *Journal of Electronics & Information Technology*, 42(2):327–332, 2020.
36. Ravi Kannan. Minkowski’s convex body theorem and integer programming. *Mathematics of Operations Research*, 12(3):415–440, 1987.
37. Richard Lindner and Chris Peikert. Better key sizes (and attacks) for LWE-based encryption. In Aggelos Kiayias, editor, *Topics in Cryptology - CT-RSA 2011*, volume 6558 of *Lecture Notes in Computer Science*, pages 319–339. Springer Berlin Heidelberg, 2011.
38. IDF MATZOV. Report on the security of lwe: improved dual lattice attack, 2022. URL: <https://zenodo.org/record/6412487>, 2022.
39. Daniele Micciancio and Oded Regev. Lattice-based cryptography. In DanielJ. Bernstein, Johannes Buchmann, and Erik Dahmen, editors, *Post-Quantum Cryptography*, pages 147–191. Springer Berlin Heidelberg, 2009.
40. National Institute of Standards and Technology. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. *Federal Information Processing Standards Publication*, NIST FIPS 202, 2015.
41. Tobias Oder, Tobias Schneider, Thomas Pöppelmann, and Tim Güneysu. Practical cca2-secure and masked ring-lwe implementation. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):142–174, 2018.
42. Thomas Pöppelmann and Tim Güneysu. Towards practical lattice-based public-key encryption on reconfigurable hardware. In Tanja Lange, Kristin Lauter, and Petr Lisonek, editors, *Selected Areas in Cryptography - SAC 2013*, pages 68–85. Springer, 2014.
43. Amaury Pouly and Yixin Shen. Provable dual attacks on learning with errors. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024*, pages 256–285, Cham, 2024. Springer Nature Switzerland.
44. Tobias Schneider, Clara Paglialonga, Tobias Oder, and Tim Güneysu. Efficiently masking binomial sampling at arbitrary orders for lattice-based crypto. In Dongdai Lin and Kazue Sako, editors, *Public-Key Cryptography - PKC 2019 - 22nd IACR International Conference on Practice and Theory of Public-Key Cryptography, Beijing, China, April 14-17, 2019, Proceedings, Part II*, Lecture Notes in Computer Science, pages 534–564. Springer, 2019.
45. Roman Vershynin. Introduction to the non-asymptotic analysis of random matrices. In Yonina C. Eldar and Gitta Kutyniok, editors, *Compressed Sensing: Theory and Applications*, pages 210–268. Cambridge University Press, 2012.
46. Jiang Zhang, Yu Yu, Shuqin Fan, Zhenfeng Zhang, and Kang Yang. Tweaking the asymmetry of asymmetric-key cryptography on lattices: Kems and signatures of smaller sizes. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020*, pages 37–65. Springer, 2020.

Appendix A

A.1 Proof of Theorem 2

Proof. To prove this theorem, we introduce a sequence of games as shown in Fig. 10. In these games, we omit the detailed computation of Aigis-PKE⁺.KeyGen and Aigis-PKE⁺.Enc for notational simplicity.

GAMES $\mathbf{G_0-G_5}$		$G(x_1, x_2)$	
1. $(pk, sk) \leftarrow \text{Aigis-PKE}^+.\text{KeyGen}$	// $\mathbf{G_0-G_5}$	1. return $G_0(x_1, x_2)$	// $\mathbf{G_1-G_2}$
$H_1 \xleftarrow{\$} \text{Fcs}[* , 256], s \xleftarrow{\$} \{0, 1\}^\kappa$	// $\mathbf{G_0-G_5}$	2. $\rho := F(x_1, H_1(pk))$	// $\mathbf{G_3-G_5}$
$sk = (sk, pk, H_1(pk), s)$	// $\mathbf{G_0-G_5}$	$c := \text{Aigis-PKE}^+.\text{Enc}(pk, x_1; \rho)$	
$H_2 \xleftarrow{\$} \text{Fcs}[\kappa + 256, r + 256]$	// $\mathbf{G_0}$	if $x_2 \neq H_1(pk) \vee c = c^*$	
$H_3 \xleftarrow{\$} \text{Fcs}[\kappa + n, 256]$	// $\mathbf{G_0-G_1}$	return $G_0(x_1, x_2)$	
$H_4 \xleftarrow{\$} \text{Fcs}[n, 256]$	// $\mathbf{G_2-G_3}$	else return $G_1(c)$	
$G_0 \xleftarrow{\$} \text{Fcs}[\kappa + 256, 256]$	// $\mathbf{G_1-G_5}$		
$F \xleftarrow{\$} \text{Fcs}[\kappa + 256, r]$	// $\mathbf{G_1-G_5}$	Aigis-Enc ⁺ .Decap(sk, c)	
$G_1 \xleftarrow{\$} \text{Fcs}[n, 256]$	// $\mathbf{G_3-G_5}$	1. if $c = c^*$, return $\perp$	
2. $b \xleftarrow{\$} \{0, 1\}, M^* \xleftarrow{\$} \{0, 1\}^\kappa$	// $\mathbf{G_0-G_5}$	else $M := \text{Aigis-PKE}^+.\text{Dec}(sk, c)$	
$(K_0^*, \rho^*) = H_2(M^*, H_1(pk))$	// $\mathbf{G_0}$	2. if $M = \perp$, return $H_3(s, c)$	// $\mathbf{G_0-G_1}$
$\rho^* = F(M^*, H_1(pk))$	// $\mathbf{G_1-G_4}$	if $M = \perp$, return $H_4(c)$	// $\mathbf{G_2-G_3}$
$\rho^* \xleftarrow{\$} \{0, 1\}^r$	// $\mathbf{G_5}$	else $(K, \rho) := H_2(M, H_1(pk))$	// $\mathbf{G_0}$
$c^* = \text{Aigis-PKE}^+.\text{Enc}(pk, M^*; \rho^*)$	// $\mathbf{G_0-G_5}$	$K = G(M, H_1(pk))$	// $\mathbf{G_1-G_3}$
$K_0^* = G(M^*, H_1(pk))$	// $\mathbf{G_1-G_4}$	$\rho = F(M, H_1(pk))$	// $\mathbf{G_1-G_3}$
$K_0^* \xleftarrow{\$} \{0, 1\}^{256}$	// $\mathbf{G_5}$	3. if $c = \text{Aigis-PKE}^+.\text{Enc}(pk, M; \rho)$	
$K_1^* \xleftarrow{\$} \{0, 1\}^{256}$	// $\mathbf{G_0-G_5}$	return K	
3. $b' \leftarrow \mathcal{A}^{H_1, H_2, H_3, \text{Aigis-Enc}^+.\text{Decap}}(pk, c^*, K_b^*)$	// $\mathbf{G_0}$	else return $H_3(s, c)$	// $\mathbf{G_0-G_1}$
$b' \leftarrow \mathcal{A}_1^{H_1, F, G, H_3, \text{Aigis-Enc}^+.\text{Decap}}(pk, c^*, K_b^*)$	// $\mathbf{G_1-G_5}$	else return $H_4(c)$	// $\mathbf{G_2-G_3}$
4. return $\llbracket b' = b \rrbracket$	// $\mathbf{G_0-G_5}$		
		1. if $c = c^*$, return $\perp$	
		else return $G_1(c)$	

Fig. 10: Games $\mathbf{G_0}$ to $\mathbf{G_5}$ in the proof of Theorem 2. In these games, we denote the randomness space of Aigis-PKE⁺ as $\{0, 1\}^r$ and the ciphertext space of Aigis-PKE⁺ as $\{0, 1\}^n$.

Game $\mathbf{G_0}$: This game is the IND-CCA security game of Aigis-Enc⁺ with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{Aigis-Enc}^+}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr[1 \leftarrow \mathbf{G_0}] - \frac{1}{2} \right|. \quad (21)$$

Game $\mathbf{G_1}$: Compared with game $\mathbf{G_0}$, this game has the following three differences:

1. Two new random oracles F and G are introduced.

2. When computing (c^*, K_b^*) and $\text{Aigis-Enc}^+.\text{Decap}(sk, c)$, the $H_2(x_1, x_2)$ is replaced with $(G(x_1, x_2), F(x_1, x_2))$, where $F(x_1, x_2)$ is the randomness used by $\text{Aigis-PKE}^+.\text{Enc}$ and $G(x_1, x_2)$ is the (potential) session key.
3. The adversary $\mathcal{A}$ is replaced with $\mathcal{A}_1$, which runs $\mathcal{A}$ and returns its final output. In particular, for $\mathcal{A}$'s H_2 query (x_1, x_2) , $\mathcal{A}_1$ simulates it by querying F and G and answers $(F(x_1, x_2), G(x_1, x_2))$ to $\mathcal{A}$.

One can note that $\mathcal{A}_1$ makes q_{H_1} queries to H_1 , q_{H_2} queries to both F and G , q_{H_3} queries to H_3 , and q_D queries to $\text{Aigis-Enc}^+.\text{Decap}$.

According to the basic rules of random oracle, game hop $\mathbf{G}_0$ to $\mathbf{G}_1$ is only a conceptual change, thus we have

$$\Pr[1 \leftarrow \mathbf{G}_0] = \Pr[1 \leftarrow \mathbf{G}_1]. \quad (22)$$

Game $\mathbf{G}_2$: In this game, the $H_3(s, c)$ used by $\text{Aigis-Enc}^+.\text{Decap}$ is replaced with $H_4(c)$, where H_4 is a new random oracle with input space $\{0, 1\}^n$ and output space $\{0, 1\}^{256}$.

If the adversary $\mathcal{A}_1$ never queries H_3 with an input in the form of $(s, *)$, the $H_3(s, c)$ used by $\text{Aigis-Enc}^+.\text{Decap}$ is equivalent to $H_4(c)$ in $\mathcal{A}_1$'s view. Since $\mathcal{A}_1$ queries H_3 at most q_{H_3} times and s is uniformly sampled from $\{0, 1\}^\kappa$ at the game beginning, we have

$$|\Pr[1 \leftarrow \mathbf{G}_1] - \Pr[1 \leftarrow \mathbf{G}_2]| \leq \frac{q_{H_3}}{2^\kappa}. \quad (23)$$

Game $\mathbf{G}_3$: This game is the same as $\mathbf{G}_2$, except that the simulation of G is replaced with the following operation: For the (x_1, x_2) , compute $\rho = F(x_1, H_1(pk))$ and $c = \text{Aigis-PKE}^+.\text{Enc}(pk, x_1; \rho)$, and then return $G_0(x_1, x_2)$ if $x_2 \neq H_1(pk)$ or $c = c^*$. Otherwise, return $G_1(c)$, where G_1 is a new random oracle with input space $\{0, 1\}^n$ and output space $\{0, 1\}^{256}$.

Denote **Bad** as the event that the adversary $\mathcal{A}_1$ ever queries G with two inputs $(x_1, H_1(pk))$ and $(x'_1, H_1(pk))$ such that $x_1 \neq x'_1$ and

$$\text{Aigis-PKE}^+.\text{Enc}(pk, x_1; F(x_1, H_1(pk))) = \text{Aigis-PKE}^+.\text{Enc}(pk, x'_1; F(x'_1, H_1(pk))).$$

Based on the basic rules of random oracle, if **Bad** does not occur, the game hop $\mathbf{G}_2$ to $\mathbf{G}_3$ is only a conceptual change. Thus we have

$$|\Pr[1 \leftarrow \mathbf{G}_2] - \Pr[1 \leftarrow \mathbf{G}_3]| \leq \Pr[\text{Bad}]. \quad (24)$$

Since the decryption algorithm $\text{Aigis-Enc}^+.\text{Dec}$ is deterministic, if $\mathcal{A}_1$ indeed queried G using $(x_1, H_1(pk))$ and $(x'_1, H_1(pk))$ trigger **Bad**, then we have $x_1 \in S$ or $x'_1 \in S$, where S is defined as

$$S := \left\{ x \in \{0, 1\}^\kappa \left| \begin{array}{l} c = \text{Aigis-PKE}^+.\text{Enc}(pk, x; F(x, H_1(pk))), \\ \text{Aigis-PKE}^+.\text{Dec}(sk, c) \neq x \end{array} \right. \right\}. \quad (25)$$

That is, conditioned on **Bad** occurs, $\mathcal{A}_1$ must have queried G with some $(x, H_1(pk))$ such that $x \in S$. For any $x \in \{0, 1\}^\kappa$, we can compute

$$\begin{aligned} & \Pr \left[x \in S : (pk, sk) \leftarrow \text{Aigis-PKE}^+. \text{KeyGen}, F \xleftarrow{\$} \text{Fcs}[\kappa + 256, r] \right] \\ & \leq \Pr \left[\begin{array}{l} \text{Aigis-PKE}^+. \text{Dec}(sk, c) \neq x : \\ \quad (pk, sk) \leftarrow \text{Aigis-PKE}^+. \text{KeyGen}, y \xleftarrow{\$} \{0, 1\}^r, \\ \quad c = \text{Aigis-PKE}^+. \text{Enc}(pk, x; y) \end{array} \right] \\ & \stackrel{(a)}{\leq} \delta. \end{aligned}$$

Here (a) follows from the definition of δ given in Definition 1. Notice that $\mathcal{A}_1$ makes at most q_{H_2} queries to G , and hence by using the union bound, we obtain $\Pr[\text{Bad}] \leq q_{H_2} \cdot \delta$. Combining this inequality with Eq. (24), we have

$$|\Pr[1 \leftarrow \mathbf{G}_2] - \Pr[1 \leftarrow \mathbf{G}_3]| \leq q_{H_2} \cdot \delta. \quad (26)$$

Game $\mathbf{G}_4$: This game is the same as game $\mathbf{G}_3$, except that the decapsulation oracle $\text{Aigis-Enc}^+. \text{Decap}$ is changed as it directly returns $K = G_1(c)$ for $c \neq c^*$. That is, the answer of decapsulation oracle in this game no longer uses the secret key sk .

Denote Bad' as the event that the adversary $\mathcal{A}_1$ ever queries G with an input $(x, H_1(pk))$ such that $x \in S$, where S is defined in Eq. (25). In game $\mathbf{G}_3$, if Bad' does not occur, by querying G the adversary $\mathcal{A}_1$ can only get $G_1(c)$ such that $\text{Aigis-PKE}^+. \text{Dec}(sk, c) \neq \perp$ and

$$c = \text{Aigis-PKE}^+. \text{Enc}(pk, M; F(M, H_1(pk))),$$

where $M = \text{Aigis-PKE}^+. \text{Dec}(sk, c)$. Hence, in game $\mathbf{G}_3$, the $H_4(c)$ returned by the $\text{Aigis-Enc}^+. \text{Decap}(sk, c)$ under the condition that $\text{Aigis-PKE}^+. \text{Dec}(sk, c) = \perp$ or $M = \text{Aigis-PKE}^+. \text{Dec}(sk, c) \neq \perp \wedge c \neq \text{Aigis-PKE}^+. \text{Enc}(pk, M; F(M, H_1(pk)))$ is equivalent to $G_1(c)$ in $\mathcal{A}_1$'s view. This means that game $\mathbf{G}_3$ is equivalent to game $\mathbf{G}_4$ if Bad' does not occur, and thus

$$|\Pr[1 \leftarrow \mathbf{G}_3] - \Pr[1 \leftarrow \mathbf{G}_4]| \leq \Pr[\text{Bad}'] \stackrel{(a)}{\leq} q_{H_2} \cdot \delta. \quad (27)$$

Here (a) uses the same reasoning as $\Pr[\text{Bad}] \leq q_{H_2} \cdot \delta$ in the game hop $\mathbf{G}_2$ to $\mathbf{G}_3$.

Game $\mathbf{G}_5$: This game is the same as $\mathbf{G}_4$, except that the K_0^* and ρ^* are both replaced with fresh random values.

Denote Bad_{M^*} as the event that the adversary $\mathcal{A}_1$ ever queries F or G with $(M^*, H_1(pk))$. Now, game $\mathbf{G}_4$ is equivalent to game $\mathbf{G}_5$ if Bad_{M^*} does not occur. Hence, we have

$$|\Pr[1 \leftarrow \mathbf{G}_4] - \Pr[1 \leftarrow \mathbf{G}_5]| \leq \Pr[\text{Bad}_{M^*} : \mathbf{G}_5] \stackrel{(a)}{=} \Pr[1 \leftarrow \mathbf{G}_5^a]. \quad (28)$$

Game $\mathbf{G}_5^a$	Game $\mathbf{G}_5^b$
1. $(pk, sk) \leftarrow \text{Aigis-PKE}^+.\text{KeyGen}$ $H_1 \xleftarrow{\$} \text{Fcs}[* , 256], s \xleftarrow{\$} \{0, 1\}^\kappa$ $sk = (sk, pk, H_1(pk), s)$ $G_1 \xleftarrow{\$} \text{Fcs}[n, 256], F \xleftarrow{\$} \text{Fcs}[\kappa + 256, r]$ $G_0 \xleftarrow{\$} \text{Fcs}[\kappa + 256, 256]$	1. $(pk, sk) \leftarrow \text{Aigis-PKE}^+.\text{KeyGen}$ $H_1 \xleftarrow{\$} \text{Fcs}[* , 256], s \xleftarrow{\$} \{0, 1\}^\kappa$ $sk = (sk, pk, H_1(pk), s)$ $G_1 \xleftarrow{\$} \text{Fcs}[n, 256], F \xleftarrow{\$} \text{Fcs}[\kappa + 256, r]$ $G_0 \xleftarrow{\$} \text{Fcs}[\kappa + 256, 256]$
2. $b \xleftarrow{\$} \{0, 1\}, M^* \xleftarrow{\$} \{0, 1\}^\kappa$ $\rho^* \xleftarrow{\$} \{0, 1\}^r$ $c^* = \text{Aigis-PKE}^+.\text{Enc}(pk, M^*; \rho^*)$ $K_0^* \xleftarrow{\$} \{0, 1\}^{256}, K_1^* \xleftarrow{\$} \{0, 1\}^{256}$	2. $b \xleftarrow{\$} \{0, 1\}, M^* \xleftarrow{\$} \{0, 1\}^\kappa, M' \xleftarrow{\$} \{0, 1\}^\kappa$ $\rho^* \xleftarrow{\$} \{0, 1\}^r$ $c^* = \text{Aigis-PKE}^+.\text{Enc}(pk, M'; \rho^*)$ $K_0^* \xleftarrow{\$} \{0, 1\}^{256}, K_1^* \xleftarrow{\$} \{0, 1\}^{256}$
3. run $\mathcal{A}_1^{H_1, F, G, H_3, \text{Aigis-Enc}^+.\text{Decap}}(pk, c^*, K_b^*)$	3. run $\mathcal{A}_1^{H_1, F, G, H_3, \text{Aigis-Enc}^+.\text{Decap}}(pk, c^*, K_b^*)$
4. return $\llbracket \text{Bad}_{M^*} \rrbracket$	4. return $\llbracket \text{Bad}_{M^*} \rrbracket$

Fig. 11: Games $\mathbf{G}_5^a$ and $\mathbf{G}_5^b$ in the proof of Theorem 2. In these two games, the oracles queried by $\mathcal{A}_1$ are simulated in the same way as in game $\mathbf{G}_5$ of Fig. 10.

Here (a) follows from the definition of game $\mathbf{G}_5^a$ given in Fig. 11, which returns 1 if and only if Bad_{M^*} occurs in game $\mathbf{G}_5$. Since the bit b is independent of $\mathcal{A}_1$'s view in game $\mathbf{G}_5$, we have

$$\Pr [1 \leftarrow \mathbf{G}_5] = \frac{1}{2}. \quad (29)$$

We also define a game $\mathbf{G}_5^b$ in Fig. 11, which is the same as $\mathbf{G}_5^a$ except that the challenge ciphertext c^* equals to $\text{Aigis-PKE}^+.\text{Enc}(pk, M'; \rho^*)$, where M' is new challenge plaintext uniformly sampled from $\{0, 1\}^\kappa$. At this point, we define a new adversary $\mathcal{B}$ against the IND-CPA security of Aigis-PKE^+ as follows:

- $\mathcal{B}$ samples $M^*, M' \leftarrow \{0, 1\}^\kappa$ as the challenge plaintexts and send them to the IND-CPA challenger. After the challenger replies the challenge ciphertext c^* , $\mathcal{B}$ runs $\mathcal{A}_1^{H_1, F, G, H_3, \text{Aigis-Enc}^+.\text{Decap}}(pk, c^*, K_b^*)$ in game $\mathbf{G}_5^a$ (or game $\mathbf{G}_5^b$ equivalently) and simulates the random oracles on-the-fly. $\mathcal{B}$ finally returns the final output of this game, i.e. $\llbracket \text{Bad}_{M^*} \rrbracket$.

The running time of $\mathcal{B}$ is about that of $\mathcal{A}$ and $\mathcal{A}_1$, and

$$2 \cdot \text{Adv}_{\text{Aigis-PKE}^+}^{\text{IND-CPA}}(\mathcal{B}) = |\Pr [1 \leftarrow \mathbf{G}_5^a] - \Pr [1 \leftarrow \mathbf{G}_5^b]|. \quad (30)$$

For the game $\mathbf{G}_5^b$, since M^* is independent of $\mathcal{A}_1$'s view and F (resp. G) is queried at most q_{H_2} (resp. q_{H_2}) times, we have

$$\Pr [1 \leftarrow \mathbf{G}_5^b] \leq \frac{2q_{H_2}}{2^\kappa}. \quad (31)$$

Combining Eq.(21) to Eq. (23) and Eq.(26) to Eq. (31), we have

$$\begin{aligned}
\text{Adv}_{\text{Aigis-Enc}^+}^{\text{IND-CCA}}(\mathcal{A}) &\leq 2 \cdot \text{Adv}_{\text{Aigis-PKE}^+}^{\text{IND-CPA}}(\mathcal{B}) + \frac{2q_{H_2} + q_{H_3}}{2^\kappa} + 2q_{H_2} \cdot \delta \\
&\stackrel{(a)}{\leq} 2 \cdot \text{Adv}_{n,q,\eta_1,\eta_2}^{\text{ARLWE}}(\mathcal{B}_1) + 2 \cdot \text{Adv}_{n,q,2^{dt},\eta_1,\eta_2}^{\text{semi-ARLWE}}(\mathcal{B}_2) \\
&\quad + \frac{2q_{H_2} + q_{H_3}}{2^\kappa} + 2q_{H_2} \cdot \delta.
\end{aligned}$$

Here (a) follows from Theorem 1 and the running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$. This completes the proof of Theorem 2. $\square$

A.2 Proof of Theorem 3

Before giving the proof, we first introduce the semi-classical oracle and some related lemmas.

Definition 7 (Semi-Classical Oracle [7]). For a subset $S \subseteq \mathcal{X}$, the indicator function f_S from $\mathcal{X}$ to $\{0, 1\}$ equals 1 iff its input $x \in S$. The semi-classical oracle, $\mathcal{O}_S^{SC}$, is a quantum accessible oracle that performs the following two operations on its query state $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle$:

1. Initialize a single qubit register L with the state $|0\rangle_{\mathsf{L}}$.
2. Transform $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle |0\rangle_{\mathsf{L}}$ into $\sum_{x \in \mathcal{X}} \alpha_x |x\rangle |f_S(x)\rangle_{\mathsf{L}}$ and then measure L .

During the execution of a quantum oracle algorithm $\mathcal{A}^{\mathcal{O}_S^{SC}}$, let **Find** be the classical event that the measurement of $\mathcal{O}_S^{SC}$ (on L) ever has outcome 1.

Lemma 2. Let $S \subseteq \mathcal{X}$ be random. Let z be a random bitstring. (S, z may have arbitrary joint distribution $\mathfrak{D}$.) Let $\mathcal{A}$ be a quantum oracle algorithm with query times q (not necessarily unitary). Let

$$P_{\text{left}} := \Pr[b = 1 : b \leftarrow \mathcal{A}^{\emptyset}(z), (S, z) \xleftarrow{\$} \mathfrak{D}],$$

$$P_{\text{right}} := \Pr[b = 1 : b \leftarrow \mathcal{A}^{\mathcal{O}_S^{SC}}(z), (S, z) \xleftarrow{\$} \mathfrak{D}].$$

Here $\emptyset$ denotes an oracle that does nothing on the query state. Let $P_{\text{find}} := \Pr[\text{Find} : \mathcal{A}^{\mathcal{O}_S^{SC}}(z), (S, z) \xleftarrow{\$} \mathfrak{D}]$. Then

$$|P_{\text{left}} - P_{\text{right}}| \leq \sqrt{(q+1) \cdot P_{\text{find}}} \text{ and } \left| \sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}} \right| \leq \sqrt{(q+1) \cdot P_{\text{find}}}.$$

Proof. This is a direct corollary of [7, Theorem 1] by setting the function H used by it as a constant zero function.

Lemma 3 (Search in Semi-Classical Oracle [7, Corollary 1]). Suppose that S and z are independent, and that $\mathcal{A}$ is a quantum oracle algorithm making at most q queries. Let $P_{\text{max}} := \max_{x \in \mathcal{X}} \Pr[x \in S : (S, z) \xleftarrow{\$} \mathfrak{D}]$. Then we have $\Pr[\text{Find} : \mathcal{A}^{\mathcal{O}_S^{SC}}(z), (S, z) \xleftarrow{\$} \mathfrak{D}] \leq 4q \cdot P_{\text{max}}$.

Lemma 4 ([31, Lemma 2.9]). Let Ω_H ($\Omega_{H'}$) be the set of all functions $H : \{0, 1\}^{n_1} \times \{0, 1\}^{n_2} \rightarrow \{0, 1\}^m$ ($H' : \{0, 1\}^{n_2} \rightarrow \{0, 1\}^m$). Let $H \xleftarrow{\$} \Omega_H$, $H' \xleftarrow{\$} \Omega_{H'}$, $x \xleftarrow{\$} \{0, 1\}^{n_1}$. Let $F_0 = H(x, \cdot)$, $F_1 = H'(\cdot)$. Consider an oracle algorithm $\mathcal{A}^{H, F_i}$ that makes at most q queries to H and F_i ($i \in \{0, 1\}$). If x is independent from the $\mathcal{A}^{H, F_i}$'s view, we have

$$\left| \Pr[b = 1 : b \leftarrow \mathcal{A}^{H, F_0}] - \Pr[b = 1 : b \leftarrow \mathcal{A}^{H, F_1}] \right| \leq \frac{2q}{\sqrt{2^{n_1}}}.$$

Lemma 5 (Generic quantum distinguishing problem [31, Lemma 2.9]).

Let $\delta \in [0, 1]$ and $\mathcal{M}$ be a finite set. Let $N_1 : \mathcal{M} \rightarrow \{0, 1\}$ be a random function such that, for each $m \in \mathcal{M}$, $N_1(m) = 1$ with probability δ_m ($\delta_m \leq \delta$), and $N_1(m) = 0$ with probability $1 - \delta_m$. Let $N_2 : \mathcal{M} \rightarrow \{0, 1\}$ be a constant function such that $N_2(m) = 0$ for all $m \in \mathcal{M}$. For any algorithm $\mathcal{A}$ that makes at most q quantum queries, we have

$$|\Pr [b = 1 : b \leftarrow \mathcal{A}^{N_1}] - \Pr [b = 1 : b \leftarrow \mathcal{A}^{N_2}]| \leq 8(q + 1)^2 \cdot \delta.$$

Proof. For a fixed (pk, sk) pair sampled by $\text{Aigis-PKE}^+.\text{KeyGen}$ and $x \in \mathcal{M}$, where $\mathcal{M} := \{0, 1\}^\kappa$, we define sets

$$\mathcal{R}_{\text{bad}}(pk, sk, x) := \left\{ r \in \mathcal{R} : \begin{array}{l} c = \text{Aigis-PKE}^+.\text{Enc}(pk, x; r), \\ \text{Aigis-PKE}^+.\text{Dec}(sk, c) \neq x \end{array} \right\} \quad (32)$$

and

$$\mathcal{R}_{\text{good}}(pk, sk, x) := \mathcal{R} \setminus \mathcal{R}_{\text{bad}}(pk, sk, x). \quad (33)$$

Here $\mathcal{R} := \{0, 1\}^r$ and $\mathcal{M}$ (resp. $\mathcal{R}$) is the message (resp. randomness) space of Aigis-PKE^+ . Then, we define two values

$$\delta(pk, sk, x) := \frac{|\mathcal{R}_{\text{bad}}(pk, sk, x)|}{|\mathcal{R}|} \text{ and } \delta(pk, sk) := \max_{x \in \mathcal{M}} \delta(pk, sk, x). \quad (34)$$

By Definition 1, we have $\mathbb{E}[\delta(pk, sk)] \leq \delta$ since Aigis-PKE^+ is δ -correct. Here the expectation is taken over $(pk, sk) \leftarrow \text{Aigis-PKE}^+.\text{KeyGen}$.

Let $H_{\text{good}} : \mathcal{M} \rightarrow \mathcal{R}$ be a random function such that $H_{\text{good}}(x)$ is uniformly sampled from $\mathcal{R}_{\text{good}}(pk, sk, x)$ for any $x \in \mathcal{M}$. Now we introduce a sequence of hybrid games as shown in Fig. 12. In these games, we omit the detailed computation of $\text{Aigis-PKE}^+.\text{KeyGen}$ and $\text{Aigis-PKE}^+.\text{Enc}$ for notational simplicity.

Game $\mathbf{G}_0$: This game is the IND-CCA security game of Aigis-Enc^+ with the adversary $\mathcal{A}$. Obviously, we have

$$\text{Adv}_{\text{Aigis-Enc}^+}^{\text{IND-CCA}}(\mathcal{A}) = \left| \Pr [1 \leftarrow \mathbf{G}_0] - \frac{1}{2} \right|. \quad (35)$$

Game $\mathbf{G}_1$: Compared with game $\mathbf{G}_0$, this game has the following three differences:

1. Two new random oracles F and G are introduced.
2. When computing (c^*, K_b^*) and $\text{Aigis-Enc}^+.\text{Decap}(sk, c)$, the $H_2(x_1, x_2)$ is replaced with $(G(x_1, x_2), F(x_1, x_2))$, where $F(x_1, x_2)$ is the randomness used by $\text{Aigis-PKE}^+.\text{Enc}$ and $G(x_1, x_2)$ is the (potential) session key.
3. The adversary $\mathcal{A}$ is replaced with $\mathcal{A}_1$, which runs $\mathcal{A}$ and returns its final output. In particular, for $\mathcal{A}$'s H_2 query $\sum |x_1, x_2, y\rangle$, $\mathcal{A}_1$ simulates it by querying F and G and answers $\sum |x_1, x_2, y \oplus (F(x_1), G(x_2))\rangle$ to $\mathcal{A}$.

One can note that $\mathcal{A}_1$ makes q_{H_1} queries to H_1 , q_{H_2} queries to both F and G , q_{H_3} queries to H_3 , and q_D queries to $\text{Aigis-Enc}^+.\text{Decap}$.

GAMES $\mathbf{G_0-G_{11}}$		$G(x_1, x_2)$	
1. $(pk, sk) \leftarrow \text{Aigis-PKE}^+.\text{KeyGen}$	// $\mathbf{G_0-G_{11}}$	1. return $G_0(x_1, x_2)$	// $\mathbf{G_0-G_4}$
$H_1 \xleftarrow{\$} \text{Fcs}[*], s \xleftarrow{\$} \{0, 1\}^\kappa$	// $\mathbf{G_0-G_{11}}$	2. query $\mathcal{O}_S^{SC}$	// $\mathbf{G_9-G_{10}}$
$sk = (sk, pk, H_1(pk), s)$	// $\mathbf{G_0-G_{11}}$	$\rho := F(x_1, H_1(pk))$	// $\mathbf{G_5-G_{11}}$
$H_2 \xleftarrow{\$} \text{Fcs}[\kappa + 256, r + 256]$	// $\mathbf{G_0}$	$c := \text{Aigis-PKE}^+.\text{Enc}(pk, x_1; \rho)$	
$H_3 \xleftarrow{\$} \text{Fcs}[\kappa + n, 256]$	// $\mathbf{G_0-G_1}$	if $x_2 \neq H_1(pk) \vee c = c^*$	
$H_4 \xleftarrow{\$} \text{Fcs}[n, 256]$	// $\mathbf{G_2-G_5}$	return $G_0(x_1, x_2)$	
$G_0 \xleftarrow{\$} \text{Fcs}[\kappa + 256, 256]$	// $\mathbf{G_1-G_{11}}$	else return $G_1(c)$	
$F_0 \xleftarrow{\$} \text{Fcs}[\kappa + 256, r]$	// $\mathbf{G_1-G_{11}}$	$H_1(x)$	
$G_1 \xleftarrow{\$} \text{Fcs}[n, 256]$	// $\mathbf{G_5-G_{11}}$	1. return $H_1(x)$ // $\mathbf{G_0-G_{11}}$	
$H \xleftarrow{\$} \text{Fcs}[\kappa, r]$	// $\mathbf{G_3, G_7}$	$H_3(x_1, x_2)$	
2. $b \xleftarrow{\$} \{0, 1\}, M^* \xleftarrow{\$} \{0, 1\}^\kappa$	// $\mathbf{G_0-G_{11}}$	1. return $H_3(x_1, x_2)$ // $\mathbf{G_0-G_1}$	
$(K_0^*, \rho^*) = H_2(M^*, H_1(pk))$	// $\mathbf{G_0}$	$\text{Aigis-Enc}^+.\text{Decap}(sk, c)$	
$\rho^* = F(M^*, H_1(pk))$	// $\mathbf{G_1-G_9}$		
$\rho^* \xleftarrow{\$} \{0, 1\}^r$	// $\mathbf{G_{10}}$	1. if $c = c^*$, return $\perp$	
$c^* = \text{Aigis-PKE}^+.\text{Enc}(pk, M^*; \rho^*)$	// $\mathbf{G_0-G_{11}}$	else $M := \text{Aigis-PKE}^+.\text{Dec}(sk, c)$	
$K_0^* = G(M^*, H_1(pk))$	// $\mathbf{G_1-G_9}$	2. if $M = \perp$, return $H_3(s, c)$	// $\mathbf{G_0-G_1}$
$K_0^* \xleftarrow{\$} \{0, 1\}^{256}$	// $\mathbf{G_{10}}$	if $M = \perp$, return $H_4(c)$	// $\mathbf{G_2-G_5}$
$K_1^* \xleftarrow{\$} \{0, 1\}^{256}$	// $\mathbf{G_0-G_{11}}$	else $(K, \rho) := H_2(M, H_1(pk))$	// $\mathbf{G_0}$
3. $b' \leftarrow \mathcal{A}^{H_1, H_2, H_3, \text{Aigis-Enc}^+.\text{Decap}}(pk, c^*, K_b^*)$	// $\mathbf{G_0}$	$K = G(M, H_1(pk))$	// $\mathbf{G_1-G_5}$
$b' \leftarrow \mathcal{A}_1^{H_1, F, G, H_3, \text{Aigis-Enc}^+.\text{Decap}}(pk, c^*, K_b^*)$	// $\mathbf{G_1-G_{11}}$	$\rho = F(M, H_1(pk))$	// $\mathbf{G_1-G_5}$
4. return $\llbracket b' = b \rrbracket$	// $\mathbf{G_0-G_{11}}$	3. if $c = \text{Aigis-PKE}^+.\text{Enc}(pk, M; \rho)$	
$F(x_1, x_2)$		return K	
1. return $F_0(x_1, x_2)$	// $\mathbf{G_0-G_2, G_8, G_{11}}$	else return $H_3(s, c)$	// $\mathbf{G_0-G_1}$
2. query $\mathcal{O}_S^{SC}$, return $F_0(x_1, x_2)$	// $\mathbf{G_9-G_{10}}$	else return $H_4(c)$	// $\mathbf{G_2-G_5}$
3. if $x_2 \neq H_1(pk)$, return $F_0(x_1, x_2)$	// $\mathbf{G_3-G_6}$	$\text{Aigis-Enc}^+.\text{Decap}(sk, c)$ // $\mathbf{G_6-G_{11}}$	
else return $H(x_1)$	// $\mathbf{G_3, G_7}$		
else return $H_{\text{good}}(x_1)$	// $\mathbf{G_4-G_6}$	1. if $c = c^*$, return $\perp$	
		else return $G_1(c)$	

Fig. 12: Games $\mathbf{G_0}$ to $\mathbf{G_{11}}$ in the proof of Theorem 3. In these games, we denote the randomness space of Aigis-PKE^+ as $\{0, 1\}^r$ and the ciphertext space of Aigis-PKE^+ as $\{0, 1\}^n$. Notice that the oracles F and G are quantum-accessible, but we only write their classical description here for brevity.

According to the basic rules of random oracle, game hop $\mathbf{G_0}$ to $\mathbf{G_1}$ is only a conceptual change, thus we have

$$\Pr[1 \leftarrow \mathbf{G_0}] = \Pr[1 \leftarrow \mathbf{G_1}]. \quad (36)$$

Game G₂: In this game, the $H_3(s, c)$ used by Aigis-Enc⁺.Decap is replaced with $H_4(c)$, where H_4 is a new random oracle with input space $\{0, 1\}^n$ and output space $\{0, 1\}^{256}$.

Notice that the random oracle H_3 is queried at most $q_{H_3} + q_D$ times in game **G₁** and s is uniformly sampled from $\{0, 1\}^\kappa$ at the game beginning. Hence, by directly using Lemma 4, we have

$$|\Pr[1 \leftarrow \mathbf{G}_1] - \Pr[1 \leftarrow \mathbf{G}_2]| \leq \frac{2q_{H_3} + 2q_D}{\sqrt{2^\kappa}}. \quad (37)$$

Game G₃: This game is the same as **G₂**, except that the simulation of F is replaced with the following operation: For the (x_1, x_2) , return $F_0(x_1, x_2)$ if $x_2 \neq H_1(pk)$. Otherwise, return $H(x_1)$, where H is a new random oracle with input space $\mathcal{M} = \{0, 1\}^\kappa$ and output space $\mathcal{R} = \{0, 1\}^r$.

Since $H_1(pk)$ is a fixed value after H_1 and pk are sampled, the game hop **G₂** to **G₃** is only a conceptual change, thus we have

$$\Pr[1 \leftarrow \mathbf{G}_2] = \Pr[1 \leftarrow \mathbf{G}_3]. \quad (38)$$

Game G₄: This game is the same as **G₃**, except that the H used to answer $F(x_1, H_1(pk))$ is replaced with H_{good} . Recall that $H_{\text{good}} : \mathcal{M} \rightarrow \mathcal{R}$ is a random function such that $H_{\text{good}}(m)$ is uniformly sampled from $\mathcal{R}_{\text{good}}(pk, sk, m)$ for any $m \in \mathcal{M}$.

For a fixed (pk, sk) sampled by Aigis-PKE⁺.KeyGen, we construct the following algorithm $\mathcal{B}^H(pk, sk)$ with quantum oracle access to H to play game **G₃**:

1. $\mathcal{B}^H(pk, sk)$ runs game **G₃** under the fixed (pk, sk) pair, replaces the calls to H by calls to its own oracle and simulates the decapsulation oracle by using its input (pk, sk) . $\mathcal{B}^H(pk, sk)$ finally returns the output of game **G₃**.

Since in game **G₃** the random oracle F is queried at most q_{H_2} times by $\mathcal{A}_1$ and q_D times by the decapsulation oracle, $\mathcal{B}^H(pk, sk)$ makes at most $2q_{H_2} + q_D$ queries to its oracle H and

$$\Pr[1 \leftarrow \mathbf{G}_3 : (pk, sk)] = \Pr\left[1 \leftarrow \mathcal{B}^H(pk, sk) : H \xleftarrow{\$} \text{Fcs}[\kappa, r]\right]. \quad (39)$$

Since game **G₃** is the same as **G₄** except that H is changed into H_{good} , we have

$$\Pr[1 \leftarrow \mathbf{G}_4 : (pk, sk)] = \Pr[1 \leftarrow \mathcal{B}^{H_{\text{good}}}(pk, sk)]. \quad (40)$$

Here $\Pr[1 \leftarrow \mathbf{G} : (pk, sk)]$ denotes the probability of game **G** outputs 1 under the fixed (pk, sk) pair.

Let $N_1 : \mathcal{M} \rightarrow \{0, 1\}$ be a random function such that $\Pr[N_1(m) = 1] = \delta(pk, sk, m)$ and $\Pr[N_1(m) = 0] = 1 - \delta(pk, sk, m)$ for any $m \in \mathcal{M}$. Here the value $\delta(pk, sk, m)$ is defined in Eq. (34). Let $N_2 : \mathcal{M} \rightarrow \{0, 1\}$ be a constant function such that $N_2(m) = 0$ for any $m \in \mathcal{M}$. Then, by running $\mathcal{B}^H(pk, sk)$, we construct

$\mathcal{C}^N(pk, sk)$	$K(m)$
1 : $H \xleftarrow{\$} \text{Fcs}[\kappa, r]$	1 : if $N(m) = 0$
2 : $H := K$	2 : return $r \xleftarrow{\$} \mathcal{R}_{\text{good}}(pk, sk, m)$
3 : $b \leftarrow \mathcal{B}^H(pk, sk)$	3 : if $N(m) = 1$
4 : return b	4 : return $r \xleftarrow{\$} \mathcal{R}_{\text{bad}}(pk, sk, m)$

Fig. 13: Quantum oracle algorithm $\mathcal{C}^N(pk, sk)$ for the proof of Theorem 3.

a quantum oracle algorithm $\mathcal{C}^N(pk, sk)$ ($N \in \{N_1, N_2\}$) as shown in Fig. 13. Here $\mathcal{C}^N(pk, sk)$ introduces a new quantum oracle K relates to its quantum oracle N and replaces H into K . Notice that $\mathcal{C}^N(pk, sk)$ could answer one quantum query to K by (quantumly) querying N twice, and hence $\mathcal{C}^N(pk, sk)$ makes $4q_{H_2} + 2q_D$ quantum queries to N . Since $H_{\text{good}} : \mathcal{M} \rightarrow \mathcal{R}$ is a random function such that $H_{\text{good}}(m)$ is uniformly sampled from $\mathcal{R}_{\text{good}}(pk, sk, m)$ for any $m \in \mathcal{M}$, we have

$$\begin{aligned} \Pr[1 \leftarrow \mathcal{C}^{N_1}(pk, sk)] &= \Pr[1 \leftarrow \mathcal{B}^H(pk, sk) : H \xleftarrow{\$} \text{Fcs}[\kappa, r]], \\ \Pr[1 \leftarrow \mathcal{C}^{N_2}(pk, sk)] &= \Pr[1 \leftarrow \mathcal{B}^{H_{\text{good}}}(pk, sk)]. \end{aligned}$$

Then, by Lemma 5 and the fact that $\mathcal{C}^N(pk, sk)$ ($N \in \{N_1, N_2\}$) makes $4q_{H_2} + 2q_D$ quantum queries to N , we obtain

$$\begin{aligned} & \left| \Pr[1 \leftarrow \mathcal{B}^H(pk, sk) : H \xleftarrow{\$} \text{Fcs}[\kappa, r]] - \Pr[1 \leftarrow \mathcal{B}^{H_{\text{good}}}(pk, sk)] \right| \\ &= \left| \Pr[1 \leftarrow \mathcal{C}^{N_1}(pk, sk)] - \Pr[1 \leftarrow \mathcal{C}^{N_2}(pk, sk)] \right| \\ &\leq 8(4q_{H_2} + 2q_D + 1)^2 \cdot \max_{m \in \mathcal{M}} \delta(pk, sk, m) \\ &\stackrel{(a)}{=} 8(4q_{H_2} + 2q_D + 1)^2 \cdot \delta(pk, sk). \end{aligned}$$

Here (a) follows from the definition of $\delta(pk, sk)$ shown in Eq. (34). Combining the above equation with Eq. (39) and Eq. (40), we get

$$|\Pr[1 \leftarrow \mathbf{G}_3 : (pk, sk)] - \Pr[1 \leftarrow \mathbf{G}_4 : (pk, sk)]| \leq 8(4q_{H_2} + 2q_D + 1)^2 \cdot \delta(pk, sk).$$

By averaging over $(pk, sk) \leftarrow \text{Aigis-PKE}^+.\text{KeyGen}$, we finally obtain

$$|\Pr[1 \leftarrow \mathbf{G}_3] - \Pr[1 \leftarrow \mathbf{G}_4]| \leq 8(4q_{H_2} + 2q_D + 1)^2 \cdot \delta. \quad (41)$$

Game $\mathbf{G}_5$: This game is the same as $\mathbf{G}_2$, except that the simulation of G is replaced with the following operation: For the (x_1, x_2) , compute $\rho = F(x_1, H_1(pk))$ and $c = \text{Aigis-PKE}^+.\text{Enc}(pk, x_1; \rho)$, and then return $G_0(x_1, x_2)$ if $x_2 \neq H_1(pk)$ or $c = c^*$. Otherwise, return $G_1(c)$, where G_1 is a new random oracle with input space $\{0, 1\}^n$ and output space $\{0, 1\}^{256}$.

Based on the above definition, games $\mathbf{G}_4$ and $\mathbf{G}_5$ are equivalent if the following event does not occur:

$$\exists x_1, x_2 \in \mathcal{M} \text{ such that } x_1 \neq x_2 \text{ and} \\ \text{Aigis-PKE}^+.\text{Enc}(pk, x_1; F(x_1, H_1(pk))) = \text{Aigis-PKE}^+.\text{Enc}(pk, x_2; F(x_2, H_1(pk))).$$

Notice that $F(x, H_1(pk))$ is answered by $H_{\text{good}}(x)$ in game $\mathbf{G}_4$ and $\mathbf{G}_5$, where $H_{\text{good}}(x)$ is uniformly sampled from $\mathcal{R}_{\text{good}}(pk, sk, x)$ for any $x \in \mathcal{M}$. Hence, by the definition of $\mathcal{R}_{\text{good}}(pk, sk, x)$ given in Eq. (33), we can conclude that

$$g(\cdot) := \text{Aigis-PKE}^+.\text{Enc}(pk, \cdot; F(\cdot, H_1(pk))) = \text{Aigis-PKE}^+.\text{Enc}(pk, \cdot; H_{\text{good}}(\cdot))$$

is an injective function, and thus the above event can not occur. Therefore,

$$\Pr[1 \leftarrow \mathbf{G}_4] = \Pr[1 \leftarrow \mathbf{G}_5]. \quad (42)$$

Game $\mathbf{G}_6$: This game is the same as game $\mathbf{G}_5$, except that the decapsulation oracle $\text{Aigis-Enc}^+.\text{Decap}$ is changed as it directly returns $K = G_1(c)$ for $c \neq c^*$. That is, the answer of decapsulation oracle in this game no longer uses the secret key sk .

In game $\mathbf{G}_5$, since $F(x, H_1(pk))$ is answered by $H_{\text{good}}(x)$ and $H_{\text{good}}(x) \in \mathcal{R}_{\text{good}}(pk, sk, x)$ for any $x \in \mathcal{M}$, by querying G the adversary $\mathcal{A}_1$ can only get $G_1(c)$ such that $\text{Aigis-PKE}^+.\text{Dec}(c) \neq \perp$ and

$$c = \text{Aigis-PKE}^+.\text{Enc}(pk, M; F(M, H_1(pk))),$$

where $M = \text{Aigis-PKE}^+.\text{Dec}(c)$. Hence, in game $\mathbf{G}_5$, the $H_4(c)$ returned by the $\text{Aigis-Enc}^+.\text{Decap}(sk, c)$ under the condition $\text{Aigis-PKE}^+.\text{Dec}(c) = \perp$, or the condition $M = \text{Aigis-PKE}^+.\text{Dec}(c) \neq \perp \wedge c \neq \text{Aigis-PKE}^+.\text{Enc}(pk, M; F(M, H_1(pk)))$ is equivalent to $G_1(c)$ in $\mathcal{A}_1$'s view. This means that game $\mathbf{G}_5$ is equivalent to game $\mathbf{G}_6$, and thus

$$\Pr[1 \leftarrow \mathbf{G}_5] = \Pr[1 \leftarrow \mathbf{G}_6]. \quad (43)$$

Game $\mathbf{G}_7$: In game $\mathbf{G}_4$ to $\mathbf{G}_6$, we note that H has replaced with H_{good} , in this new game, we remove this replacement. That is, the H_{good} in this new game is uniformly sampled from $\text{Fcs}[\kappa, r]$.

With an analysis similar to that for the game hop $\mathbf{G}_3$ and $\mathbf{G}_4$, we can first convert the distinguishing problem between games $\mathbf{G}_6$ and $\mathbf{G}_7$ to the distinguishing problem between H_{good} and H , and then use the Lemma 5 to upper bound the corresponding distinguishing advantage. But one thing to note is that the simulations of G in games $\mathbf{G}_6$ and $\mathbf{G}_7$ also need to query F , which implies that the algorithm $\mathcal{C}$ in Fig. 13 needs to query N extra $4q_{H_2}$ times. Overall, we have

$$|\Pr[1 \leftarrow \mathbf{G}_6] - \Pr[1 \leftarrow \mathbf{G}_7]| \leq 8(8q_{H_2} + 2q_D + 1)^2 \cdot \delta. \quad (44)$$

Game $\mathbf{G}_8$: This game is the same as $\mathbf{G}_7$, except that the simulation of F directly uses F_0 , i.e., for a query (x_1, x_2) , the oracle F in this game returns $F_0(x_1, x_2)$. Similar to the game hop $\mathbf{G}_2$ to $\mathbf{G}_3$, this is a conceptual change, thus we have

$$\Pr[1 \leftarrow \mathbf{G}_7] = \Pr[1 \leftarrow \mathbf{G}_8]. \quad (45)$$

Game $\mathbf{G}_9$: This game is the same as $\mathbf{G}_8$, except that just before each query to the oracles F and G , the semi-classical oracle $\mathcal{O}_S^{SC}$ is queried on the input register of the oracles, where $S := \{(M^*, H_1(pk))\}$.

In games $\mathbf{G}_8$ and $\mathbf{G}_9$, the oracles F and G are queried at most $3q_{H_2} + 2q_D$ times in total, and thus the $\mathcal{O}_S^{SC}$ is queried at most $3q_{H_2} + 2q_D$ times in game $\mathbf{G}_9$. Now, by using Lemma 2, we have

$$|\Pr[1 \leftarrow \mathbf{G}_8] - \Pr[1 \leftarrow \mathbf{G}_9]| \leq \sqrt{(3q_{H_2} + 2q_D + 1) \cdot \Pr[\text{Find} : \mathbf{G}_9]}. \quad (46)$$

Here Find is the classical event that the measurement of $\mathcal{O}_S^{SC}$ ever has outcome 1.

Game $\mathbf{G}_{10}$: This game is the same as $\mathbf{G}_9$, except that the K_0^* and ρ^* are both replaced with fresh random values.

On the condition that Find does not occur, i.e., the measurement performed by $\mathcal{O}_S^{SC}$ always has outcome 0, $F(M^*, H_1(pk))$ and $G(M^*, H_1(pk))$ are hidden from the adversary $\mathcal{A}$'s view, which further means that these two values are uniformly random in $\mathcal{A}_1$'s view. Hence, we have

$$\Pr[\text{Find} : \mathbf{G}_9] = \Pr[\text{Find} : \mathbf{G}_{10}]. \quad (47)$$

and

$$|\Pr[1 \leftarrow \mathbf{G}_9] - \Pr[1 \leftarrow \mathbf{G}_{10}]| \leq \Pr[\text{Find} : \mathbf{G}_{10}] \stackrel{(a)}{=} \Pr[1 \leftarrow \mathbf{G}_{10}^a]. \quad (48)$$

Here (a) follows from the definition of game $\mathbf{G}_{10}^a$ given in Fig. 14, which returns 1 if and only if Find occurs in game $\mathbf{G}_{10}$.

Game $\mathbf{G}_{10}^a$	Game $\mathbf{G}_{10}^b$
1. $(pk, sk) \leftarrow \text{Aigis-PKE}^+.\text{KeyGen}$ $H_1 \xleftarrow{\$} \text{Fcs}[*], 256], s \xleftarrow{\$} \{0, 1\}^\kappa$ $sk = (sk, pk, H_1(pk), s)$ $G_1 \xleftarrow{\$} \text{Fcs}[n, 256], F_0 \xleftarrow{\$} \text{Fcs}[\kappa + 256, r]$ $G_0 \xleftarrow{\$} \text{Fcs}[\kappa + 256, 256]$	1. $(pk, sk) \leftarrow \text{Aigis-PKE}^+.\text{KeyGen}$ $H_1 \xleftarrow{\$} \text{Fcs}[*], 256], s \xleftarrow{\$} \{0, 1\}^\kappa$ $sk = (sk, pk, H_1(pk), s)$ $G_1 \xleftarrow{\$} \text{Fcs}[n, 256], F_0 \xleftarrow{\$} \text{Fcs}[\kappa + 256, r]$ $G_0 \xleftarrow{\$} \text{Fcs}[\kappa + 256, 256]$
2. $b \xleftarrow{\$} \{0, 1\}, M^* \xleftarrow{\$} \{0, 1\}^\kappa$ $\rho^* \xleftarrow{\$} \{0, 1\}^r$ $c^* = \text{Aigis-PKE}^+.\text{Enc}(pk, M^*; \rho^*)$ $K_0^* \xleftarrow{\$} \{0, 1\}^{256}, K_1^* \xleftarrow{\$} \{0, 1\}^{256}$	2. $b \xleftarrow{\$} \{0, 1\}, M^* \xleftarrow{\$} \{0, 1\}^\kappa, M' \xleftarrow{\$} \{0, 1\}^\kappa$ $\rho^* \xleftarrow{\$} \{0, 1\}^r$ $c^* = \text{Aigis-PKE}^+.\text{Enc}(pk, M'; \rho^*)$ $K_0^* \xleftarrow{\$} \{0, 1\}^{256}, K_1^* \xleftarrow{\$} \{0, 1\}^{256}$
3. run $\mathcal{A}_1^{H_1, F, G, H_3, \text{Aigis-Enc}^+}.\text{Decap}(pk, c^*, K_b^*)$	3. run $\mathcal{A}_1^{H_1, F, G, H_3, \text{Aigis-Enc}^+}.\text{Decap}(pk, c^*, K_b^*)$
4. return [Find]	4. return [Find]

Fig. 14: Games $\mathbf{G}_{10}^a$ and $\mathbf{G}_{10}^b$ in the proof of Theorem 3. In these two games, the oracles queried by $\mathcal{A}_1$ are simulated in the same way as in game $\mathbf{G}_{10}$ of Fig. 12.

Game $\mathbf{G}_{11}$: This game is the same as $\mathbf{G}_{10}$, except that it no longer queries the semi-classical oracle $\mathcal{O}_S^{SC}$.

Similar to the game hop $\mathbf{G}_8$ to $\mathbf{G}_9$, we have

$$|\Pr[1 \leftarrow \mathbf{G}_{10}] - \Pr[1 \leftarrow \mathbf{G}_{11}]| \leq \sqrt{(3q_{H_2} + 2q_D + 1) \cdot \Pr[\text{Find} : \mathbf{G}_{10}]}. \quad (49)$$

In addition, since the bit b is independent of $\mathcal{A}_1$'s view in game $\mathbf{G}_{11}$, we have

$$\Pr[1 \leftarrow \mathbf{G}_{11}] = \frac{1}{2}. \quad (50)$$

In Fig. 14, we also define a game $\mathbf{G}_{10}^b$, which is the same as $\mathbf{G}_{10}^a$ except that the challenge ciphertext c^* equals to $\text{Aigis-PKE}^+.\text{Enc}(pk, M'; \rho^*)$, where M' is new challenge plaintext uniformly sampled from $\mathcal{M}$. At this point, we define a new adversary $\mathcal{B}$ against the IND-CPA security of Aigis-PKE^+ as follows:

- $\mathcal{B}$ samples $M^*, M' \leftarrow \{0, 1\}^\kappa$ as the challenge plaintexts and send them to the IND-CPA challenger. After the challenger replies the challenge ciphertext c^* , $\mathcal{B}$ runs $\mathcal{A}_1^{H_1, F, G, H_3, \text{Aigis-Enc}^+.\text{Decap}}(pk, c^*, K_b^*)$ in game $\mathbf{G}_{10}^a$ (or game $\mathbf{G}_{10}^b$ equivalently) and simulates the random oracles by using the $2q$ -wise independent functions [7]. $\mathcal{B}$ finally returns the final output of this game, i.e. $\llbracket \text{Find} \rrbracket$.

The running time of $\mathcal{B}$ is about that of $\mathcal{A}$ and $\mathcal{A}_1$, and

$$2 \cdot \text{Adv}_{\text{Aigis-PKE}^+}^{\text{IND-CPA}}(\mathcal{B}) = |\Pr[1 \leftarrow \mathbf{G}_{10}^a] - \Pr[1 \leftarrow \mathbf{G}_{10}^b]|. \quad (51)$$

For the game $\mathbf{G}_{10}^b$, since M^* is independent of $\mathcal{A}_1$'s view and F (resp. G) is queried at most $2q_{H_2} + q_D$ (resp. $q_{H_2} + q_D$) times, by using Lemma 3 we have

$$\Pr[1 \leftarrow \mathbf{G}_{10}^b] \leq \frac{4(3q_{H_2} + 2q_D)}{2^\kappa}. \quad (52)$$

Combining Eq. (35) to Eq. (38) and Eq. (41) to Eq. (52), we have

$$\begin{aligned} \text{Adv}_{\text{Aigis-Enc}^+}^{\text{IND-CCA}}(\mathcal{A}) &\leq 4\sqrt{(6q_{H_2} + 4q_D + 2) \cdot \text{Adv}_{\text{Aigis-PKE}^+}^{\text{IND-CPA}}(\mathcal{B})} \\ &\quad + \frac{12q_{H_2} + 2q_{H_3} + 10q_D + 4}{\sqrt{2^\kappa}} + 16(8q_{H_2} + 2q_D + 4)^2 \cdot \delta \\ &\stackrel{(a)}{\leq} 4\sqrt{(6q_{H_2} + 4q_D + 2) \cdot \text{Adv}_{n,q,\eta_1,\eta_2}^{\text{ARLWE}}(\mathcal{B}_1)} \\ &\quad + 4\sqrt{(6q_{H_2} + 4q_D + 2) \cdot \text{Adv}_{n,q,2^{d_t},\eta_1,\eta_2}^{\text{semi-ARLWE}}(\mathcal{B}_2)} \\ &\quad + \frac{12q_{H_2} + 2q_{H_3} + 10q_D + 4}{\sqrt{2^\kappa}} + 16(8q_{H_2} + 2q_D + 1)^2 \cdot \delta. \end{aligned}$$

Here (a) follows from Theorem 1 and the running times of $\mathcal{B}_1$ and $\mathcal{B}_2$ are roughly the same as those of $\mathcal{A}$. This completes the proof of Theorem 3. $\square$